AWS Consolidated Notes

Generated on: 2026-03-31 23:21:03 Source folder: /Users/malapalli_aravind@optum.com/Desktop/maravin4/SD/Sources/AWS Total source files: 9

Included Documents

1. AWS-01-Compute-Networking.md - AWS Interview Notes - Part 1: Compute + Networking
2. AWS-02-Storage-Database.md - AWS Interview Notes - Part 2: Storage + Database
3. AWS-03-Security-Messaging-Integration.md - AWS Interview Notes - Part 3: Security + Messaging + Integration
4. AWS-04-DevOps-Monitoring-Architecture-Interview-QA.md - AWS Interview Notes - Part 4: DevOps + Monitoring + Architecture + Interview Q&A
5. AWS-05-EC2-ECS-EKS-Story-and-Deployment-Guide.md - AWS Deep Dive: EC2, ECS, EKS Through Story Mode + Spring Boot/React Deployment Journey
6. AWS-06-Networking-Story-Mode.md - AWS Networking Through Story Mode: How Your App Actually Talks
7. AWS-07-Storage-Story-Mode.md - AWS Storage Through Story Mode: Where Your App's Data Actually Lives
8. AWS-08-Security-Story-Mode.md - AWS Security Through Story Mode: Who Can Do What and How Nothing Leaks
9. AWS-09-Messaging-Integration-Observability-Story-Mode.md - AWS Messaging, Integration, and Observability Through Story Mode

Organization Notes

- Original source files were preserved unchanged.
- Duplicate per-file table-of-contents sections were removed in this aggregated copy.
- Each source document is grouped under its own part heading below.

Part 01: AWS Interview Notes - Part 1: Compute + Networking

Source file: AWS-01-Compute-Networking.md

> Covers: EC2, ECS, EKS, Lambda, VPC, ELB, Route 53, CloudFront, API Gateway. Written for backend/platform interviews where the interviewer expects trade-offs, production reasoning, and architecture clarity rather than surface-level definitions.

---

1. How to Think About AWS Compute

Interviewers rarely care that you can list services. They care whether you can answer:

- Where should this workload run?
- How does traffic reach it?
- How does it scale?
- What fails if one AZ goes down?
- How do you secure east-west and north-south traffic?
- Why did you choose container orchestration vs serverless vs virtual machines?

The clean mental model:

```
| Need | AWS choice |
| Full OS control, custom runtime, legacy app | EC2 |
```

```
| Containers without managing Kubernetes | ECS |
| Containers with K8s portability/ecosystem | EKS |
| Event-driven short-lived functions | Lambda |
| Network isolation and routing | VPC |
| Traffic distribution | ELB |
| DNS and routing policies | Route 53 |
| Edge caching and CDN | CloudFront |
| Managed API front door | API Gateway |
```

The senior-level answer is always a trade-off answer, not a single-service answer.

---

2. EC2

2.1 What It Is

Amazon EC2 is a virtual machine service. You get compute instances with CPU, memory, network, and attached storage. You manage the OS, patching, runtime, scaling logic, and usually the application lifecycle.

2.2 When to Use It

Use EC2 when:

- you need full machine control
- you have legacy apps not container-ready
- you need specific agents, kernels, drivers, or custom networking behavior
- you want predictable long-running servers
- you are running self-managed databases, build agents, or stateful middleware

Avoid defaulting to EC2 when the team actually wants less operational overhead.

2.3 Instance Types

Common families:

- t series: burstable, cheap, dev/test, low steady load
- m series: general purpose
- c series: compute optimized
- r series: memory optimized
- i series: storage optimized
- g/p series: GPU

Interview angle:

- Do not pick t instances for sustained CPU-heavy production services unless you understand CPU credits.
- Right-size based on workload profile, not habit.

2.4 Pricing Models

- On-Demand: flexible, highest price
- Reserved Instances / Savings Plans: commit usage, cheaper
- Spot: very cheap, can be interrupted
- Dedicated Hosts/Instances: compliance or licensing use cases

Good answer:

```
Stateless background workers can use Spot with interruption handling.
Critical low-latency APIs usually run On-Demand or Savings Plans-backed capacity.
```

2.5 Auto Scaling

EC2 Auto Scaling Groups provide:

- desired capacity
- min/max capacity

- health checks
- scaling policies
- multi-AZ distribution

Typical triggers:

- CPU
- memory via CloudWatch custom metrics
- request count per target
- queue depth

2.6 Placement and Resilience

- Spread across multiple AZs
- Use launch templates
- Put instances behind a load balancer
- Bake AMIs or use bootstrapping carefully

2.7 Interviewer Hot Points

- Difference between instance store and EBS
- Why private subnets for app servers
- How patching is handled
- Why immutable deployments are better than in-place changes
- How to survive instance loss

2.8 Common Mistakes

- Running a single EC2 instance in one AZ for production
- SSH-ing into servers and changing them manually
- Storing important data on ephemeral instance store
- Using security groups too broadly

---

3. ECS

3.1 What It Is

Amazon ECS is a managed container orchestration service. It runs containers without requiring you to manage Kubernetes control-plane complexity.

Two launch types:

- EC2 launch type: you manage the EC2 nodes
- Fargate launch type: serverless containers, no node management

3.2 Core Concepts

- Cluster: logical grouping of capacity
- Task Definition: container spec, CPU/memory, ports, env vars, IAM role
- Task: running instance of a task definition
- Service: ensures desired number of tasks stays running

3.3 Why Teams Pick ECS

- simpler than EKS
- tightly integrated with AWS IAM, ALB, CloudWatch
- good for teams that want containers without Kubernetes expertise

3.4 ECS with Fargate

Best when:

- you want less ops
- workloads are stateless

- cluster management is not core competency
- startup time and cost model are acceptable

Trade-off:

- less low-level control than EC2-backed containers
- sometimes more expensive at scale than optimized EC2 fleets

3.5 ECS Interview Talking Points

- ECS vs EKS
- Fargate vs EC2 launch type
- task role vs instance role
- service auto scaling
- ALB path-based routing to ECS services

3.6 Strong Answer Example

```
For a Java microservice platform on AWS where we want containers but not Kubernetes
operational overhead, ECS on Fargate is often the pragmatic default. We get service
discovery, ALB integration, IAM task roles, and autoscaling with a lower control-plane
burden.
```

---

4. EKS

4.1 What It Is

Amazon EKS is AWS-managed Kubernetes. AWS manages the Kubernetes control plane; you still manage worker nodes or use Fargate for pods.

4.2 When to Use It

Choose EKS when:

- the org is standardized on Kubernetes
- you need K8s APIs, CRDs, operators, Helm, service mesh, or ecosystem tools
- portability matters
- platform engineering already has K8s maturity

Do not choose EKS just because "Kubernetes is popular."

4.3 Operational Reality

Even though EKS is managed, you still own a lot:

- worker nodes or Fargate profiles
- cluster add-ons
- networking model
- ingress
- observability
- cost control
- pod security
- upgrade strategy

4.4 Node Groups

- Managed Node Groups: AWS-managed lifecycle for worker groups
- Self-managed nodes: more control, more operational burden
- Fargate on EKS: serverless pods for some workloads

4.5 EKS Networking

EKS uses VPC networking. Pods get VPC IPs through the AWS VPC CNI.

Implications:

- IP planning matters
- subnet exhaustion is a real problem
- security groups and network design need thought

4.6 What Interviewers Probe

- Why EKS over ECS?
- How ingress works
- How secrets/config are handled
- How rolling updates and autoscaling work
- How you secure the cluster
- How you observe and debug workloads

4.7 Senior-Level Trade-off

```
EKS gives maximum flexibility and ecosystem depth, but it is not lower effort than ECS. It
 is the right answer when Kubernetes itself is a requirement, not when containers alone
are the requirement.
```

---

5. Lambda

5.1 What It Is

AWS Lambda is serverless compute. You upload code or container images, and AWS runs functions in response to events.

5.2 Best Use Cases

- API backends with moderate latency sensitivity
- event-driven processing
- file processing
- cron jobs
- queue consumers
- integration glue logic

5.3 When Lambda Is a Bad Fit

- very long-running workloads
- low-latency workloads sensitive to cold starts
- heavy stateful processing
- applications needing stable connections or specialized system-level control

5.4 Key Concepts

- Invocation types: synchronous, asynchronous, poll-based
- Concurrency: number of parallel executions
- Reserved concurrency: hard limit/protection
- Provisioned concurrency: reduce cold-start impact
- Timeout and memory settings affect performance

5.5 Cold Starts

Cold starts matter more for:

- JVM functions
- VPC-attached Lambdas if badly configured
- low-latency APIs

Mitigations:

- provisioned concurrency
- lighter runtime/package

- reduce initialization cost

5.6 Lambda and VPC

Lambda can run inside a VPC to access private resources, but that adds networking considerations and historically increased cold-start sensitivity.

5.7 Interviewer Favorites

- Lambda vs ECS/Fargate
- idempotency for async processing
- retries and DLQs
- limits and timeouts
- handling fan-out events

---

6. VPC

6.1 What It Is

Amazon VPC is your logically isolated network in AWS. It defines IP ranges, subnets, routing, gateways, and traffic controls.

Think of it as the network boundary inside which your AWS resources communicate.

6.2 Core Building Blocks

- VPC CIDR
- Subnets
- Route tables
- Internet Gateway
- NAT Gateway
- Security Groups
- Network ACLs
- VPC Endpoints

6.3 Public vs Private Subnets

Public subnet:

- route to Internet Gateway
- used for public-facing ALBs, bastions if unavoidable

Private subnet:

- no direct internet route
- used for app servers, ECS tasks, EKS nodes, RDS

Strong pattern:

```
Internet -> ALB in public subnets -> app tier in private subnets -> DB in private subnets
```

6.4 Security Groups vs NACLs

Security Groups:

- stateful
- attached to ENIs/resources
- allow rules only
- primary control in most architectures

NACLs:

- stateless
- attached at subnet level
- allow and deny rules
- lower-level coarse control

Interview answer:

```
Use security groups for resource-level traffic control.
Use NACLs sparingly when subnet-level deny logic is needed.
```

6.5 NAT Gateway

Purpose:

- allows private-subnet resources to access the internet outbound
- blocks inbound internet-initiated traffic

Example:

- private EC2 instance downloads patches through NAT Gateway

6.6 VPC Endpoints

Avoid public internet path for AWS services.

Two broad types:

- Gateway Endpoint: S3, DynamoDB
- Interface Endpoint: powered by PrivateLink for many AWS services

Great answer:

```
For private workloads talking to S3 or Secrets Manager, I prefer VPC endpoints to reduce
exposure and avoid unnecessary NAT traffic.
```

6.7 Multi-AZ Network Design

- use at least two AZs
- create public/private subnets in each AZ
- route traffic locally where possible
- remember NAT Gateway is AZ-scoped for resilience design

6.8 VPC Interview Traps

- thinking private subnet means no outbound internet ever
- confusing security groups with NACLs
- ignoring CIDR planning
- putting databases in public subnets

---

7. Elastic Load Balancing

AWS offers multiple load balancers.

7.1 Application Load Balancer (ALB)

Layer 7 HTTP/HTTPS load balancer.

Use it for:

- host-based routing
- path-based routing
- HTTP header rules
- web apps, APIs, microservices
- ECS/EKS ingress patterns

7.2 Network Load Balancer (NLB)

Layer 4 TCP/UDP load balancer.

Use it for:

- very high performance
- static IP needs
- non-HTTP protocols

- TLS passthrough style use cases

7.3 Gateway Load Balancer

Used for inserting virtual appliances such as network firewalls.

Less commonly expected in general interviews unless the role is network-heavy.

7.4 Health Checks

Critical concept:

- load balancer only routes to healthy targets
- your health endpoint should reflect readiness, not just process alive

7.5 Interviewer Comparison

```
| Service | Layer | Best for |
| ALB | L7 | HTTP APIs, path/host routing |
| NLB | L4 | TCP/UDP, extreme performance, static IP |
| GWLB | specialized | network appliances |
```

7.6 Common Mistakes

- using NLB when advanced HTTP routing is required
- weak health checks
- single-AZ target groups

---

8. Route 53

8.1 What It Is

Route 53 is AWS DNS and traffic-routing service.

It provides:

- domain registration
- hosted zones
- DNS records
- health checks
- routing policies

8.2 Important Routing Policies

- Simple: one resource
- Weighted: split traffic by percentage
- Latency-based: route to lowest-latency region
- Failover: primary/secondary based on health
- Geolocation/Geoproximity: route by user geography

8.3 Interview Use Cases

- blue-green cutover with weighted records
- DR with failover routing
- multi-region active-active with latency routing

8.4 What Senior Candidates Mention

- DNS is not instant because of TTL
- Route 53 helps steer traffic but does not make an app multi-region by itself

---

9. CloudFront

9.1 What It Is

CloudFront is AWS CDN. It caches content at edge locations close to users.

9.2 Best Use Cases

- static asset delivery
- global APIs
- media/content distribution
- caching public or semi-public responses
- fronting S3 or ALB origins

9.3 Key Concepts

- Edge locations
- Origin
- Cache behavior
- TTLs
- Signed URLs / signed cookies
- Origin Access Control for S3

9.4 Strong Interview Points

- CloudFront reduces latency and origin load
- Dynamic content can still benefit through TLS termination and edge routing
- Cache invalidation is expensive relative to versioned asset strategy

9.5 Security Angle

- keep S3 private and let only CloudFront access it
- use WAF with CloudFront for edge protection

---

10. API Gateway

10.1 What It Is

Amazon API Gateway is a managed service for publishing APIs at scale. It handles request intake, routing, throttling, auth integration, and observability.

10.2 API Gateway Works Well For

- serverless APIs backed by Lambda
- lightweight service front doors
- request validation and throttling
- partner APIs

10.3 Common Features

- authentication/authorization integration
- request/response transformation
- usage plans and API keys
- throttling
- stage variables
- custom domains

10.4 REST vs HTTP API

Broad interview-safe answer:

- HTTP APIs are simpler and cheaper for many common use cases
- REST APIs have broader feature depth in some scenarios

10.5 API Gateway vs ALB

Use API Gateway when:

- you need managed API concerns such as throttling, usage plans, authorizers, request validation
- Lambda is the backend

Use ALB when:

- you are routing to containers or instances over HTTP
- you want simpler, cheaper L7 load balancing without full API-management features

---

11. High-Value Comparisons

11.1 EC2 vs ECS vs EKS vs Lambda

```
| Service | You manage | Best when |
| EC2 | OS, runtime, scaling, patching | full control or legacy apps |
| ECS | app containers, some cluster choices | AWS-native containers with low ops |
| EKS | K8s workloads and platform ops | Kubernetes ecosystem required |
| Lambda | function code only | event-driven or bursty serverless |
```

11.2 ALB vs API Gateway

```
| Need | Better fit |
| Container/instance HTTP routing | ALB |
| managed API features and serverless front door | API Gateway |
```

11.3 ECS vs EKS

```
| Question | ECS | EKS |
| Operational simplicity | better | worse |
| Kubernetes portability | low | high |
| AWS-native ease | high | medium |
| ecosystem flexibility | medium | high |
```

---

12. Architecture Walkthroughs

12.1 Standard Web App

```
Users
  ->
Route 53
  ->
CloudFront
  ->
ALB
  ->
ECS / EKS / EC2 app tier in private subnets
  ->
RDS / ElastiCache / S3
```

Why it is strong:

- edge caching
- TLS termination
- multi-AZ resilience
- private app and data tiers

12.2 Serverless API

```
Client
  ->
Route 53
  ->
API Gateway
  ->
Lambda
  ->
DynamoDB / SQS / EventBridge
```

Why it is strong:

- no servers
- event-driven
- easy burst handling

12.3 Kubernetes Platform

```
Route 53
  ->
ALB Ingress
  ->
EKS services across multiple AZs
  ->
RDS, S3, internal services
```

Mention:

- IAM roles for service accounts
- autoscaling
- observability
- subnet/IP planning

---

13. Common Interview Traps

Trap 1

"We need containers, so EKS."

Correct thinking:

Containers do not automatically imply Kubernetes. ECS is often enough.

Trap 2

"Private subnet means no internet access."

Correct thinking:

Private subnet means no direct route from the internet. Outbound access can still happen via NAT Gateway or VPC endpoints.

Trap 3

"ALB and API Gateway are the same."

Correct thinking:

ALB is load balancing. API Gateway is managed API front-door functionality.

Trap 4

"Lambda is always cheaper."

Correct thinking:

Not for high-throughput steady-state workloads where containers or EC2 may be more cost-efficient.

Trap 5

"Multi-AZ means multi-region."

Correct thinking:

No. Multi-AZ improves regional resilience. Multi-region is a bigger architecture decision.

Trap 6

"VPC Peering is how we connect everything."

Correct thinking:

VPC Peering is point-to-point and doesn't scale. Transit Gateway is the hub-spoke model for many VPCs.

---

14. Global Accelerator

14.1 What It Is

AWS Global Accelerator provides static anycast IP addresses that route traffic to optimal AWS endpoints via the AWS global network backbone.

Unlike CloudFront which is a CDN (caches content), Global Accelerator is a network accelerator (routes packets faster).

14.2 When to Use Global Accelerator

```
Use Global Accelerator when:
   You need static IP addresses (whitelisting, compliance)
   Non-HTTP protocols: TCP, UDP, gRPC, gaming, IoT
   Low-latency global routing WITHOUT caching
   Instant regional failover (health-check based)
   Multi-region active-active with single entry point

Use CloudFront when:
   HTTP/HTTPS traffic
   Content needs caching (static assets, API responses)
   Web applications, media delivery
   WAF integration at edge
```

14.3 How It Works

```
                            2 Static Anycast IPs
                            (Global Accelerator)




  AWS Edge Location          AWS Edge Location           AWS Edge Location
   (closest to user)

          AWS Private Backbone (not public internet)



    Regional Endpoint Groups

     us-east-1        eu-west-1        ap-south-1
     ALB/NLB/EC2      ALB/NLB/EC2      ALB/NLB/EC2
     weight: 50%      weight: 30%      weight: 20%
```

14.4 Global Accelerator vs CloudFront

```
| Aspect | Global Accelerator | CloudFront |
| Layer | L4 (TCP/UDP) | L7 (HTTP/HTTPS) |
| Caching | No | Yes |
| Static IPs | Yes (2 anycast IPs) | No (DNS-based) |
| Protocol support | Any TCP/UDP | HTTP/HTTPS only |
| Use case | Low-latency routing, failover | Content delivery, caching |
| WAF | No | Yes |
| Price model | Fixed + data transfer | Request + data transfer |
```

14.5 Interview Gold Answer

```
"For a global gRPC service that needs static IPs and instant failover,
I'd use Global Accelerator. For a web app serving static assets globally,
I'd use CloudFront. They solve different problems - routing vs caching."
```

---

15. VPC Connectivity - Transit Gateway, Peering, PrivateLink

This is a high-value architect topic. Sushree (SA-certified) will likely probe here.

15.1 The Problem

How do you connect:

- Multiple VPCs in the same region?
- VPCs across regions?
- VPCs to on-premises data centers?
- Your services to other AWS accounts without exposing to internet?

15.2 VPC Peering

```
What it is:
  Direct network connection between two VPCs.
  Traffic stays on AWS backbone, never hits internet.

How it works:
  VPC-A <----peering----> VPC-B
  Both must accept the peering request.
  Route tables must be updated in both VPCs.

Limitations:
  [x] NOT transitive: AB and BC does NOT mean A can talk to C
  [x] CIDR ranges must not overlap
  [x] Doesn't scale well (n VPCs = n(n-1)/2 peerings)
  [x] Cross-region peering has data transfer costs

When to use:
  [ok] Simple two-VPC connectivity
  [ok] Low number of VPCs
  [ok] Direct, high-bandwidth communication needed
```

15.3 Transit Gateway

```
What it is:
  Regional network hub that connects VPCs and on-premises networks.
  Hub-and-spoke model - all VPCs connect to the TGW, not to each other.

How it works:
```

```
                    Transit Gateway



         VPC-A            VPC-B            VPC-C

                     VPN / Direct Connect

                       On-Premises
```

```
Benefits:
  [ok] Transitive routing (A can reach C through TGW)
  [ok] Scales to thousands of VPCs
  [ok] Centralized routing tables
  [ok] Supports VPN and Direct Connect attachments
  [ok] Cross-region peering via TGW peering

When to use:
  [ok] 10+ VPCs that need interconnection
  [ok] Hub-and-spoke topology
  [ok] Hybrid cloud (AWS + on-prem)
  [ok] Centralized egress/ingress through shared services VPC
```

15.4 AWS PrivateLink

```
What it is:
  Expose a service from your VPC to other VPCs/accounts
  WITHOUT VPC peering, WITHOUT public internet.
  Traffic stays entirely within AWS network.

How it works:

   Service Provider VPC (Account A)


       Your App    <- NLB required (or GWLB)
       (ECS/EC2)



      VPC Endpoint <- Endpoint Service
        Service



             PrivateLink (AWS backbone)


   Consumer VPC (Account B)


      Interface    <- Gets private IP in consumer VPC
      Endpoint



      Consumer
      Application
```

```
When to use:
  [ok] SaaS provider exposing service to customers' VPCs
  [ok] Shared services across accounts without full VPC connectivity
  [ok] Compliance: no data over internet, no CIDR overlap concerns
  [ok] Accessing AWS services privately (S3, DynamoDB, Secrets Manager)

Key points:
  -> Consumer gets a private IP (ENI) in their VPC
  -> No CIDR overlap issues
  -> Unidirectional: consumer initiates to provider
  -> Provider controls who can connect (allowlist)
```

15.5 Comparison Matrix

```
| Aspect | VPC Peering | Transit Gateway | PrivateLink |
| Topology | Point-to-point | Hub-and-spoke | Service-to-consumer |
| Transitivity | No | Yes | N/A (unidirectional) |
| Scale | Low (mesh explosion) | High (1000s of VPCs) | High |
| CIDR overlap | Not allowed | Not allowed | Allowed |
| Cross-account | Yes | Yes | Yes |
| Cross-region | Yes (with cost) | Yes (TGW peering) | No |
| On-prem connectivity | No | Yes (VPN/DX) | No |
| Use case | Simple 2-VPC | Enterprise multi-VPC | Expose service privately |
```

15.6 Interview Decision Tree

```
"How do I connect VPCs?"

  Is it just 2-3 VPCs directly communicating?
    -> VPC Peering

  Is it many VPCs that all need to talk to each other + on-prem?
    -> Transit Gateway

  Am I exposing a SERVICE to consumers without giving full network access?
    -> PrivateLink

  Do I need private access to AWS services like S3?
    -> VPC Endpoints (Gateway for S3/DynamoDB, Interface for others)
```

---

16. IRSA - IAM Roles for Service Accounts (EKS)

If Sushree asks about EKS + IAM, this is the key topic.

16.1 The Problem

In Kubernetes, pods need to call AWS APIs (S3, DynamoDB, Secrets Manager, etc.).

Old approach: Attach IAM role to EC2 worker node.

```
Problem with node-level IAM:
  Every pod on that node gets the same permissions.
  Pod A (needs S3) and Pod B (needs DynamoDB) both get S3 + DynamoDB.
  This violates least privilege.
```

16.2 IRSA Solution

IRSA = IAM Roles for Service Accounts

```
How it works:
  1. Create an IAM role with specific permissions
  2. Create a Kubernetes ServiceAccount
  3. Annotate the ServiceAccount with the IAM role ARN
```

```
    4. Pod uses that ServiceAccount
    5. AWS SDK in the pod automatically assumes the role via OIDC

    Pod -> K8s ServiceAccount -> OIDC -> IAM Role -> AWS Resources

  Result:
    [ok] Pod-level IAM permissions
    [ok] Least privilege per workload
    [ok] No credentials stored in pod
    [ok] Automatic credential rotation
```

16.3 Setup Flow

```
[yaml]
 # 1. Create IAM role with trust policy for EKS OIDC provider
 #    Trust policy allows the specific ServiceAccount to assume this role

 # 2. Create Kubernetes ServiceAccount with annotation
 apiVersion: v1
 kind: ServiceAccount
 metadata:
   name: my-app-sa
   namespace: production
   annotations:
     eks.amazonaws.com/role-arn: arn:aws:iam::123456789:role/my-app-role

 # 3. Use ServiceAccount in Pod/Deployment
 spec:
   serviceAccountName: my-app-sa
   containers:
     - name: my-app
       image: my-app:latest
       # AWS SDK automatically uses the IRSA credentials
```

16.4 Interview Gold Answer

```
"For EKS workloads accessing AWS services, I use IRSA to provide
pod-level IAM permissions. Each microservice gets its own ServiceAccount
annotated with a specific IAM role, following least privilege. The pod
gets temporary credentials via OIDC - no hardcoded keys, automatic rotation."
```

16.5 Alternative: EKS Pod Identity (newer)

```
AWS introduced EKS Pod Identity as a simpler alternative to IRSA.
- No OIDC provider setup needed
- Uses EKS Pod Identity Agent (DaemonSet)
- Simpler IAM trust policy

If asked: "I'm aware of EKS Pod Identity as the newer approach,
but IRSA is still widely used and I'm comfortable with both."
```

---

17. Cost Awareness (Sushree + Surendra will appreciate this)

Surendra has a FinOps certification. Cost awareness is a differentiator.

17.1 Cross-AZ Data Transfer

```
THE HIDDEN COST MOST DEVELOPERS MISS:

  Same AZ:     Free (within VPC)
```

```
  Cross-AZ:    $0.01/GB each direction ($0.02 round trip)
  Cross-Region: $0.02/GB+ (varies by region)
  Internet:    $0.09/GB out (first 10TB)

Example:
  Service A (AZ-a) -> Service B (AZ-b): 100GB/day
  Cost: 100GB  $0.02  30 days = $60/month just for one service pair

  10 services doing this = $600/month in hidden data transfer

Mitigation strategies:
  [ok] Co-locate chatty services in same AZ (but reduces HA)
  [ok] Use caching to reduce cross-AZ calls
  [ok] Compress payloads
  [ok] Batch requests instead of chatty calls
  [ok] Use VPC endpoints to avoid NAT Gateway data charges
```

17.2 NAT Gateway Costs

```
NAT Gateway pricing:
  $0.045/hour (~$32/month just to exist)
  + $0.045/GB processed

For high-traffic private workloads:
  1TB/month through NAT = $45 processing + $32 hourly = $77/month PER AZ

Cost-saving alternatives:
  [ok] VPC Endpoints for AWS services (S3, DynamoDB, Secrets Manager)
      -> No NAT Gateway needed for those calls
  [ok] NAT Instance (self-managed, cheaper, less resilient)
  [ok] Single NAT Gateway if HA is not critical (dev/test)
```

17.3 Load Balancer Costs

```
ALB: $0.0225/hour + LCU charges
NLB: $0.0225/hour + LCU charges (but different LCU calculation)

Hidden cost: too many ALBs
  10 ALBs = $162/month just in hourly charges

Solution:
  Consolidate with path-based routing on fewer ALBs
  Use K8s Ingress to share one ALB across services
```

17.4 Lambda vs Fargate vs EC2 Cost Crossover

```
Rule of thumb:

  Low traffic / bursty     -> Lambda cheapest
  Medium steady traffic    -> Fargate often cheaper
  High steady traffic      -> EC2 with Reserved/Savings Plans cheapest

Crossover point (rough):
  Lambda becomes expensive around 1M+ requests/day with moderate duration

Interview answer:
  "I'd start with Lambda for the event-driven workload, monitor costs,
   and consider moving to Fargate if we see sustained high utilization
   where Lambda's per-request pricing becomes expensive."
```

17.5 Interview Cost Question Pattern

```
Q: "How would you optimize costs for this architecture?"

Strong answer structure:
  1. Right-size compute (check utilization, downsize over-provisioned)
  2. Reserved capacity / Savings Plans for predictable workloads
  3. Spot for fault-tolerant workloads
  4. VPC Endpoints to avoid NAT charges
  5. Consolidate load balancers
  6. Review data transfer patterns (cross-AZ, cross-region)
  7. S3 lifecycle policies and intelligent tiering
  8. Turn off dev/test resources after hours
```

---

18. Rapid Revision Sheet

Services in One Line

- EC2: virtual machines with maximum control
- ECS: AWS-managed container orchestration
- EKS: AWS-managed Kubernetes
- Lambda: event-driven serverless compute
- VPC: isolated network boundary
- ALB: L7 HTTP/HTTPS load balancer
- NLB: L4 TCP/UDP load balancer
- Route 53: DNS and traffic routing
- CloudFront: CDN and edge caching (L7, HTTP)
- Global Accelerator: network routing with static IPs (L4, TCP/UDP)
- API Gateway: managed API front door
- Transit Gateway: hub for connecting multiple VPCs and on-prem
- PrivateLink: expose services privately across accounts/VPCs
- IRSA: pod-level IAM for EKS workloads

Questions You Must Be Able to Answer

- Why not ECS instead of EKS?
- Why private subnets for app and DB tiers?
- Why ALB instead of NLB?
- Why API Gateway instead of ALB?
- How would you make this multi-AZ?
- How would you reduce latency for global users?
- How would you expose an internal service securely?
- When Transit Gateway vs VPC Peering vs PrivateLink?
- How do EKS pods get IAM permissions? (IRSA)
- What are hidden costs in AWS architectures?
- When Global Accelerator vs CloudFront?

Decision Quick Reference

```
Containers but don’t need K8s ecosystem?    -> ECS
Need K8s APIs, CRDs, Helm, operators?       -> EKS
Event-driven, bursty, short-lived?          -> Lambda
Full OS control, legacy, compliance?        -> EC2

HTTP routing, path/host based?              -> ALB
TCP/UDP, static IP, extreme perf?           -> NLB
gRPC global routing, instant failover?      -> Global Accelerator
HTTP caching at edge?                       -> CloudFront
```

```
2-3 VPCs direct connection?                 -> VPC Peering
Many VPCs + on-prem hub-spoke?              -> Transit Gateway
Expose service without network peering?     -> PrivateLink
```

Gold Standard Sentence

```
My AWS compute choice depends on how much runtime control we need,
how much operational burden the team can absorb, and whether the
architecture is request-driven, event-driven, or container-platform
oriented. For networking, I consider traffic patterns, security
boundaries, cost implications, and whether we need caching, routing,
or private connectivity.
```

Part 02: AWS Interview Notes - Part 2: Storage + Database

Source file: AWS-02-Storage-Database.md

> Covers: S3, EBS, EFS, RDS, DynamoDB, Aurora, ElastiCache. This is the part interviewers use to test whether you understand persistence, scale, durability, consistency, performance bottlenecks, and cost trade-offs.

---

1. How to Think About AWS Storage

Core interview question:

- Is this block storage, file storage, or object storage?
- Is the access pattern transactional, analytical, cache-heavy, or append-only?
- Do we need SQL joins and strict schema, or massive scale with simple access patterns?
- Do we need low latency, high durability, shared access, or cheap archival?

Simple mental map:

```
| Need | Service |
| object storage | S3 |
| block volume for one instance | EBS |
| shared file system | EFS |
| relational DB | RDS / Aurora |
| NoSQL key-value/document | DynamoDB |
| in-memory cache | ElastiCache |
```

---

2. S3

2.1 What It Is

Amazon S3 is durable object storage. You store objects in buckets, not files in directories and not blocks on disks.

Key characteristics:

- virtually unlimited scale
- very high durability
- not a block device
- not a POSIX file system

2.2 Best Use Cases

- static website assets
- backups
- logs
- data lake storage
- media
- document storage
- event-driven processing with object-created notifications

2.3 Important Concepts

- Bucket
- Object
- Key
- Versioning
- Lifecycle rules
- Storage classes
- Pre-signed URLs
- Multipart upload

2.4 Storage Classes

- Standard: frequent access
- Standard-IA: infrequent access
- One Zone-IA: lower cost, lower resilience
- Glacier tiers: archive

Interview angle:

Know lifecycle transitions and retrieval trade-offs.

2.5 Security

- bucket policies
- IAM policies
- Block Public Access
- SSE-S3 / SSE-KMS
- pre-signed URLs for controlled client upload/download

2.6 S3 Performance and Design

- use prefix design only when necessary for organization; S3 scales well
- use multipart upload for large files
- do not use S3 like a low-latency transactional database

2.7 Interviewer Favorites

- Why S3 instead of EFS?
- How do you securely let users upload files?
- How do lifecycle policies reduce cost?
- How do you serve private content globally?

2.8 Strong Answer

```
For user-uploaded documents, I would store files in S3, keep metadata in a database,
expose uploads using pre-signed URLs, enable versioning, encrypt at rest, and serve
downloads through CloudFront if low-latency global access matters.
```

---

3. EBS

3.1 What It Is

Amazon EBS is network-attached block storage for EC2.

Think:

- behaves like a disk volume
- good for databases and file systems running on one instance
- tied to an AZ

3.2 Best Use Cases

- boot volumes
- relational DB on EC2
- application data requiring block semantics
- transactional workloads

3.3 Important Properties

- persistent beyond instance stop/start
- snapshots to S3
- provisioned capacity and performance characteristics
- typically attached to one instance at a time

3.4 Volume Types

- gp3: general purpose SSD, common default
- io1/io2: high IOPS, critical databases
- st1/sc1: throughput-optimized HDD / cold HDD for niche cases

3.5 Interviewer Angle

- EBS is not shared file storage like EFS
- EBS is AZ-scoped
- snapshot strategy matters for backup and restore

---

4. EFS

4.1 What It Is

Amazon EFS is managed shared file storage for Linux workloads.

Think:

- NFS-style shared file system
- multiple instances can mount it
- elastic capacity

4.2 Best Use Cases

- shared content repositories
- ML/data processing needing shared files
- legacy apps needing shared filesystem semantics
- container workloads needing RWX-style storage

4.3 Trade-off

EFS is easier for shared access, but usually slower and more expensive than local or block storage for some patterns.

4.4 Interviewer Comparison

```
| Service | Access model | Typical fit |
| EBS | block, usually single-instance | DB or single-instance filesystem |
| EFS | shared file system | multi-instance shared files |
| S3 | object | assets, backup, data lake |
```

5. RDS

5.1 What It Is

Amazon RDS is managed relational database service. AWS handles backups, patching, monitoring hooks, and failover features for supported engines.

Common engines:

- MySQL
- PostgreSQL
- MariaDB
- Oracle
- SQL Server

5.2 When to Use It

Use RDS when:

- you need SQL
- you need ACID transactions
- joins matter
- schema is relational
- operational overhead should be lower than self-managed DBs

5.3 Multi-AZ

Multi-AZ means:

- synchronous standby replica in another AZ
- failover for high availability
- not primarily a read-scaling feature

This distinction is asked often.

5.4 Read Replicas

Read replicas are for:

- read scaling
- offloading analytics/reporting style queries
- sometimes disaster recovery patterns

Read replicas are not the same as Multi-AZ standby.

5.5 Backups

- automated backups with retention
- manual snapshots
- point-in-time recovery

5.6 Interviewer Hot Points

- Multi-AZ vs read replica
- why RDS over DynamoDB
- what happens during failover
- how to tune connections and indexes

5.7 Strong Answer

```
For an order management system requiring transactions, joins, and referential integrity, I
 would start with RDS PostgreSQL in Multi-AZ mode, add read replicas only if read traffic
justifies it, and use connection pooling plus caching before scaling blindly.
```

6. Aurora

6.1 What It Is

Aurora is AWS's cloud-optimized relational database compatible with MySQL and PostgreSQL.

It separates compute and storage more aggressively than standard RDS engines and is built for higher performance and managed resilience.

6.2 Why Aurora Exists

It addresses common pain points of traditional relational databases in the cloud:

- better scalability
- faster failover
- managed storage replication
- improved performance profile

6.3 Key Properties

- storage auto-scales
- six-way storage replication across three AZs
- reader endpoints for replicas
- writer endpoint for primary

6.4 Aurora vs RDS

Aurora is still under the RDS family but has a different architecture and operational profile.

Interview-safe summary:

```
Choose Aurora when you want managed relational databases with stronger scalability and
availability characteristics than standard RDS engines, and your cost/engine compatibility
 trade-offs make sense.
```

6.5 Aurora Interview Traps

- saying Aurora is "just RDS MySQL"
- confusing reader endpoint with HA standby
- not discussing cost

---

7. DynamoDB

7.1 What It Is

Amazon DynamoDB is a fully managed NoSQL database optimized for key-value and document workloads with very high scale and low-latency access.

7.2 Best Use Cases

- session stores
- user profiles
- shopping carts
- event metadata
- high-scale request-driven systems
- workloads where access patterns are known up front

7.3 Data Modeling Mindset

This is the interview separator.

In relational design you model entities first. In DynamoDB you model access patterns

first.

Ask:

- What queries must be fast?
- What is the partition key?
- Do I need sort-key range queries?
- Do I need GSIs?

7.4 Core Concepts

- partition key
- sort key
- item
- GSI
- LSI
- provisioned vs on-demand capacity
- TTL
- streams

7.5 Partition Key Design

Bad partition key design creates hot partitions.

Senior candidates mention:

- write distribution
- traffic skew
- sharding keys if necessary

7.6 Consistency

- eventual consistency by default for many reads
- strongly consistent reads available in some cases

7.7 DynamoDB Strengths

- scale
- predictable low latency
- fully managed
- no server patching

7.8 DynamoDB Weaknesses

- no rich joins
- data modeling is harder if access patterns are unclear
- ad hoc querying is limited compared to SQL

7.9 Interviewer Favorites

- RDS vs DynamoDB
- how to avoid hot partitions
- when to use GSIs
- how streams help event-driven architecture

---

8. ElastiCache

8.1 What It Is

ElastiCache is managed in-memory caching, commonly Redis or Memcached.

8.2 Use Cases

- read caching

- session storage
- rate limiting
- leaderboard/ranking
- distributed locks with care
- pub/sub or lightweight ephemeral coordination in some designs

8.3 Redis vs Memcached

- Redis: richer data structures, persistence options, more common
- Memcached: simpler distributed cache, less feature-rich

8.4 Caching Patterns

- cache-aside
- write-through
- write-behind

Interviewers usually want you to know cache-aside.

8.5 Cache Risks

- stale data
- cache stampede
- inconsistent invalidation
- overusing cache to hide bad schema/query design

8.6 Strong Answer

```
I use ElastiCache to reduce database read pressure for hot keys and expensive queries, but
 I treat cache invalidation and TTL strategy as first-class design concerns because cache
consistency bugs are application bugs.
```

---

9. Redshift (Surendra will likely probe this)

9.1 What It Is

Amazon Redshift is a fully managed data warehouse optimized for analytical queries on large datasets.

Key characteristics:

- columnar storage
- massively parallel processing (MPP)
- SQL-based analytics
- optimized for OLAP, not OLTP

9.2 When to Use Redshift

```
Use Redshift when:
  [ok] You need to analyze terabytes/petabytes of data
  [ok] Queries are analytical (aggregations, joins across large tables)
  [ok] Workload is batch/BI reports, not real-time transactions
  [ok] You need SQL interface for analysts/BI tools

Do NOT use Redshift when:
  [x] You need sub-millisecond transactional responses
  [x] Data is small (< 100GB) - RDS is simpler
  [x] Access pattern is key-value lookups
```

9.3 Redshift vs RDS vs DynamoDB

```
| Aspect | RDS | DynamoDB | Redshift |
| Workload | OLTP | Key-value/Document | OLAP |
```

```
| Query style | Transactional SQL | Access by key | Analytical SQL |
| Scale | Vertical + Read Replicas | Horizontal | MPP cluster |
| Latency | Low (milliseconds) | Very low | Higher (seconds+) |
| Best for | App backend | High-scale lookups | BI/Analytics |
```

9.4 Key Concepts

- Cluster: leader node + compute nodes
- Distribution style: how data is spread across nodes (KEY, EVEN, ALL)
- Sort keys: optimize range queries
- Redshift Spectrum: query S3 data directly without loading
- Materialized views: pre-computed results for expensive queries
- Concurrency Scaling: auto-add capacity for burst queries

9.5 Distribution Style Decision

```
KEY distribution:
  When you frequently join on a column, distribute by that column
  Both tables joined will have matching rows on same node

EVEN distribution:
  Default. Spreads data evenly. Good when no clear join pattern.

ALL distribution:
  Copies entire table to every node. Only for small dimension tables.
```

9.6 Interview Gold Answer

```
"For our BI reporting layer, I'd use Redshift as the analytics warehouse.
Transactional data flows from RDS/DynamoDB via ETL jobs into Redshift.
I'd design distribution keys based on common join patterns, use sort keys
for date-range queries, and leverage Redshift Spectrum for cold data in S3
without loading it into the cluster."
```

---

10. Athena

10.1 What It Is

Amazon Athena is a serverless query service that runs SQL directly against data in S3.

No infrastructure to manage. Pay per query.

10.2 When to Use Athena

```
Use Athena when:
  [ok] Ad-hoc queries on S3 data lake
  [ok] Log analysis (CloudTrail, ALB logs, VPC Flow Logs)
  [ok] Exploratory analytics without loading data
  [ok] Cost-sensitive - no standing infrastructure

Use Redshift instead when:
  [x] Complex, repeated queries with many joins
  [x] Sub-second dashboard performance needed
  [x] High concurrency requirements
```

10.3 Data Formats

Athena works best with columnar formats:

- Parquet: columnar, compressed, best performance
- ORC: columnar, Hive-optimized
- JSON, CSV: supported but slower and costlier

10.4 Partitioning

Critical for performance and cost:

```
s3://my-bucket/logs/year=2026/month=03/day=30/

Query with partition filter:
  SELECT * FROM logs WHERE year='2026' AND month='03'
  -> Only scans that folder, not entire bucket
```

10.5 Athena vs Redshift

```
| Aspect | Athena | Redshift |
| Infrastructure | Serverless | Cluster |
| Pricing | Per TB scanned | Hourly + storage |
```

| Latency | Seconds-minutes | Faster for complex |

```
| Best for | Ad-hoc, infrequent | Repeated, complex |
| Concurrency | Limited | Better with scaling |
```

10.6 Interview Answer

```
"For ad-hoc analysis of CloudTrail logs, I'd use Athena directly on S3.
But for production dashboards with complex joins, I'd load data into
Redshift where I can optimize distribution and sort keys."
```

---

11. RDS Proxy

11.1 What It Is

RDS Proxy is a managed database proxy that sits between your application and RDS/Aurora.

11.2 Why It Exists

```
Problem: Lambda + RDS = connection explosion
  Each Lambda invocation opens a new DB connection
  100 concurrent Lambdas = 100 connections
  RDS has connection limits (varies by instance size)
  Result: connection exhaustion, errors

Solution: RDS Proxy
  Proxy maintains a connection pool to the database
  Lambda connects to proxy (fast, reuses connections)
  Proxy multiplexes to actual DB connections
```

11.3 Benefits

- Connection pooling: reduces DB connection pressure
- Failover handling: faster failover, app doesn't need reconnect logic
- IAM authentication: integrate with IRSA/IAM for credential-free access
- TLS enforcement: can enforce encrypted connections

11.4 When to Use

```
[ok] Serverless + RDS (Lambda, Fargate with many short tasks)
[ok] High connection churn applications
[ok] Want IAM-based DB authentication
[ok] Need faster failover during Multi-AZ switch

[x] Overkill for low-connection steady-state apps on EC2/ECS
```

11.5 Interview Gold Answer

```
"For Lambda functions accessing RDS, I'd put RDS Proxy in front to solve
connection pooling. Without it, we'd hit connection limits under load.
Proxy also speeds up Multi-AZ failover and supports IAM authentication."
```

---

12. Data Lake Architecture (High-Value for BI Interviews)

12.1 What Is a Data Lake

A data lake is a centralized repository for storing structured and unstructured data at any scale, typically on S3.

12.2 Modern AWS Data Lake Pattern

```
                        DATA SOURCES
  RDS | DynamoDB | Kinesis | APIs | SaaS | On-prem | IoT




                        INGESTION
  AWS Glue | Kinesis Firehose | DMS | Lambda | Step Functions




                  S3 DATA LAKE (ZONES)


     RAW            PROCESSED       CURATED
    (Bronze)      (Silver)     (Gold)
    as-is data      cleaned         analytics




                       CONSUMPTION
  Athena (ad-hoc) | Redshift (BI) | QuickSight | SageMaker (ML)
```

12.3 Zone Definitions

```
Raw/Bronze Zone:
   Data as-is from sources
   No transformation
   Immutable, append-only
   For audit trail and reprocessing

Processed/Silver Zone:
   Cleaned, validated, normalized
   Schema enforced
   Format converted (JSON -> Parquet)
   PII masked if needed

Curated/Gold Zone:
   Business-ready aggregations
   Pre-joined, denormalized for analytics
```

```
  Exposed to BI tools
```

12.4 AWS Glue Role

- Glue Crawler: auto-discover schema, populate Glue Data Catalog
- Glue Data Catalog: metadata store (like Hive metastore)
- Glue ETL: serverless Spark jobs for transformation
- Glue Studio: visual ETL designer

12.5 Interview Gold Answer

```
"For our analytics platform, I'd design a data lake on S3 with bronze/silver/gold
zones. Raw data lands in bronze, Glue ETL transforms to Parquet in silver,
and business aggregations go to gold. Athena queries silver for exploration,
Redshift Spectrum or direct load for production BI dashboards."
```

---

13. High-Value Comparisons

9.1 S3 vs EBS vs EFS

```
| Service | Type | Shared? | Typical fit |
| S3 | object | via API, not mounted disk semantics | assets, backups, logs |
| EBS | block | typically one instance | database disks, boot volumes |
| EFS | file | yes | shared filesystem |
```

9.2 RDS vs Aurora vs DynamoDB

```
| Need | Best fit |
| SQL, joins, conventional relational workloads | RDS |
| relational plus stronger cloud-native scaling/resilience | Aurora |
| massive scale key-value/document with access-pattern design | DynamoDB |
```

9.3 ElastiCache vs Database

Cache is not your source of truth unless the system is explicitly designed that way.

Use cache to accelerate reads, absorb bursts, and reduce DB load.

---

10. Architecture Patterns

10.1 File Upload Platform

```
Client
  ->
API issues pre-signed URL
  ->
S3 stores object
  ->
metadata stored in RDS/DynamoDB
  ->
CloudFront serves downloads
```

10.2 Classic Transactional App

```
App tier
  ->
RDS PostgreSQL Multi-AZ
  ->
ElastiCache Redis for hot reads
```

```
  ->
S3 for documents and backups
```

10.3 Event-Driven High-Scale Platform

```
API / Lambda / services
  ->
DynamoDB for request-time lookups
  ->
S3 for immutable objects
  ->
ElastiCache for hot keys
```

---

11. Common Interview Traps

Trap 1

"S3 is a file system."

Correct:

No. It is object storage with API access semantics.

Trap 2

"RDS Multi-AZ scales reads."

Correct:

No. Multi-AZ primarily improves availability. Read replicas scale reads.

Trap 3

"DynamoDB is always better because it scales more."

Correct:

Scale is not the only criterion. Query flexibility, transactions, schema relationships, and developer productivity matter.

Trap 4

"Cache will fix a slow system."

Correct:

Sometimes. But poor data modeling, missing indexes, and bad query patterns must still be fixed.

Trap 5

"EFS can replace S3 for everything."

Correct:

No. Shared filesystem and object store solve different problems.

---

16. Rapid Revision Sheet

One-Line Definitions

- S3: durable object storage
- EBS: block storage for EC2
- EFS: shared managed file system
- RDS: managed relational database

- Aurora: cloud-optimized relational database in the RDS family
- DynamoDB: managed NoSQL key-value/document DB
- ElastiCache: managed in-memory caching
- Redshift: columnar data warehouse for analytics
- Athena: serverless SQL on S3
- RDS Proxy: connection pooling for serverless + RDS
- Glue: serverless ETL and data catalog

Questions You Must Be Able to Answer

- Why RDS over DynamoDB?
- Why Aurora over standard RDS?
- Why S3 over EFS?
- When do read replicas help?
- How do you design a good DynamoDB partition key?
- When Redshift vs Athena?
- Why RDS Proxy for Lambda + RDS?
- What are data lake zones (bronze/silver/gold)?
- How does Glue fit in a data pipeline?

Decision Quick Reference

```
Transactional SQL workload?            -> RDS / Aurora
Key-value at massive scale?            -> DynamoDB
Analytical warehouse?                  -> Redshift
Ad-hoc queries on S3?                  -> Athena
Lambda + RDS connection issues?        -> RDS Proxy
ETL and data catalog?                  -> Glue
Cache hot reads?                       -> ElastiCache
```

Gold Standard Sentence

```
My storage and database choices depend on data model (relational vs NoSQL vs object),
access pattern (transactional vs analytical vs key-based), scale requirements,
and whether the workload is OLTP or OLAP. For analytics, I design data lakes
with proper zones and use the right query engine for each use case.
```

- When should cache-aside be used?

Gold Standard Sentence

```
I choose AWS persistence services by matching the data model, access pattern, consistency
needs, latency target, and operational burden rather than by chasing whichever service
scales the most on paper.
```

Part 03: AWS Interview Notes - Part 3: Security + Messaging + Integration

Source file: AWS-03-Security-Messaging-Integration.md

> Covers: IAM, KMS, Cognito, SQS, SNS, EventBridge, Step Functions. This part is where interviewers check whether you can build secure systems, decouple services correctly, and reason about async workflows without creating operational chaos.

---

1. How to Think About Security and Integration

Security and integration questions usually test whether you understand:

- identity vs authentication vs authorization
- encryption at rest vs encryption in transit
- point-to-point coupling vs pub/sub
- queue vs event bus vs workflow orchestration
- retries, idempotency, dead-letter handling

Good architects reduce coupling while keeping failure modes understandable.

---

2. IAM

2.1 What It Is

AWS IAM controls who can do what on which resource.

This is the foundational AWS security service.

2.2 Key Concepts

- User: long-term identity, usually avoid for apps
- Group: collection of users
- Role: assumed identity, primary mechanism for workloads
- Policy: JSON permissions document

2.3 Golden Rule

Prefer roles over static credentials.

Examples:

- EC2 instance role
- ECS task role
- Lambda execution role
- EKS IAM role for service accounts

2.4 Policy Evaluation

Important principle:

- explicit deny beats allow
- if no allow matches, access is denied

2.5 Least Privilege

Interviewers expect you to say:

- scope permissions narrowly
- avoid * wherever possible
- separate human access from workload access
- rotate away from static access keys

2.6 Common IAM Use Cases

- service accessing S3 bucket
- Lambda reading Secrets Manager value
- ECS task publishing to SQS

2.7 Interview Trap

"Attach admin policy and move on."

That is not an engineering answer. It is a security failure.

---

3. KMS

3.1 What It Is

AWS KMS manages cryptographic keys used for encryption.

It commonly supports:

- encrypting S3 objects
- encrypting RDS storage
- encrypting EBS volumes
- envelope encryption patterns

3.2 Core Concepts

- CMK/KMS key
- customer managed vs AWS managed keys
- key policies
- grants
- envelope encryption

3.3 Envelope Encryption

High-value interview topic:

1. data is encrypted with a data key
2. data key is encrypted by a KMS key
3. encrypted data and encrypted data key are stored

Why it matters:

- scalable
- efficient
- centralizes master key control

3.4 Key Rotation and Access

- customer managed keys offer stronger control
- IAM and key policies both matter
- audit access with CloudTrail

3.5 What Interviewers Ask

- difference between AWS managed and customer managed keys
- when to use KMS directly vs let an AWS service integrate with it
- how encryption access is controlled

---

4. Secrets Manager vs Parameter Store

4.1 The Question You Will Be Asked

"Why use Secrets Manager instead of Parameter Store?"

This is a very common interview question.

4.2 Comparison

```
| Aspect | Secrets Manager | Parameter Store |
| Primary purpose | Secrets (DB creds, API keys) | Config values + secrets |
| Automatic rotation | Yes (built-in for RDS, etc.) | No |
| Cost | $0.40/secret/month + API calls | Free tier + lower cost |
| Max size | 64KB | 8KB (standard), 10KB (advanced) |
| Versioning | Yes | Yes |
| Cross-account | Yes | Yes |
| KMS encryption | Always | Optional SecureString |
```

4.3 When to Use Each

```
Use Secrets Manager when:
  [ok] Credentials need automatic rotation (RDS, Redshift)
  [ok] Secrets are shared across accounts
  [ok] Compliance requires managed rotation
  [ok] Cost is not the primary concern

Use Parameter Store when:
  [ok] Storing configuration (feature flags, endpoints)
  [ok] Budget-conscious secret storage without rotation
  [ok] Simple key-value config for apps
  [ok] Already in SSM ecosystem
```

4.4 Interview Gold Answer

```
"For database credentials that need rotation, I use Secrets Manager-it has
built-in rotation for RDS. For application config values and non-rotating
secrets, Parameter Store is simpler and cheaper. Both integrate with IAM
for access control and KMS for encryption."
```

---

5. Cognito

4.1 What It Is

Amazon Cognito is managed identity for application users.

It helps with:

- sign-up/sign-in
- user directories
- token issuance
- federation with external identity providers

4.2 Key Pieces

- User Pools: authentication, users, tokens
- Identity Pools: temporary AWS credentials for client access scenarios

4.3 Where It Fits

Use Cognito when your application needs end-user identity and you do not want to build auth from scratch.

4.4 Interviewer Angle

Do not confuse:

- IAM: AWS resource access control
- Cognito: end-user identity and authentication for applications

4.5 Strong Answer

```
If I need end-user login for a web app and token-based authentication without building a
full auth system, Cognito User Pools is a reasonable managed choice. IAM roles solve
workload authorization in AWS, not end-user login.
```

---

5. SQS

5.1 What It Is

Amazon SQS is a managed message queue.

Core value:

- decouples producers from consumers
- buffers spikes
- improves resilience

5.2 Queue Types

- Standard Queue: at-least-once delivery, best-effort ordering
- FIFO Queue: ordering and deduplication guarantees with throughput trade-offs

5.3 Important Concepts

- visibility timeout
- long polling
- dead-letter queue
- redrive
- message retention

5.4 What Interviewers Care About

- at-least-once means consumers must be idempotent
- visibility timeout must exceed processing time reasonably
- DLQ design matters

5.5 Common Use Cases

- async order processing
- background jobs
- buffering traffic spikes
- Lambda event source

5.6 Strong Answer

```
I use SQS when I want temporal decoupling between services. The producer should not care
whether the consumer is temporarily slow or down, as long as the message is durably
queued.
```

---

6. SNS

6.1 What It Is

Amazon SNS is a pub/sub notification service.

It fans out a message to multiple subscribers.

6.2 Best Use Cases

- fan-out to multiple SQS queues
- push notifications
- email/SMS notifications
- broadcasting events to multiple consumers

6.3 SNS vs SQS

- SNS pushes to subscribers
- SQS stores messages for polling consumers

Very common architecture:

```
Publisher -> SNS topic -> multiple SQS queues -> independent consumers
```

This is one of the strongest basic integration patterns in AWS.

6.4 Interviewer Hot Points

- why SNS plus SQS is better than one service calling five other services directly
- how each consumer can retry independently

---

7. EventBridge

7.1 What It Is

Amazon EventBridge is an event bus for routing events between AWS services, SaaS systems, and your applications.

7.2 When to Use It

Use EventBridge when:

- you are routing events by rules
- many producers and consumers exist
- you want event-driven architecture with loose coupling
- AWS service events are part of the design

7.3 Core Model

- producer emits event
- event lands on bus
- rules match event pattern
- targets receive event

7.4 EventBridge vs SNS

SNS is simpler broadcast pub/sub. EventBridge is richer event routing and event-bus style integration.

Interview-safe distinction:

```
SNS is excellent for fan-out notifications.
EventBridge is stronger when routing decisions depend on event content and multiple rule-
based targets.
```

7.5 Good Use Cases

- order-created event routed differently by order type
- AWS service events triggering remediation
- SaaS integrations

---

8. Step Functions

8.1 What It Is

AWS Step Functions is a workflow orchestration service.

Use it when a process has multiple steps, branching, retries, and error handling.

8.2 Best Use Cases

- business workflows
- ETL pipelines
- approval flows
- saga-like orchestration
- multi-step serverless processing

8.3 Why It Matters

It externalizes orchestration logic instead of hiding it in code scattered across services.

8.4 Strong Features

- explicit state transitions
- retries
- catch/fallback handling
- parallel branches
- auditability of workflow execution

8.5 Step Functions vs Simple Chained Lambdas

Interview answer:

```
If the workflow has multiple steps, conditional branches, retries, and compensation
handling, Step Functions is preferable to hand-rolled orchestration because it makes state
 and failure paths explicit.
```

8.6 Step Functions and Saga Pattern

This is a strong answer in system design interviews:

- choreography: services react to events themselves
- orchestration: central coordinator drives the flow

Step Functions supports orchestration-style workflows.

---

10. Kinesis (for Surendra’s data streaming interest)

10.1 What It Is

Amazon Kinesis is a family of services for real-time data streaming.

10.2 Kinesis Components

```
Kinesis Data Streams:
   Real-time ingestion and processing
   You manage consumers (Lambda, EC2, ECS)
   Retention: 1-365 days
   Shard-based scaling

Kinesis Data Firehose:
   Fully managed delivery to destinations
   S3, Redshift, Elasticsearch, Splunk
   Near real-time (buffers 60s-900s)
   Zero administration

Kinesis Data Analytics:
   SQL or Flink on streaming data
   Real-time transformations
```

10.3 Kinesis vs SQS vs Kafka

```
| Aspect | Kinesis | SQS | MSK (Kafka) |
| Model | Stream (ordered, replay) | Queue (consume & delete) | Stream |
| Ordering | Per shard | FIFO only | Per partition |
| Replay | Yes (retention window) | No | Yes |
| Consumer model | Multiple consumers read same data | One consumer per message | Multiple |
| Management | Managed | Fully managed | Semi-managed |
| Best for | Real-time analytics | Async tasks, decoupling | High-throughput streaming |
```

10.4 When to Use Kinesis

```
[ok] Real-time analytics (clickstream, IoT, logs)
[ok] Multiple consumers need same data stream
[ok] Need to replay data within retention window
[ok] Event sourcing patterns
[x] Simple async task processing (use SQS)
[x] Need infinite retention (use Kafka/MSK or S3)
```

10.5 Architecture Pattern: Log Analytics

```
Application logs
  -> CloudWatch Logs
  -> Subscription filter
  -> Kinesis Firehose
  -> S3 (Parquet, partitioned)
  -> Athena / Redshift Spectrum
```

10.6 Interview Gold Answer

```
"For real-time clickstream analytics, I'd use Kinesis Data Streams
with Lambda consumers for sub-second processing, and Kinesis Firehose
for buffered delivery to S3 for batch analytics. If I just need
async task processing without replay, SQS is simpler."
```

---

11. WAF (Web Application Firewall)

11.1 What It Is

AWS WAF protects web applications from common web exploits.

It can be attached to:

- CloudFront
- ALB
- API Gateway
- AppSync

11.2 What WAF Protects Against

```
[ok] SQL injection
[ok] Cross-site scripting (XSS)
[ok] Bad bots and scrapers
[ok] Geographic blocking
[ok] Rate limiting (request throttling)
[ok] IP reputation filtering
```

11.3 WAF Components

- Web ACL: container for rules
- Rules: match conditions + action (allow/block/count)
- Rule groups: reusable sets of rules
- Managed rules: AWS or marketplace rule sets

11.4 Common Interview Pattern

```
Q: "How would you protect your API from attacks?"

A: "I'd put WAF on the ALB or API Gateway with:
   - AWS Managed Rules for common threats (SQL injection, XSS)
   - Rate-based rules to prevent DDoS/abuse
   - IP set rules for known bad actors
```

```
  - Geo restrictions if business requires
  And use Shield Standard (free) for network-layer DDoS."
```

11.5 WAF vs Security Groups vs NACLs

```
| Layer | Service | What it does |
| L3-4 | Security Groups | Instance-level stateful firewall |
| L3-4 | NACLs | Subnet-level stateless firewall |
| L7 | WAF | Application-layer HTTP rule filtering |
| L3-4 | Shield | DDoS protection |
```

---

12. High-Value Comparisons

9.1 IAM vs Cognito

```
| Service | Main purpose |
| IAM | AWS resource authorization |
| Cognito | end-user identity/authentication |
```

9.2 SQS vs SNS vs EventBridge

```
| Need | Best fit |
| durable queue with consumers pulling | SQS |
| simple fan-out pub/sub | SNS |
| event-bus routing with rule matching | EventBridge |
```

9.3 EventBridge vs Step Functions

```
| Need | Best fit |
| route events to targets | EventBridge |
| manage multi-step workflow with state | Step Functions |
```

9.4 SNS + SQS Pattern

Excellent when:

- one event has multiple downstream consumers
- each consumer needs isolated retry/failure behavior

---

10. Architecture Patterns

10.1 Order Processing

```
API
  ->
Order service
  ->
SNS topic
  ->
inventory queue -> inventory consumer
payment queue   -> payment consumer
email queue     -> notification consumer
```

Why strong:

- decoupled fan-out
- each downstream service scales independently
- failures isolated by queue

10.2 Event Routing Platform

```
Producer services
  ->
EventBridge bus
  ->
rules by event type
  ->
Lambda / Step Functions / SQS / other targets
```

Why strong:

- rule-based event routing
- central event backbone

10.3 Secure File Processing

```
User authenticates via Cognito
  ->
upload to S3
  ->
event triggers SQS or EventBridge
  ->
processor with IAM role
  ->
data encrypted using KMS-backed keys
```

---

11. Common Interview Traps

Trap 1

"IAM is for app user login."

Correct:

Usually no. IAM governs AWS identities and permissions. Cognito handles end-user authentication for apps.

Trap 2

"SQS guarantees exactly once."

Correct:

Standard queues are at-least-once. Design idempotent consumers.

Trap 3

"SNS replaces queues."

Correct:

SNS is fan-out pub/sub. It does not replace durable per-consumer buffering the way SQS does.

Trap 4

"EventBridge is just another queue."

Correct:

It is an event-routing bus, not a simple queue.

Trap 5

"Encryption at rest means only KMS exists."

Correct:

KMS manages keys and integrates with services. The end-to-end security story includes IAM, policies, network controls, audit logs, and application design.

15. Rapid Revision Sheet

One-Line Definitions

- IAM: who can do what in AWS
- KMS: key management for encryption
- Secrets Manager: managed secrets with rotation
- Parameter Store: config and secrets storage
- Cognito: app user authentication/identity
- SQS: durable queue
- SNS: fan-out pub/sub
- EventBridge: event bus with rule-based routing
- Step Functions: workflow orchestration
- Kinesis: real-time data streaming
- WAF: L7 web application firewall

Questions You Must Be Able to Answer

- Why role-based access over static keys?
- When do you use SNS plus SQS together?
- When is EventBridge better than SNS?
- Why must SQS consumers be idempotent?
- When is Step Functions preferable to Lambda chaining?
- How does KMS fit into encryption at rest?
- Secrets Manager vs Parameter Store?
- Kinesis vs SQS for streaming?
- Where does WAF fit in the security stack?

Decision Quick Reference

```
Need credential rotation?               -> Secrets Manager
Simple config + budget-conscious?       -> Parameter Store
Real-time stream with replay?           -> Kinesis
Async task queue?                       -> SQS
Fan-out to multiple consumers?          -> SNS + SQS
Event routing by content?               -> EventBridge
Multi-step workflow with state?         -> Step Functions
L7 attack protection?                   -> WAF
```

Gold Standard Sentence

```
I separate security concerns into identity, authorization, and encryption,
and I separate integration concerns into queues, pub/sub, event routing,
streaming, and workflow orchestration so each failure mode stays explicit
and manageable.
```

Part 04: AWS Interview Notes - Part 4: DevOps + Monitoring + Architecture + Interview Q&A

Source file: AWS-04-DevOps-Monitoring-Architecture-Interview-QA.md

> Covers: CI/CD on AWS, CloudWatch, Well-Architected Framework, architecture instincts, and 25 interview Q&As. This final part is meant to sharpen your production thinking and help you answer follow-up questions with senior-level depth.

1. CI/CD on AWS

1.1 What Interviewers Want

They want to know whether you can take code from commit to production safely.

A strong answer usually includes:

- source control trigger
- build/test stage
- security/static checks
- artifact creation
- deployment strategy
- rollback plan
- observability after release

1.2 Common AWS-Native Pipeline Components

- CodeCommit or GitHub as source
- CodeBuild for build/test
- CodeDeploy for deployments
- CodePipeline for orchestration
- ECR for container registry
- CloudFormation / CDK / Terraform for infrastructure

1.3 Typical Container Delivery Flow

```
Git push
  ->
pipeline trigger
  ->
unit/integration tests
  ->
security scan + quality gate
  ->
build Docker image
  ->
push to ECR
  ->
deploy to ECS/EKS
  ->
run smoke tests
  ->
monitor metrics and rollback if needed
```

1.4 Deployment Strategies

- Rolling update: replace gradually
- Blue-green: new environment then switch traffic
- Canary: send small percentage first

Interview-safe trade-off:

```
Blue-green reduces deployment risk but doubles environment cost temporarily.
Canary is excellent when you want to detect issues with real traffic gradually.
Rolling is simpler but may expose more users if a bad version passes health checks.
```

1.5 Infrastructure as Code

Interviewers expect this.

Why it matters:

- reproducibility
- reviewability
- reduced drift
- safer environment creation

Typical tools:

- CloudFormation
- CDK
- Terraform

1.6 Secrets in CI/CD

Never hardcode secrets in pipelines.

Use:

- Secrets Manager
- Parameter Store
- IAM roles

1.7 Strong CI/CD Answer

```
My production pipeline should be automated, auditable, and reversible. I want immutable
artifacts, environment-specific configuration outside the artifact, progressive deployment
 where risk justifies it, and post-deploy alarms tied to rollback decisions.
```

---

2. CloudWatch

2.1 What It Is

CloudWatch is AWS’s monitoring and observability service family for metrics, logs, alarms, dashboards, and events.

2.2 Core Capabilities

- Metrics
- Logs
- Alarms
- Dashboards
- Log Insights
- Events integration

2.3 What to Monitor

Infrastructure:

- CPU
- memory if custom metric
- disk
- network

Application:

- request latency
- error rate
- throughput
- queue depth
- downstream dependency failures

Business:

- orders per minute
- payment failures
- signup conversions

Senior candidates mention all three layers.

2.4 Alarms

Good alarms are:

- actionable
- tied to SLOs or meaningful thresholds
- not noisy

Examples:

- ALB 5xx spike
- Lambda errors and throttles
- SQS queue depth growing abnormally
- RDS CPU + connection exhaustion

2.5 Logs

CloudWatch Logs is useful for:

- centralized application logs
- Lambda execution logs
- ECS/EKS integrated logging paths
- searching with Logs Insights

2.6 Golden Observability Principle

If you cannot answer "what failed, where, and since when?", your monitoring is not mature enough.

2.7 Common Mistakes

- monitoring only CPU
- ignoring latency percentiles
- no correlation IDs
- too many useless alerts

---

3. X-Ray (Distributed Tracing)

3.1 What It Is

AWS X-Ray is a distributed tracing service that helps you analyze and debug applications, especially microservices.

3.2 What X-Ray Solves

```
Problem:
  In microservices, one user request touches many services.
  When something is slow or fails, which service is the problem?

Solution:
  X-Ray traces the entire request path across services,
  showing latency breakdown and error points.
```

3.3 Key Concepts

- Trace: end-to-end request path
- Segment: work done by one service
- Subsegment: downstream calls (DB, HTTP, AWS SDK)
- Service Map: visual graph of service dependencies
- Annotations: indexed metadata for filtering
- Metadata: non-indexed debug info

3.4 How to Enable

```
For Lambda:
  Enable Active Tracing in function config
  SDK auto-instruments AWS SDK calls

For ECS/EKS:
  Run X-Ray daemon as sidecar
  Instrument code with X-Ray SDK

For API Gateway:
  Enable tracing in stage settings
```

3.5 X-Ray Service Map Example

```
                                    RDS      <- avg 45ms
                              (MySQL)

 Client API GW

       avg 5ms       avg 120ms       Lambda    <- avg 80ms
                              (Order)




                                     SQS     <- avg 3ms
```

3.6 Interview Gold Answer

```
"For debugging latency in our microservices, I enable X-Ray tracing.
It shows me the service map and latency breakdown per segment.
If an API is slow, I can see whether the bottleneck is the Lambda code,
the database query, or a downstream service call."
```

---

4. CloudTrail (Audit)

4.1 What It Is

AWS CloudTrail records API calls made in your AWS account.

Every action-console, CLI, SDK, service-is logged.

4.2 Why It Matters

```
Security: Who deleted that S3 bucket?
Compliance: Prove that only authorized users accessed data.
Debugging: What changed before the system broke?
Forensics: Investigate security incidents.
```

4.3 Key Concepts

- Management events: control plane (CreateBucket, RunInstances)
- Data events: data plane (S3 GetObject, Lambda Invoke)
- Trail: configuration for what to log and where to store
- Log files: JSON in S3, can be queried with Athena

4.4 Best Practices

```
[ok] Enable CloudTrail in all regions
[ok] Enable log file integrity validation
[ok] Store logs in a separate, locked-down S3 bucket
[ok] Enable S3 data events for sensitive buckets
[ok] Use Athena to query logs for investigations
[ok] Set up CloudWatch alarms for suspicious activity
```

4.5 CloudTrail + Athena Pattern

```
[sql]
-- Who deleted objects from sensitive bucket in last 7 days?
SELECT eventTime, userIdentity.arn, requestParameters
FROM cloudtrail_logs
WHERE eventName = 'DeleteObject'
  AND requestParameters LIKE '%sensitive-bucket%'
  AND eventTime > date_add('day', -7, current_date)
```

4.6 Interview Gold Answer

```
"CloudTrail is our audit log for AWS. I ensure it's enabled in all regions,
with logs stored in a separate account's S3 bucket for tamper resistance.
For investigations, I query logs with Athena. For real-time alerting on
suspicious activity, I set up CloudWatch Events rules."
```

---

5. Well-Architected Framework

AWS Well-Architected Framework pillars:

1. Operational Excellence
2. Security
3. Reliability
4. Performance Efficiency
5. Cost Optimization
6. Sustainability

These names alone are not enough. You need to explain them.

3.1 Operational Excellence

- automate operations
- improve via feedback loops
- make changes small and reversible
- learn from incidents

3.2 Security

- least privilege
- traceability
- data protection
- secure all layers

3.3 Reliability

- recover from failure
- scale to meet demand
- test recovery procedures
- remove single points of failure

3.4 Performance Efficiency

- pick the right resource types
- use managed services where appropriate

- monitor and evolve

3.5 Cost Optimization

- right-size workloads
- use pricing models wisely
- measure cost by architecture component
- eliminate idle overprovisioning

3.6 Sustainability

- efficient resource usage
- reduce waste
- optimize demand and architecture footprint

3.7 Strong Interview Pattern

When given any architecture, evaluate it across at least:

- availability
- security
- cost
- operability
- scaling

That is effectively Well-Architected thinking.

---

4. Architecture Thinking for Interviews

4.1 Start with Requirements

Ask or state:

- expected traffic
- latency target
- availability target
- read/write pattern
- data sensitivity
- regional scope
- budget constraints

4.2 Then Design Along These Axes

- compute model
- data model
- network path
- failure handling
- observability
- security
- cost

4.3 A Good AWS Answer Sounds Like This

```
For this workload I would keep the stateless API tier behind an ALB across multiple AZs,
store transactional data in Multi-AZ RDS, use Redis for hot reads, publish async work to
SQS, emit metrics and structured logs to CloudWatch, and keep least-privilege IAM plus
KMS-backed encryption enabled by default.
```

4.4 What Separates Mid-Level from Senior Answers

Mid-level answer:

- names services

Senior answer:

- names trade-offs
- describes failure modes
- describes security boundaries
- describes scaling behavior
- describes operational implications

---

5. 25 High-Value Interview Q&As

1. When would you choose ECS over EKS?

Choose ECS when the main requirement is running containers on AWS with lower operational complexity. Choose EKS when Kubernetes APIs, tooling, portability, or platform standardization matter enough to justify the added operational burden.

2. What is the difference between Multi-AZ and read replicas in RDS?

Multi-AZ is primarily for high availability and failover. Read replicas are primarily for read scaling and offloading read traffic.

3. Why use private subnets for application servers?

Private subnets reduce direct exposure to the internet. Public traffic should terminate at controlled entry points such as ALB or API Gateway, while app servers stay reachable only through internal paths.

4. When would Lambda be a poor choice?

For long-running, stateful, low-latency-sensitive, or highly predictable heavy-throughput workloads where container or instance-based compute is operationally and economically better.

5. What is the difference between ALB and NLB?

ALB is Layer 7 and supports HTTP-aware routing such as host/path rules. NLB is Layer 4 and is better for TCP/UDP, very high performance, and certain static-IP use cases.

6. Why is S3 not suitable as a database?

S3 is object storage, not a transactional query engine. It lacks relational semantics, low-latency row-level updates, and database-style indexing/query behavior.

7. When do you choose DynamoDB over RDS?

When access patterns are known, scale is very high, low-latency key-based access matters, and relational joins are not central to the problem.

8. What is the danger of a bad DynamoDB partition key?

It creates hot partitions, uneven traffic distribution, throttling, and poor scale behavior.

9. Why use ElastiCache?

To reduce database read pressure, improve latency for hot data, and absorb bursty traffic. It should complement, not replace, good data modeling.

10. What does least privilege mean in IAM?

Grant only the minimum actions on the minimum resources required for a principal to do its job, and nothing more.

11. Why are IAM roles preferred over access keys?

Roles avoid hardcoded long-lived credentials, improve rotation posture, and fit AWS's temporary-credential model for workloads.

12. What is envelope encryption in KMS?

Data is encrypted with a data key, and the data key is encrypted with a KMS-managed master key. This scales better than using a master key directly for all data operations.

13. What problem does SQS solve?

It decouples producers and consumers, buffers spikes, improves resilience, and allows asynchronous processing.

14. Why must SQS consumers be idempotent?

Because standard SQS provides at-least-once delivery, so duplicates can occur and the consumer must handle them safely.

15. When is SNS plus SQS a strong pattern?

When one event must fan out to multiple independent consumers and each consumer needs isolated retry and failure handling.

16. When is EventBridge better than SNS?

When event routing depends on event content or you want an event-bus model with rule-based dispatch to multiple targets.

17. When should Step Functions be used?

When the workflow has multiple steps, branching, retries, compensation, or long-running orchestration that should be explicit rather than buried inside application code.

18. How do you design a secure file-upload system on AWS?

Use Cognito or your auth layer for identity, issue pre-signed S3 URLs, keep the bucket private, encrypt with KMS-backed settings, store metadata separately, and serve downloads through controlled access such as CloudFront if needed.

19. How would you make a web application highly available in one region?

Deploy compute across multiple AZs behind a load balancer, keep databases in Multi-AZ mode, store static assets in S3, use health checks and autoscaling, and avoid single-instance dependencies.

20. What would you monitor for a production API?

Latency, error rate, throughput, saturation metrics, downstream dependency health, queue depth, and key business metrics.

21. What is a good rollback strategy for production deployments?

Use immutable artifacts, health checks, deployment stages like canary or blue-green where risk justifies it, and automatic or manual rollback triggered by verified failure signals.

22. Why is Infrastructure as Code important?

It makes infrastructure reproducible, reviewable, versioned, and less prone to manual drift or undocumented changes.

23. How would you reduce AWS cost without harming reliability?

Right-size compute, use Savings Plans or Reserved capacity where stable, use Spot for fault-tolerant workloads, optimize storage classes, remove idle resources, and cache or tune before brute-force scaling.

24. How would you answer "design a scalable notifications system on AWS"?

I would accept requests through API Gateway or app services, publish events to SNS or EventBridge, buffer provider-specific work in SQS queues, process asynchronously with Lambda or containers, persist delivery state in a database, and monitor queue depth, retries, and provider failure rates.

25. What is the best way to answer AWS architecture questions in interviews?

Start from requirements, choose services based on workload characteristics, explain trade-offs, call out failure handling and security boundaries, and show how you would observe and operate the system in production.

---

6. Final Revision Sheet

Core Architecture Defaults

- public entry through ALB or API Gateway
- compute in private subnets
- data tier private and highly available
- async work through SQS or event-based routing
- encryption with KMS
- least privilege with IAM
- metrics/logs/alarms in CloudWatch
- distributed tracing with X-Ray
- audit trail with CloudTrail

What Interviewers Keep Testing

- Can you compare services instead of just naming them?
- Do you understand failure modes?
- Do you know when to use managed services?
- Can you protect the system properly?
- Can you justify cost/performance trade-offs?
- Can you debug distributed systems? (X-Ray)
- Can you answer "who did what when?" (CloudTrail)

Decision Quick Reference

```
Debug latency in microservices?        -> X-Ray
Audit who changed what?                -> CloudTrail
Application metrics and logs?          -> CloudWatch
Alerts on thresholds?                  -> CloudWatch Alarms
Visual service dependency map?         -> X-Ray Service Map
Query audit logs?                      -> Athena on CloudTrail S3
```

Final Gold Standard Sentence

```
The best AWS architecture is not the one with the most services;
it is the one that meets the workload’s reliability, security,
scalability, operability, and cost goals with the least unnecessary
complexity-and includes observability (CloudWatch, X-Ray) and
auditability (CloudTrail) from day one.
```

Part 05: AWS Deep Dive: EC2, ECS, EKS Through Story Mode + Spring Boot/React Deployment Journey

Source file: AWS-05-EC2-ECS-EKS-Story-and-Deployment-Guide.md

> A detailed guide for understanding EC2, ECS, and EKS in a practical way. This note first explains them through a story of a growing product team, then walks from a local Java Spring Boot + React codebase to production-grade AWS deployments using each option.

---

1. Why This Guide Exists

When people ask about EC2, ECS, and EKS, they often get three shallow answers:

- EC2 is virtual machines
- ECS is AWS containers
- EKS is managed Kubernetes

That is factually correct but operationally useless.

What you really need to know is:

- where your code runs
- what you are responsible for
- how deployment changes from one model to another
- what your team gains and what your team must now operate
- how a real application moves from local development to AWS production

This guide answers exactly that.

---

2. The Story Mode: How Teams Usually Evolve

Let us imagine a product team building an internal-to-external platform.

The application is:

- React frontend
- Java Spring Boot backend
- PostgreSQL database
- Redis cache later
- file uploads later
- APIs consumed by web and maybe mobile clients later

At the start, the team has only local code.

2.1 Phase 1: "We just need this live"

The team is small.

- 2 backend engineers
- 1 frontend engineer
- 1 DevOps-minded engineer or maybe none
- one staging environment
- one production environment
- traffic is low

The team says:

"We need something understandable. We do not want to learn Kubernetes right now. We just want our app running on AWS."

That usually leads to EC2.

Why?

- easiest mental model if the team already knows servers
- simple SSH-style debugging, at least initially
- no immediate container orchestration learning curve
- direct control over JVM tuning, OS packages, reverse proxy, and deployment scripts

The team might do:

- React build served by Nginx
- Spring Boot JAR running as a systemd service
- both hosted on one or more EC2 instances
- ALB in front

This works. It is not elegant, but it works.

2.2 Phase 2: "Deployments are getting messy"

Now the team has more traffic and more releases.

Pain starts showing:

- "Which server has which version?"
- "Why does prod behave differently from staging?"
- "Why are we patching machines manually?"
- "Why did this deployment break one node but not the others?"

At this point, the team wants:

- immutable deployments
- predictable runtime packaging
- easier rollback
- easier autoscaling
- less server-level operational work

That usually leads to ECS.

Why?

- package app as Docker images
- deployment becomes image-based, not server-script-based
- Fargate removes server/node management entirely
- good AWS-native integration with ALB, IAM, CloudWatch, Secrets Manager
- lower operational overhead than Kubernetes

Now the team runs:

- Spring Boot container as an ECS service
- maybe React as a separate Nginx container or, more commonly, React static files on S3 + CloudFront
- deployments through ECR + ECS service updates

This is the point where many companies stop. They do not need EKS.

2.3 Phase 3: "We are now a platform team, not just an app team"

The company grows.

- many microservices
- different teams
- shared platform capabilities
- sidecars, operators, service mesh, Helm charts, GitOps, policy enforcement
- maybe some multi-cloud or Kubernetes standardization requirement

Now the company says:

"We want Kubernetes because the ecosystem itself matters to us."

That leads to EKS.

Why?

- Kubernetes APIs and ecosystem
- rich scheduling and workload primitives
- Helm, CRDs, operators, admission controls

- standardized platform across many teams
- advanced autoscaling and workload policies

But the cost is real:

- more complexity
- more moving parts
- more cluster expertise required
- more failure modes
- more responsibility even though AWS manages the control plane

So the story is not:

"EC2 is old, ECS is better, EKS is best."

The real story is:

- EC2 is simplest when you want server control
- ECS is most pragmatic for AWS-native container operations
- EKS is correct when Kubernetes capabilities are truly needed

2.4 Visual Timeline of a Typical Team's Evolution

```
[text]
   Month 1-3           Month 4-9           Month 12+

   "Just ship it"      "Stabilize ops"     "Platform thinking"


      EC2      >    ECS      >     EKS
                     (Fargate)


   JAR on server       Docker images       K8s manifests
   systemd             Task definitions    Helm/operators
   Manual deploy       ECR + CI/CD         GitOps
   SSH debugging       CloudWatch logs     Service mesh
   1-2 services        3-10 services       10-50+ services
   1 team              2-3 teams           Platform team

   Ops burden: LOW     Ops burden: MEDIUM  Ops burden: HIGH
   (at first)          (but controlled)    (but powerful)

    This is not a mandatory progression.
   Many companies stay at ECS permanently. That is valid.
   Move to EKS only when K8s capabilities are truly needed.
```

---

3. Starting Point: Your Local Application

Let us assume your laptop currently has something like this:

```
[text]
 my-app/
   backend/
     src/main/java/...
     pom.xml
     Dockerfile           (maybe not yet)
   frontend/
     src/...
     package.json
     Dockerfile           (maybe not yet)
   docker-compose.yml     (maybe not yet)
   README.md
```

The backend is a Spring Boot application.

- exposes REST APIs
- connects to PostgreSQL
- maybe uses Redis
- may use environment variables for DB credentials, JWT secret, API keys

The frontend is React.

- calls backend APIs
- gets built into static assets
- may need environment-specific API base URL

Before AWS, it likely runs as:

- backend on localhost:8080
- frontend on localhost:3000 or 5173
- local DB in Docker or installed locally

That local setup is good for development, but AWS introduces questions local machines hide:

- where does the database live?
- how does the frontend find the backend?
- where are secrets stored?
- how do you deploy safely?
- how do you scale?
- how do you recover from instance or container loss?

---

4. What Must Be Done Before Any AWS Deployment

This section is critical. Whether you choose EC2, ECS, or EKS, these preparations matter.

4.1 Separate Build-Time and Run-Time Configuration

Your app should not hardcode environment details.

Backend examples:

- SPRINGPROFILESACTIVE
- DB_HOST
- DB_PORT
- DB_NAME
- DB_USERNAME
- DB_PASSWORD
- JWT_SECRET
- REDIS_HOST

Frontend examples:

- API base URL
- feature flags
- analytics IDs

The goal:

- local, staging, and prod should use the same code artifact
- only config should change

How This Looks in Spring Boot

```
[yaml]
 # application.yml - single file, environment-driven
 spring:
   datasource:
     url: jdbc:postgresql://${DB_HOST:localhost}:${DB_PORT:5432}/${DB_NAME:myapp}
     username: ${DB_USERNAME:postgres}
```

```
      password: ${DB_PASSWORD:postgres}
    redis:
      host: ${REDIS_HOST:localhost}
      port: ${REDIS_PORT:6379}

  server:
    port: ${SERVER_PORT:8080}

  app:
    jwt-secret: ${JWT_SECRET:dev-secret-do-not-use-in-prod}
```

This means:

- locally it falls back to localhost defaults
- on AWS you inject real values via environment variables, Secrets Manager, or Parameter Store
- the same JAR or Docker image works everywhere

How This Looks in React

Create a .env.production file:

```
[text]
REACT_APP_API_BASE_URL=https://api.yourapp.com
```

Or for Vite:

```
[text]
VITE_API_BASE_URL=https://api.yourapp.com
```

Then in code:

```
[javascript]
const API_BASE = import.meta.env.VITE_API_BASE_URL || 'http://localhost:8080';
```

This URL changes per environment at build time.

4.2 Decide How the Frontend Will Be Served

For a React SPA, there are two common models:

Model A: Static Hosting

- build React into static assets
- upload to S3
- serve through CloudFront

This is usually the best model for a normal SPA.

Model B: Frontend Container

- build React app
- copy static output into Nginx image
- run that container on EC2, ECS, or EKS

This is valid if:

- the organization wants everything in containers
- frontend is SSR or has server logic
- deployment model is standardized around containers

For a plain React SPA, static hosting is usually simpler and cheaper.

4.3 Externalize State

Do not keep important state inside the app container or inside one EC2 machine.

Use managed services where possible:

- RDS for PostgreSQL/MySQL
- ElastiCache for Redis
- S3 for file storage

- SQS/EventBridge for asynchronous workflows

If you keep state inside your compute layer, scaling and recovery become hard.

4.4 Containerize the Backend Even If You Start with EC2

Even if your first deployment is EC2, having a Docker image is a strong move.

Why?

- consistent runtime
- easier migration to ECS/EKS later
- easier local parity
- easier CI/CD

Backend Dockerfile (Spring Boot)

```
[dockerfile]
 # ---- Build Stage ----
 FROM maven:3.9-eclipse-temurin-21 AS build
 WORKDIR /app
 COPY pom.xml .
 RUN mvn dependency:go-offline          # cache dependencies
 COPY src ./src
 RUN mvn clean package -DskipTests      # produce JAR

 # ---- Runtime Stage ----
 FROM eclipse-temurin:21-jre-alpine
 WORKDIR /app
 COPY --from=build /app/target/*.jar app.jar

 # Non-root user for security
 RUN addgroup -S appgroup && adduser -S appuser -G appgroup
 USER appuser

 EXPOSE 8080
 ENTRYPOINT ["java", "-jar", "app.jar"]
```

Why multi-stage?

- build tools stay out of the runtime image
- final image is smaller and more secure
- no source code ships to production

Frontend Dockerfile (React/Vite with Nginx)

```
[dockerfile]
 # ---- Build Stage ----
 FROM node:20-alpine AS build
 WORKDIR /app
 COPY package*.json ./
 RUN npm ci
 COPY . .
 RUN npm run build                       # produces /app/dist or /app/build

 # ---- Serve Stage ----
 FROM nginx:alpine
 COPY --from=build /app/dist /usr/share/nginx/html
 COPY nginx.conf /etc/nginx/conf.d/default.conf
 EXPOSE 80
 CMD ["nginx", "-g", "daemon off;"]
```

The nginx.conf should handle SPA routing:

```
[nginx]
 server {
     listen 80;
```

```
    root /usr/share/nginx/html;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;   # SPA fallback
    }

    location /api/ {
        proxy_pass http://backend-host:8080;  # only if co-located
    }
}
```

Remember: if React is hosted on S3 + CloudFront, you do not need this frontend Dockerfile at all.

docker-compose.yml for Local Development

```
[yaml]
version: '3.8'
services:
  backend:
    build: ./backend
    ports:
      - "8080:8080"
    environment:
      DB_HOST: db
      DB_PORT: 5432
      DB_NAME: myapp
      DB_USERNAME: postgres
      DB_PASSWORD: postgres
      REDIS_HOST: redis
    depends_on:
      - db
      - redis

  frontend:
    build: ./frontend
    ports:
      - "3000:80"

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: myapp
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    volumes:
      - pgdata:/var/lib/postgresql/data

  redis:
    image: redis:7-alpine

volumes:
  pgdata:
```

This gives your team a one-command local environment that mirrors production structure.

4.5 Add Health Endpoints

Spring Boot should expose health endpoints, ideally through Actuator.

You want at least:

- liveness-style signal: process is alive
- readiness-style signal: app can serve traffic

This matters because ALB, ECS, and Kubernetes all depend on health checks.

Spring Boot Actuator Setup

Add to pom.xml:

```
[xml]
 <dependency>
     <groupId>org.springframework.boot</groupId>
     <artifactId>spring-boot-starter-actuator</artifactId>
 </dependency>
```

Add to application.yml:

```
[yaml]
 management:
   endpoints:
     web:
       exposure:
         include: health,info
   endpoint:
     health:
       probes:
         enabled: true          # enables /actuator/health/liveness and /readiness
       show-details: never      # do not leak internals publicly
   server:
     port: ${MANAGEMENT_PORT:8080}  # can run on separate port for security
```

Now you have:

- /actuator/health - general health
- /actuator/health/liveness - Kubernetes liveness / ALB basic check
- /actuator/health/readiness - Kubernetes readiness / ALB deep check

How Each Platform Uses These

```
| Platform | Health endpoint used | Configured where |
| ALB | '/actuator/health' | Target group health check settings |
| ECS | '/actuator/health' | Task definition 'healthCheck' or ALB target group |
| EKS | '/actuator/health/liveness' and '/readiness' | Pod 'livenessProbe' and 'readinessProbe'
|
```

4.6 Handle Database Migrations

This is often forgotten until the first production deployment breaks.

Your schema must evolve with your code. Use a migration tool.

Flyway (most common with Spring Boot)

Add to pom.xml:

```
[xml]
 <dependency>
     <groupId>org.flywaydb</groupId>
     <artifactId>flyway-core</artifactId>
 </dependency>
```

Place SQL migrations in src/main/resources/db/migration/:

```
[text]
 V1__create_users_table.sql
 V2__add_email_column.sql
 V3__create_orders_table.sql
```

Spring Boot auto-runs Flyway on startup. Migrations are versioned and tracked.

Why this matters for AWS:

- on EC2, the first instance to start runs migrations

- on ECS, only one task should run migrations before others start (use init containers or a pre-deploy task)
- on EKS, use a Kubernetes Job or init container for migrations before the main Deployment rolls out

Never manually run DDL scripts against production via SSH.

4.7 Create a CI/CD Baseline

At minimum your pipeline should:

- run unit tests
- build backend artifact or image
- build frontend assets or image
- scan dependencies and images if possible
- publish artifact
- deploy to target environment

The deployment target changes by EC2/ECS/EKS, but the discipline should be there from day one.

---

5. Path 1: Deploying with EC2

EC2 means you are deploying onto virtual machines.

You manage:

- OS
- package/runtime installation
- process management
- patching
- scaling policy
- deployment scripts or deployment tooling

5.1 Story Version of EC2

The team says:

"We understand Linux servers. We want direct control. We are okay managing instances. We need the shortest path from local app to production."

So they choose EC2.

5.2 Recommended EC2 Architecture for Spring Boot + React

Option 1: Pragmatic Production Pattern

```
[text]
 Users
   ->
 Route 53
   ->
 CloudFront
   ->
 S3 (React static files)

 API calls
   ->
 ALB
   ->
 EC2 Auto Scaling Group in private subnets
   ->
 Spring Boot service
   ->
```

```
  RDS / ElastiCache / S3
```

This is often the best EC2-based architecture.

Reason:

- React does not need a VM if it is a static SPA
- backend uses EC2 where server control matters
- ALB gives health checks and multi-instance traffic distribution

Option 2: Everything on EC2

```
 [text]
  Users
    ->
  ALB
    ->
  EC2 instances running:
       - Nginx serving React build
       - Spring Boot JAR or Docker container
    ->
  RDS
```

This works but is usually less clean than putting React on S3 + CloudFront.

5.3 What You Actually Do from Local Code

Step 1: Build the frontend

You create a production build:

- npm install
- npm run build

This generates static files like:

- HTML
- CSS
- JS bundles

Step 2: Build the Spring Boot backend

You package the backend:

- Maven: mvn clean package
- Gradle: ./gradlew build

Now you have a JAR file.

Step 3: Create the EC2 machine image strategy

You have two ways:

Basic way

- launch instance
- SSH in
- install Java, Nginx, maybe Docker
- copy files
- configure systemd service

This is okay for learning, not ideal for mature production.

Better way

- use a launch template
- use user data or baked AMIs
- automate setup
- let Auto Scaling create identical instances

This is the professional pattern.

Step 4: Run Spring Boot as a service

Typical pattern:

- create Linux user for app
- place JAR under app directory
- create systemd unit
- inject environment variables
- start service on boot

Actual systemd Unit File

```ini
# /etc/systemd/system/myapp.service
[Unit]
Description=My Spring Boot Application
After=network.target

[Service]
Type=simple
User=appuser
Group=appuser
WorkingDirectory=/opt/myapp
ExecStart=/usr/bin/java -Xms512m -Xmx1024m -jar /opt/myapp/app.jar
EnvironmentFile=/opt/myapp/.env
Restart=on-failure
RestartSec=10
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

The /opt/myapp/.env file holds runtime config:

```text
SPRING_PROFILES_ACTIVE=prod
DB_HOST=myapp-db.xxxx.us-east-1.rds.amazonaws.com
DB_PORT=5432
DB_NAME=myapp
DB_USERNAME=myapp_user
DB_PASSWORD=retrieved-from-secrets-manager
JWT_SECRET=retrieved-from-secrets-manager
```

In production, prefer fetching secrets at startup from Secrets Manager rather than storing them in a file. You can use a bootstrap script that calls aws secretsmanager get-secret-value and writes the env file before starting the service.

Enable and Start

```bash
sudo systemctl daemon-reload
sudo systemctl enable myapp.service
sudo systemctl start myapp.service
sudo systemctl status myapp.service
# View logs
journalctl -u myapp.service -f
```

Now EC2 behaves like a stable service host.

Step 5: Put ALB in front

ALB does:

- HTTPS termination
- routing

- health checks
- traffic distribution across instances

If you run multiple backend instances, ALB targets them and only routes to healthy ones.

Step 6: Move backend into private subnets

Good production design:

- ALB in public subnets
- EC2 app instances in private subnets
- no public IPs on backend instances
- access outbound internet via NAT Gateway if needed

Step 7: Use RDS for the database

Do not keep PostgreSQL on the same EC2 server as your backend unless this is just a demo.

Use RDS so that:

- database lifecycle is separate from app lifecycle
- backups are easier
- high availability is possible
- scaling and patching improve

5.4 How Deployment Works on EC2

There are several models.

Model A: SSH-based deployment

- copy new JAR or Docker image to server
- restart service

Simple, but risky and less reproducible.

Model B: Blue-Green with Auto Scaling Groups

- create new launch template version or AMI
- start new instances with new app version
- ALB health checks validate them
- shift traffic
- drain old instances

This is much safer.

5.5 How Scaling Works on EC2

Scaling is at the instance level.

You use Auto Scaling Groups and define:

- min instances
- desired instances
- max instances
- scaling triggers

Triggers can be:

- CPU usage
- memory via CloudWatch custom metrics
- request count per target
- queue depth

Important caveat:

If your app stores user session state locally on one instance, horizontal scaling becomes painful.

Better patterns:

- stateless backend
- JWT or distributed session store

- Redis for shared session/cache state if needed

5.6 Security on EC2

You must think about:

- security groups for ALB and EC2
- IAM role attached to EC2 instances
- secret retrieval from Secrets Manager or Parameter Store
- OS patching
- SSH minimization or elimination via SSM Session Manager

A mature answer is:

"I avoid broad SSH access and prefer SSM Session Manager, private subnets, least-privilege security groups, and instance IAM roles instead of static credentials."

5.7 Operational Burden on EC2

This is the real trade-off.

You still own:

- machine patching
- JVM/runtime upgrades
- disk management
- instance replacement strategy
- deployment tooling
- log shipping and monitoring agents

EC2 is powerful because it gives control. EC2 is expensive in effort because it gives control.

5.8 When EC2 Is the Right Answer

Use EC2 when:

- you need full OS/runtime control
- you have a legacy or non-container-ready stack
- your team is comfortable with VM operations
- compliance or agent installation requires machine-level access
- the app is simple enough that container orchestration is unnecessary

---

6. Path 2: Deploying with ECS

ECS means your unit of deployment becomes the container, not the machine.

You still think about compute, but at a higher abstraction level.

With ECS on Fargate, you do not manage servers at all.

6.1 Story Version of ECS

The team says:

"We want the consistency of containers and easier deployments, but we do not want Kubernetes complexity."

That is a classic ECS team.

6.2 Best ECS Architecture for Spring Boot + React

Recommended Architecture

```
[text]
 Users
   ->
 Route 53
```

```
  ->
CloudFront
  ->
S3 (React static site)

API calls
  ->
ALB
  ->
ECS Service (Spring Boot containers on Fargate or EC2)
  ->
RDS / ElastiCache / S3 / SQS
```

This is the cleanest model for most business applications.

Alternative: Frontend Also on ECS

```
[text]
Users
  ->
ALB
  ->
ECS service 1: Nginx serving React build
ECS service 2: Spring Boot API
```

This is fine if the team wants full container standardization.

6.3 What Changes from Local Development

Now you create Docker images.

Backend container

The backend Docker image usually:

- starts from a JDK/JRE base image
- copies the Spring Boot JAR
- exposes port 8080
- starts the app with java -jar

Frontend container, if containerized

The frontend Docker image usually:

- builds the React app in one stage
- copies the static output into Nginx in another stage
- exposes port 80

This is where your local code becomes deployable infrastructure artifacts.

6.4 Core ECS Concepts You Must Understand

- Cluster: logical place where ECS workloads run
- Task Definition: blueprint of container config
- Task: running container set from task definition
- Service: keeps desired number of tasks alive
- Launch Type: EC2 or Fargate

For a Spring Boot API, a task definition includes:

- image URI from ECR
- CPU and memory
- port mapping
- environment variables
- secrets
- logging config
- task IAM role

6.5 ECS with Fargate vs ECS with EC2

ECS with Fargate

Use when:

- you want minimal ops
- app is stateless
- team wants containers without node management

Benefits:

- no EC2 fleet management
- no patching of worker nodes by you
- simple scaling model

Trade-offs:

- less low-level control
- can cost more than EC2 at sustained large scale

ECS with EC2

Use when:

- you want ECS scheduling but also node-level control
- you have special agent/runtime needs
- you want to optimize cost on large steady workloads

Trade-off:

- now you are back to managing worker instances

For most app teams, ECS on Fargate is the default starting point.

6.6 Step-by-Step Journey from Local Code to ECS

Step 1: Write Dockerfiles

You containerize backend and optionally frontend.

Step 2: Push images to ECR

ECR is your private container registry.

Typical flow:

- build image locally or in CI
- tag image
- push image to ECR

Actual ECR Commands

```
[bash]
 # 1. Authenticate Docker to ECR
 aws ecr get-login-password --region us-east-1 | \
   docker login --username AWS --password-stdin 123456789012.dkr.ecr.us-
 east-1.amazonaws.com

 # 2. Create repository (first time only)
 aws ecr create-repository --repository-name myapp-backend

 # 3. Build the image
 docker build -t myapp-backend:latest ./backend

 # 4. Tag for ECR
 docker tag myapp-backend:latest \
   123456789012.dkr.ecr.us-east-1.amazonaws.com/myapp-backend:v1.0.0

 # 5. Push
 docker push 123456789012.dkr.ecr.us-east-1.amazonaws.com/myapp-backend:v1.0.0
```

In CI/CD, this is automated. The version tag usually comes from the git commit SHA or a

semantic version.

Now AWS has a versioned artifact to deploy.

Step 3: Create the ECS task definition

For Spring Boot, define:

- container image
- port 8080
- memory and CPU
- health check path or container health command
- environment variables and secrets
- CloudWatch logs configuration

Actual Task Definition (JSON)

```
[json]
 {
   "family": "myapp-backend",
   "networkMode": "awsvpc",
   "requiresCompatibilities": ["FARGATE"],
   "cpu": "512",
   "memory": "1024",
   "executionRoleArn": "arn:aws:iam::123456789012:role/ecsTaskExecutionRole",
   "taskRoleArn": "arn:aws:iam::123456789012:role/myapp-backend-task-role",
   "containerDefinitions": [
     {
       "name": "myapp-backend",
       "image": "123456789012.dkr.ecr.us-east-1.amazonaws.com/myapp-backend:v1.0.0",
       "portMappings": [
         {
           "containerPort": 8080,
           "protocol": "tcp"
         }
       ],
       "environment": [
         { "name": "SPRING_PROFILES_ACTIVE", "value": "prod" },
         { "name": "DB_HOST", "value": "myapp-db.xxxx.us-east-1.rds.amazonaws.com" },
         { "name": "DB_PORT", "value": "5432" },
         { "name": "DB_NAME", "value": "myapp" }
       ],
       "secrets": [
         {
           "name": "DB_PASSWORD",
           "valueFrom": "arn:aws:secretsmanager:us-east-1:123456789012:secret:myapp/db-
 password"
         },
         {
           "name": "JWT_SECRET",
           "valueFrom": "arn:aws:secretsmanager:us-east-1:123456789012:secret:myapp/jwt-
 secret"
         }
       ],
       "healthCheck": {
         "command": ["CMD-SHELL", "curl -f http://localhost:8080/actuator/health || exit
 1"],
         "interval": 30,
         "timeout": 5,
         "retries": 3,
         "startPeriod": 60
       },
       "logConfiguration": {
         "logDriver": "awslogs",
```

```
          "options": {
            "awslogs-group": "/ecs/myapp-backend",
            "awslogs-region": "us-east-1",
            "awslogs-stream-prefix": "ecs"
          }
        },
        "essential": true
      }
    ]
  }
```

Key details to notice:

- executionRoleArn lets ECS pull the image and push logs
- taskRoleArn lets the application itself call AWS services like S3 or SQS
- secrets pulls values from Secrets Manager at task startup - they appear as environment variables in the container
- healthCheck keeps unhealthy containers from receiving traffic
- startPeriod gives Spring Boot time to initialize before health checks begin failing

Step 4: Create the ECS service

The ECS service ensures:

- desired number of tasks is running
- failed tasks are replaced
- deployment rollout is managed

Step 5: Attach to ALB

ALB routes traffic to the ECS service.

If multiple services exist:

- /api/* -> Spring Boot service
- / -> frontend service if frontend is containerized

Or, if React is on S3 + CloudFront:

- browser loads frontend from CloudFront
- frontend calls backend API domain served by ALB

Step 6: Put tasks in private subnets

Best practice:

- ALB public
- ECS tasks private
- RDS private

The tasks do not need public inbound reachability.

Step 7: Configure service autoscaling

ECS can scale tasks based on:

- CPU
- memory
- ALB request count per target
- custom CloudWatch metrics

This is one of the biggest gains over hand-managed EC2 deployment.

6.7 How Deployment Works on ECS

Deployment flow:

```
[text]
 Code change
   ->
 CI builds Docker image
   ->
```

```
image pushed to ECR
  ->
ECS service updated to new task definition
  ->
new tasks start
  ->
ALB health checks pass
  ->
old tasks drained and removed
```

This is much more reproducible than copying JARs to VMs.

6.8 Security on ECS

Key ideas:

- task IAM roles, not static AWS keys
- Secrets Manager / Parameter Store for secrets
- security groups attached to tasks or ENIs depending on mode
- private subnets for backend tasks

One particularly important concept:

- task role gives AWS permissions to the running application
- execution role allows ECS to pull image and send logs

Do not confuse these two.

6.9 Operational Burden on ECS

Compared with EC2, ECS removes or reduces:

- app packaging inconsistency
- server snowflake problems
- host-level deployment complexity
- some scaling complexity

With Fargate, it also removes node management.

But you still own:

- container image quality
- task sizing
- deployment safety
- secrets handling
- app observability
- database architecture

6.10 When ECS Is the Right Answer

Use ECS when:

- you want containers without Kubernetes overhead
- you are AWS-first
- your platform needs are moderate, not highly customized
- your team wants faster operational maturity than raw EC2

For many companies, ECS is the best practical answer for Spring Boot microservices on AWS.

---

7. Path 3: Deploying with EKS

EKS means you are now operating on top of Kubernetes.

AWS manages the control plane, but you still operate substantial platform complexity.

7.1 Story Version of EKS

The company says:

"We need Kubernetes itself, not just containers. We want standardized platform abstractions across many services and teams."

That is the right reason to choose EKS.

The wrong reason is:

"Kubernetes is popular."

7.2 Recommended EKS Architecture for Spring Boot + React

Most Practical Model

```
[text]
Users
  ->
Route 53
  ->
CloudFront
  ->
S3 (React static site)

API calls
  ->
ALB Ingress
  ->
EKS Deployment/Service for Spring Boot
  ->
RDS / ElastiCache / S3 / SQS
```

Full Kubernetes Model

```
[text]
Users
  ->
ALB Ingress Controller / AWS Load Balancer Controller
  ->
Frontend Deployment + Service
  ->
Backend Deployment + Service
  ->
Stateful external services
```

Again, for a pure React SPA, S3 + CloudFront is usually simpler than running frontend pods.

7.3 What You Need Beyond Containers

EKS requires Kubernetes resources such as:

- Namespace
- Deployment
- Service
- Ingress
- ConfigMap
- Secret
- HorizontalPodAutoscaler
- ServiceAccount

If your team is not comfortable with these concepts, EKS will slow you down.

7.4 Step-by-Step Journey from Local Code to EKS

Step 1: Containerize the applications

Same as ECS.

You still build Docker images and push to ECR.

Step 2: Create Kubernetes manifests or Helm charts

For Spring Boot backend, you define:

- Deployment with replica count
- Service exposing pods internally
- Ingress for external HTTP routing
- ConfigMap for non-secret configuration
- Secret or external secret integration for credentials

Actual Kubernetes Manifests

Namespace:

```
[yaml]
apiVersion: v1
kind: Namespace
metadata:
  name: myapp
```

ConfigMap - non-secret config:

```
[yaml]
apiVersion: v1
kind: ConfigMap
metadata:
  name: myapp-backend-config
  namespace: myapp
data:
  SPRING_PROFILES_ACTIVE: "prod"
  DB_HOST: "myapp-db.xxxx.us-east-1.rds.amazonaws.com"
  DB_PORT: "5432"
  DB_NAME: "myapp"
```

Secret (or use External Secrets Operator to pull from Secrets Manager):

```
[yaml]
apiVersion: v1
kind: Secret
metadata:
  name: myapp-backend-secrets
  namespace: myapp
type: Opaque
stringData:
  DB_PASSWORD: "your-db-password"      # in practice, sealed or external
  JWT_SECRET: "your-jwt-secret"
```

ServiceAccount with IRSA:

```
[yaml]
apiVersion: v1
kind: ServiceAccount
metadata:
  name: myapp-backend-sa
  namespace: myapp
  annotations:
    eks.amazonaws.com/role-arn: arn:aws:iam::123456789012:role/myapp-backend-role
```

Deployment:

```
[yaml]
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp-backend
```

```yaml
  namespace: myapp
spec:
  replicas: 3
  selector:
    matchLabels:
      app: myapp-backend
  template:
    metadata:
      labels:
        app: myapp-backend
    spec:
      serviceAccountName: myapp-backend-sa
      containers:
        - name: myapp-backend
          image: 123456789012.dkr.ecr.us-east-1.amazonaws.com/myapp-backend:v1.0.0
          ports:
            - containerPort: 8080
          envFrom:
            - configMapRef:
                name: myapp-backend-config
            - secretRef:
                name: myapp-backend-secrets
          resources:
            requests:
              cpu: "250m"
              memory: "512Mi"
            limits:
              cpu: "500m"
              memory: "1024Mi"
          livenessProbe:
            httpGet:
              path: /actuator/health/liveness
              port: 8080
            initialDelaySeconds: 60
            periodSeconds: 10
          readinessProbe:
            httpGet:
              path: /actuator/health/readiness
              port: 8080
            initialDelaySeconds: 30
            periodSeconds: 5
```

Service:

```yaml
apiVersion: v1
kind: Service
metadata:
  name: myapp-backend
  namespace: myapp
spec:
  selector:
    app: myapp-backend
  ports:
    - port: 80
      targetPort: 8080
  type: ClusterIP
```

Ingress (using AWS Load Balancer Controller):

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
```

```
    name: myapp-backend-ingress
    namespace: myapp
    annotations:
      kubernetes.io/ingress.class: alb
      alb.ingress.kubernetes.io/scheme: internet-facing
      alb.ingress.kubernetes.io/target-type: ip
      alb.ingress.kubernetes.io/listen-ports: '[{"HTTPS": 443}]'
      alb.ingress.kubernetes.io/certificate-arn: arn:aws:acm:us-
  east-1:123456789012:certificate/xxxxx
      alb.ingress.kubernetes.io/healthcheck-path: /actuator/health
  spec:
    rules:
      - host: api.yourapp.com
        http:
          paths:
            - path: /
              pathType: Prefix
              backend:
                service:
                  name: myapp-backend
                  port:
                    number: 80
```

HorizontalPodAutoscaler:

```
  [yaml]
  apiVersion: autoscaling/v2
  kind: HorizontalPodAutoscaler
  metadata:
    name: myapp-backend-hpa
    namespace: myapp
  spec:
    scaleTargetRef:
      apiVersion: apps/v1
      kind: Deployment
      name: myapp-backend
    minReplicas: 2
    maxReplicas: 10
    metrics:
      - type: Resource
        resource:
          name: cpu
          target:
            type: Utilization
            averageUtilization: 70
```

This is significantly more YAML than ECS requires. That is the trade-off.

Step 3: Create the EKS cluster and worker capacity

You choose:

- managed node groups
- self-managed nodes
- Fargate profiles for some pods

Managed node groups are the usual default.

Step 4: Install ingress and cluster add-ons

Typical EKS platform components:

- AWS Load Balancer Controller
- metrics server
- external-dns maybe
- cluster autoscaler or Karpenter

- logging/monitoring stack
- external secrets solution maybe

This is where EKS becomes a platform, not just a deployment target.

Step 5: Deploy the Spring Boot app

The Deployment ensures the desired number of pods.

Kubernetes rolling updates can:

- create new pods
- wait for readiness
- gradually terminate old pods

This is conceptually similar to ECS service deployment but with more knobs and more complexity.

Step 6: Expose externally through Ingress

On AWS, a common pattern is:

- Kubernetes Ingress resource
- AWS Load Balancer Controller provisions ALB
- ALB routes to Kubernetes services

Step 7: Add autoscaling

You can scale at multiple levels:

- Horizontal Pod Autoscaler scales pod count
- Cluster Autoscaler or Karpenter scales nodes
- VPA may adjust resource requests in some setups

This flexibility is powerful, but it requires disciplined resource tuning.

7.5 Security on EKS

Security becomes broader.

You now manage:

- Kubernetes RBAC
- namespace isolation
- pod security posture
- image policies
- network policies if used
- IAM integration for workloads

The key AWS concept here is IRSA or the newer EKS Pod Identity.

For example:

- backend pod needs S3 read/write
- instead of static keys, bind pod identity to IAM permissions

This is the Kubernetes equivalent of good task/instance role hygiene.

7.6 Operational Burden on EKS

This is where many teams underestimate the cost.

Even with managed control plane, you still own:

- node upgrades
- cluster add-ons
- ingress controller behavior
- resource requests/limits
- pod disruption policies
- DNS and certificate integration
- observability stack
- security policies
- upgrade testing across Kubernetes versions

EKS is not "ECS but more advanced." It is a different operational category.

7.7 When EKS Is the Right Answer

Use EKS when:

- Kubernetes expertise exists or is strategically important
- multiple teams need a shared platform model
- you need Helm, operators, CRDs, advanced scheduling, or broader K8s ecosystem tools
- platform portability matters

If your goal is simply "run Spring Boot containers on AWS," EKS is often unnecessary.

---

8. How the React Frontend Fits in Each Model

This needs separate attention because teams often overcomplicate frontend hosting.

8.1 Best Default for a React SPA

Best default:

- build React static files
- host on S3
- serve through CloudFront

Why?

- cheap
- scalable
- globally cacheable
- no app servers needed
- clean separation from backend compute choice

Then choose EC2, ECS, or EKS only for the backend API.

8.2 When to Put Frontend on EC2/ECS/EKS

You may run frontend on compute if:

- you use SSR framework behavior
- you need runtime rendering or middleware
- organizational deployment standards require containerized frontend
- you want same deployment pattern for frontend and backend

8.3 Practical Recommendation

For your scenario, a strong default architecture is:

```
[text]
 React -> S3 + CloudFront
 Spring Boot API -> EC2 or ECS or EKS
 Database -> RDS
 Files -> S3
 Cache -> ElastiCache
 Secrets -> Secrets Manager
 DNS -> Route 53
```

This lets you compare EC2/ECS/EKS mainly for the backend layer where the compute trade-off actually matters.

8.4 Solving the Frontend-Backend Connectivity Problem

This is a gap many guides skip. When React is on CloudFront and the API is on ALB, they are on different domains. That means CORS (Cross-Origin Resource Sharing) becomes relevant.

The Problem

```
[text]
 React on:  https://app.yourcompany.com   (CloudFront)
 API on:    https://api.yourcompany.com   (ALB)

 Browser makes a request from app.yourcompany.com to api.yourcompany.com.
 This is a cross-origin request.
 Browser blocks it unless the API explicitly allows it via CORS headers.
```

Solution 1: CORS in Spring Boot (Most Common)

```
[java]
 @Configuration
 public class CorsConfig implements WebMvcConfigurer {

     @Override
     public void addCorsMappings(CorsRegistry registry) {
         registry.addMapping("/api/**")
                 .allowedOrigins(
                     "https://app.yourcompany.com",
                     "http://localhost:3000"           // local dev
                 )
                 .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
                 .allowedHeaders("*")
                 .allowCredentials(true)
                 .maxAge(3600);
     }
 }
```

Make the allowed origins configurable via environment variable for different environments:

```
[yaml]
 # application.yml
 app:
   cors:
     allowed-origins: ${CORS_ALLOWED_ORIGINS:http://localhost:3000}
```

Solution 2: Same Domain via CloudFront (Avoids CORS Entirely)

An elegant alternative: put both frontend and API behind the same CloudFront distribution.

```
[text]
 https://app.yourcompany.com/         -> CloudFront -> S3 (React)
 https://app.yourcompany.com/api/*    -> CloudFront -> ALB (Spring Boot)
```

CloudFront path-based routing:

- Default behavior: serves S3 static files
- /api/* behavior: forwards to ALB origin

Benefits:

- no CORS needed (same origin)
- single TLS certificate
- CloudFront handles both static assets and API proxying

Trade-off:

- slightly more CloudFront configuration
- API responses pass through CloudFront (ensure caching is disabled for API paths)

Solution 3: API Gateway Custom Domain

If using API Gateway instead of ALB:

- set custom domain api.yourcompany.com on API Gateway
- API Gateway handles CORS configuration natively

Recommendation

For most teams: Solution 2 (same CloudFront distribution) is cleanest. If separate domains are required: Solution 1 (Spring Boot CORS config) works fine.

---

9. Networking, Security, Scaling, and Observability Across All Three

No matter which compute path you choose, these patterns remain important.

9.1 VPC Layout

Good baseline:

- two or more AZs
- public subnets for ALB and maybe NAT Gateway
- private subnets for app compute
- private subnets for database

This reduces exposure and improves resilience.

9.2 Security Pattern

Minimum good pattern:

- HTTPS at ALB
- backend compute in private subnets
- security groups with least privilege
- IAM roles for workloads
- secrets in Secrets Manager or Parameter Store
- no hardcoded credentials

9.3 Scaling Pattern

- EC2 scales by adding/removing instances
- ECS scales by adding/removing tasks
- EKS scales by adding/removing pods and maybe nodes

The more abstract the platform, the more granular the scaling unit becomes.

9.4 Logging and Metrics

You need:

- application logs
- request metrics
- error rates
- JVM metrics
- health check insight
- tracing if possible

Typical AWS-friendly observability:

- CloudWatch logs
- CloudWatch metrics/alarms
- OpenTelemetry + managed tracing stack if available
- dashboards for latency, error rate, CPU, memory, saturation

9.5 Deployment Safety

No matter what you use:

- prefer rolling or blue-green over ad-hoc restarts
- verify readiness checks
- keep artifacts immutable

- have rollback strategy

9.6 Service Discovery

When you have multiple backend services that need to find each other:

```
| Platform | Service Discovery Method | How It Works |
| EC2 | ALB/Route 53 or manual config | Services discover each other via DNS or load balancer
endpoints |
| ECS | AWS Cloud Map | ECS auto-registers tasks into Cloud Map. Other services query DNS names
like ‘backend.myapp.local‘ |
| EKS | Kubernetes DNS (CoreDNS) | Services get stable DNS names like ‘myapp-
backend.myapp.svc.cluster.local‘ automatically |
```

This matters once you move beyond a single backend service.

---

10. Cost Comparison: EC2 vs ECS vs EKS

This is a question interviewers and managers both ask. Numbers below are approximate US East 1 prices to build intuition.

10.1 Base Platform Cost

```
[text]
 EC2:
   No platform fee. You pay only for the instances you run.
   t3.medium (2 vCPU, 4 GB): ~$30/month On-Demand

 ECS on Fargate:
   No cluster fee. You pay per task vCPU + memory.
   0.5 vCPU, 1 GB task running 24/7: ~$18/month
   1 vCPU, 2 GB task running 24/7: ~$36/month

 ECS on EC2:
   Same as EC2 instance pricing. ECS scheduling is free.

 EKS:
   $0.10/hour for control plane = ~$73/month (just for the cluster existing)
   + worker node costs (EC2 instances or Fargate)
   So EKS always has a baseline cost even before any workloads run.
```

10.2 Realistic Small App Cost (2 backend replicas + database)

```
[text]
 EC2 path:
   2x t3.medium                = ~$60/month
   ALB                         = ~$22/month
   RDS db.t3.micro             = ~$15/month
   NAT Gateway                 = ~$35/month
   Total                        $132/month

 ECS Fargate path:
   2 tasks (0.5 vCPU, 1 GB)    = ~$36/month
   ALB                         = ~$22/month
   RDS db.t3.micro             = ~$15/month
   NAT Gateway                 = ~$35/month
   Total                        $108/month

 EKS path:
   EKS control plane           = ~$73/month
   2x t3.medium worker nodes   = ~$60/month
   ALB                         = ~$22/month
```

```
    RDS db.t3.micro             = ~$15/month
    NAT Gateway                 = ~$35/month
    Total                        $205/month
```

10.3 Cost Insight

```
  [text]
   Key takeaway:

    ECS Fargate is often cheapest for small-medium workloads.
    EC2 becomes cheaper at scale when you use Reserved Instances or Savings Plans.
    EKS has a fixed overhead ($73/month) that only makes sense when many services share the
  cluster.

    A single-service team paying $73/month just for the K8s control plane
    when ECS has zero platform fee - that is a common cost mistake.

    EKS cost justification usually starts at 5-10+ services sharing one cluster.
```

10.4 Hidden Costs to Watch

- NAT Gateway: $32/month + $0.045/GB. Use VPC endpoints for S3/DynamoDB/Secrets Manager.
- Cross-AZ traffic: $0.02/GB round trip. Co-locate chatty services or use caching.
- ALB proliferation: Each ALB costs ~$22/month minimum. Consolidate with path-based routing.
- ECR storage: $0.10/GB/month. Clean up old images with lifecycle policies.
- CloudWatch logs: $0.50/GB ingested. Log wisely, not everything.

---

11. Migration Journey: EC2 -> ECS -> EKS

This progression matters because many teams do not start with the final platform.

11.1 Local -> EC2

Best when:

- learning phase
- small team
- simple deployment needs
- server familiarity is high

What improves first:

- external DB
- ALB
- Auto Scaling Group
- proper health checks

11.2 EC2 -> ECS

This is the most common maturity jump.

What usually drives it:

- server drift
- painful deployments
- need for immutable artifacts
- desire for autoscaling and faster rollout

What changes technically:

- JAR deployment becomes Docker image deployment
- ASG-centric thinking becomes service/task-centric thinking

- host-level config becomes task definition config

11.3 ECS -> EKS

This should happen only when platform needs justify it.

Drivers may include:

- many teams and many services
- Kubernetes standardization
- operator/CRD ecosystem need
- advanced traffic and policy controls
- GitOps or service mesh adoption

If those are absent, staying on ECS is usually the better decision.

---

12. How to Choose in Real Life

12.1 Choose EC2 If

- you need OS-level control
- app is not yet containerized and you need speed of first deployment
- team is strong in Linux/VM ops and weak in containers/orchestration
- workload needs custom agents, drivers, or machine-level tuning

12.2 Choose ECS If

- you want containers with low operational overhead
- you are AWS-centric
- team does not want Kubernetes complexity
- you want the best balance of simplicity and operational maturity

12.3 Choose EKS If

- Kubernetes is a platform requirement
- multiple teams need advanced orchestration capabilities
- your org already operates K8s well
- ecosystem flexibility matters more than simplicity

12.4 My Practical Recommendation for Your Spring Boot + React App

If you are starting from local code today and want a strong, realistic AWS path:

Recommendation order

1. Put React on S3 + CloudFront
2. Put PostgreSQL on RDS
3. Containerize Spring Boot
4. Deploy Spring Boot on ECS Fargate unless you have a strong reason for EC2 or EKS

Why this is strong:

- low ops
- clean architecture
- good scalability
- simple rollback path
- easy to explain in interviews

Choose EC2 instead if you explicitly need machine control. Choose EKS instead if Kubernetes is truly required by the org/platform.

---

13. Interview-Ready Answer Framework

If an interviewer asks:

"You have a local Spring Boot backend and React frontend. How would you leverage EC2, ECS, or EKS on AWS?"

Here is the structure of a strong answer.

13.1 Start With Workload Framing

Say:

```
[text]
 First I would separate frontend hosting, backend compute, and stateful services.
 For a React SPA, I would usually host static assets on S3 + CloudFront.
 For the Spring Boot API, I would choose EC2, ECS, or EKS based on the required level of
 control versus operational complexity.
 Database would be externalized to RDS, cache to ElastiCache if needed, and secrets to
 Secrets Manager.
```

13.2 Then Compare the Three Clearly

Say:

```
[text]
 If I need full server control or have legacy constraints, I would use EC2 behind an ALB
 with Auto Scaling Groups.

 If I want containers with the lowest operational burden on AWS, I would containerize
 Spring Boot, push the image to ECR, and run it on ECS Fargate behind an ALB.

 If the organization is standardized on Kubernetes and needs Helm, operators, or advanced
 workload policies, I would run the same container on EKS with Deployments, Services, and
 ALB-backed Ingress.
```

13.3 Then Finish With Trade-Offs

Say:

```
[text]
 My default recommendation for a normal business application would be React on
 S3/CloudFront and Spring Boot on ECS Fargate, because it gives container benefits without
 Kubernetes overhead. I would move to EC2 for machine control, or EKS when Kubernetes
 capabilities are a real platform requirement.
```

That is a senior answer because it is not tool-recitation. It is architecture reasoning.

---

14. Final Revision Sheet

EC2 in One Line

Run your app on VMs when you need maximum control and accept the operational cost of managing machines.

ECS in One Line

Run your app as containers in an AWS-native way when you want strong operational simplicity without Kubernetes.

EKS in One Line

Run your app on Kubernetes when the Kubernetes ecosystem and platform capabilities themselves are required.

What Changes as You Move EC2 -> ECS -> EKS

```
| Dimension | EC2 | ECS | EKS |
| Deployment unit | server or process | container task | pod/workload |
| You manage | OS + app | container + service config | Kubernetes platform + workloads |
| Scaling unit | instance | task | pod/node |
| Operational complexity | low to medium initially, higher over time | medium | high |
| Best fit | VM control | pragmatic AWS containers | Kubernetes platform needs |
```

Gold Standard Architecture for Most Spring Boot + React Apps

```text
React SPA -> S3 + CloudFront
Spring Boot API -> ECS Fargate behind ALB
Database -> RDS
Secrets -> Secrets Manager
Files -> S3
Metrics/Logs -> CloudWatch
DNS -> Route 53
```

Gold Standard Decision Sentence

```text
I choose EC2 when I need machine-level control, ECS when I want the most pragmatic AWS
container platform, and EKS when Kubernetes capabilities are strategically necessary. For
a standard Spring Boot API with a React SPA, my default would usually be React on
S3/CloudFront and the backend on ECS Fargate unless a strong constraint pushes me toward
EC2 or EKS.
```

Part 06: AWS Networking Through Story Mode: How Your App Actually Talks

Source file: AWS-06-Networking-Story-Mode.md

> You have a Spring Boot backend, a React frontend, and a PostgreSQL database running on your laptop. Everything just works. Then you move to AWS and suddenly nothing can talk to anything. This guide explains AWS networking the way it actually matters - how your components find each other, how users reach your app, and what every piece of the network does in plain language.

---

1. How It Works on Your Laptop (The Baseline)

Before AWS, your app works like this:

```text
React frontend  (localhost:3000)
      |
      | HTTP calls to localhost:8080/api/*
      v
Spring Boot backend  (localhost:8080)
      |
      | JDBC connection to localhost:5432
      v
PostgreSQL  (localhost:5432)
```

Everything is on one machine. Everything uses localhost. No network complexity.

Why does this "just work"?

- all processes share the same machine
- localhost means "this computer"

- port numbers separate the processes (3000, 8080, 5432)
- no firewalls between them
- no DNS needed
- no routing needed

That is the baseline. Now let's see what changes.

---

2. What Changes When You Move to AWS

On AWS, your components run on separate machines inside a private network.

```
[text]
 React        -> runs on S3 + CloudFront (or a container)
 Spring Boot  -> runs on EC2 / ECS / EKS (a different machine)
 PostgreSQL   -> runs on RDS (yet another machine)
```

Now localhost is gone. Each component has its own IP address. They are no longer neighbors. They need:

- a network to live in (VPC)
- addresses to find each other (private IPs, DNS names)
- doors to allow traffic through (security groups)
- a way for users from the internet to reach the frontend and API
- a way for the backend to call external APIs if needed

Let's build this up, one piece at a time.

---

3. VPC - Your Private Building in the Cloud

3.1 Real-Life Analogy

Think of a VPC as a private office building.

- the building has walls - nothing outside can see in
- inside, there are floors and rooms
- you control who enters, who leaves, and which rooms connect to which
- you decide the address system for the rooms (IP ranges)

On AWS, a VPC is your isolated network. When you create one, you say:

"I want a private network with this range of IP addresses."

3.2 CIDR - The Address System

When you create a VPC, you choose a CIDR block like 10.0.0.0/16.

Real-life version:

```
[text]
 Think of it like apartment numbering.

 10.0.0.0/16 means:
   "I own all addresses from 10.0.0.0 to 10.0.255.255"
   That is 65,536 addresses.

 Why does this matter?
   Every EC2 instance, every ECS task, every RDS instance, every ALB
   gets an IP address from this range.

 If you run out of IPs, you cannot launch more resources.
```

Common choices:

```text
[text]
 10.0.0.0/16     -> huge, 65K addresses, good for production
 10.0.0.0/20     -> moderate, 4K addresses, good for smaller environments
 172.16.0.0/16   -> also common
```

You do not need to memorize CIDR math. Just know:

- smaller number after / = more addresses
- plan ahead so you do not run out
- different VPCs should not overlap if you ever want to connect them

3.3 One VPC per Environment is Normal

Typical pattern:

```text
[text]
 VPC: myapp-dev     (10.1.0.0/16)
 VPC: myapp-staging (10.2.0.0/16)
 VPC: myapp-prod    (10.3.0.0/16)
```

Each environment is fully isolated at the network level.

---

4. Subnets - Rooms Inside Your Building

4.1 What Are Subnets

A subnet is a slice of your VPC's IP range, placed in a specific Availability Zone.

Real-life version:

```text
[text]
 VPC = the building
 Subnet = a room on a specific floor

 Each room (subnet) has:
   - a portion of the building's addresses
   - a location (Availability Zone)
   - rules about who can enter and leave (route tables)
```

4.2 Public vs Private Subnets

This is the single most important networking concept.

Public Subnet

A subnet is public if its route table has a route to the Internet Gateway.

Real-life: a room with a door to the street. Anyone can walk in if the door is open.

Used for:

- load balancers (ALB/NLB)
- NAT Gateways
- bastions if unavoidable

Private Subnet

A subnet is private if its route table has no route to the Internet Gateway.

Real-life: an internal room with no street-facing door. The only way in is through another room.

Used for:

- your Spring Boot backend (EC2, ECS tasks, EKS pods)
- your database (RDS, ElastiCache)
- anything that should not be directly reachable from the internet

Why This Matters for Your App

```text
Your React frontend              -> served from S3/CloudFront (not in VPC at all)
Your Spring Boot backend          -> private subnet (no internet exposure)
Your PostgreSQL database (RDS)    -> private subnet (definitely no internet exposure)
Your ALB (load balancer)          -> public subnet (must receive internet traffic)
```

The ALB sits in the public subnet like a reception desk. It faces the outside world. Users talk to the ALB. The ALB then forwards requests to your backend in the private subnet.

Your backend never directly faces the internet.

4.3 Multi-AZ Layout

For resilience, you spread subnets across multiple Availability Zones.

```text
                        VPC: 10.0.0.0/16

           AZ: us-east-1a        AZ: us-east-1b

        Public Subnet         Public Subnet
        10.0.1.0/24           10.0.2.0/24
        (ALB, NAT GW)         (ALB, NAT GW)

        Private Subnet        Private Subnet
        10.0.10.0/24          10.0.20.0/24
        (Spring Boot)         (Spring Boot)

        Private Subnet        Private Subnet
        10.0.100.0/24         10.0.200.0/24
        (RDS, ElastiCache)    (RDS, ElastiCache)
```

Why two AZs?

- if one data center has a problem, the other keeps running
- ALB automatically routes to the healthy AZ
- RDS can failover to the standby in the other AZ

This is the minimum production layout.

---

5. How Your Components Talk to Each Other

This is the question: "On my laptop, Spring Boot calls localhost:5432 for the database. What happens on AWS?"

5.1 Backend -> Database

On AWS, your Spring Boot backend calls the database using a DNS name, not localhost.

```text
Local:
  spring.datasource.url=jdbc:postgresql://localhost:5432/myapp

AWS:
  spring.datasource.url=jdbc:postgresql://myapp-db.abc123.us-
east-1.rds.amazonaws.com:5432/myapp
```

That long hostname is the RDS endpoint. AWS creates it automatically when you create the database.

How does it work underneath?

```text
1. Spring Boot says: "I need to connect to myapp-db.abc123.us-east-1.rds.amazonaws.com"
```

```
  2. DNS resolves that name to a private IP like 10.0.100.42
  3. The network routes the request within the VPC
  4. Security group on RDS checks: "Is this caller allowed on port 5432?"
  5. If yes, connection is established
```

All of this happens inside the VPC. The database is never exposed to the internet.

5.2 Backend -> Cache (Redis/ElastiCache)

Same pattern:

```
[text]
 Local:
   spring.redis.host=localhost

 AWS:
   spring.redis.host=myapp-cache.abc123.cache.amazonaws.com
```

ElastiCache creates a DNS endpoint. Your backend resolves it to a private IP inside the VPC. Traffic never leaves the private network.

5.3 Frontend -> Backend

This depends on how you host the frontend.

If React is on S3 + CloudFront (most common pattern)

The React app runs in the user's browser, not on AWS compute.

```
[text]
 1. User opens https://app.yourcompany.com
 2. CloudFront serves the React HTML/JS/CSS from S3
 3. React app loads in the user's browser
 4. React makes API calls to https://api.yourcompany.com/api/orders
 5. That request goes over the internet to your ALB
 6. ALB forwards it to your Spring Boot backend in the private subnet
 7. Backend responds. ALB sends response back to the browser.
```

Important realization:

The frontend does not talk to the backend inside AWS. The user's browser talks to the backend through the internet and ALB.

```
[text]
   User's Browser (React running here)
         |
         | HTTPS request to api.yourcompany.com
         v
   Internet
         |
         v
   ALB (public subnet)
         |
         | HTTP forward to backend
         v
   Spring Boot (private subnet)
         |
         v
   RDS (private subnet)
```

If React is in a container (ECS/EKS alongside backend)

Then the frontend container serves static files. Users still hit it through the ALB. Internally, the frontend container does not call the backend - the browser does.

The key insight: React is client-side. The API call always originates from the user's browser, not from the frontend server.

5.4 Backend -> Backend (Microservices)

If you have multiple backend services:

```
[text]
 Service A needs to call Service B
```

Options on AWS:

```
| Method | How it works | When to use |
| Through ALB (internal) | Internal ALB routes to Service B | Simple, works on EC2/ECS/EKS |
| AWS Cloud Map | ECS registers services with DNS names like ‘service-b.myapp.local‘ | ECS
service-to-service |
| Kubernetes DNS | Service B is reachable at ‘service-b.myapp.svc.cluster.local‘ | EKS |
| Direct IP/hostname | Hardcoded or config-driven | Small scale, not recommended |
```

For your first app with one backend service, this does not apply yet. But it matters when you grow.

5.5 Backend -> External APIs

Your Spring Boot might call third-party APIs (payment gateway, email provider, etc.).

```
[text]
 Spring Boot (private subnet)
       |
       | "I need to call https://api.stripe.com"
       |
       | But I am in a private subnet - I have no internet route!
       v
   NAT Gateway (public subnet)
       |
       | NAT translates private IP to public IP
       v
   Internet -> api.stripe.com
       |
       | Response comes back through NAT
       v
   Spring Boot receives the response
```

This is covered in detail in the next section.

---

6. How Users Reach Your App From the Internet

This is north-south traffic - from outside AWS into your app.

6.1 The Full Chain

```
[text]
 User types: https://app.yourcompany.com

 Step 1: DNS Resolution
    Browser asks: "What is the IP of app.yourcompany.com?"
    Route 53 answers: "It’s a CloudFront distribution at d123xxx.cloudfront.net"

 Step 2: CloudFront
    The request hits a CloudFront edge location near the user.
    CloudFront serves the React static files (HTML, JS, CSS) from S3.

 Step 3: Browser loads React
    React app is now running in the user’s browser.

 Step 4: React makes API call
    Browser sends: GET https://api.yourcompany.com/api/orders
```

```
   DNS resolves api.yourcompany.com to the ALB’s IP address.

Step 5: ALB receives the request
   The ALB lives in a public subnet.
   It has an Internet Gateway route, so internet traffic can reach it.
   ALB terminates TLS (HTTPS -> HTTP internally).

Step 6: ALB forwards to backend
   ALB knows which targets are healthy (via health checks).
   It forwards to a Spring Boot instance/task/pod in a private subnet.

Step 7: Backend processes
   Spring Boot queries RDS, maybe Redis.
   Builds response.

Step 8: Response travels back
   Backend -> ALB -> Internet -> User’s browser.
```

6.2 Internet Gateway - The Front Door

The Internet Gateway (IGW) is attached to your VPC. It allows resources in public subnets to communicate with the internet.

Real-life analogy:

```
[text]
IGW = the main entrance of your building.

If your room (subnet) has a hallway to the main entrance  -> public subnet
If your room has NO hallway to the main entrance          -> private subnet
```

Important:

- the IGW itself does not cost money
- it does not limit bandwidth significantly
- you need exactly one per VPC
- it is the reason your ALB can receive internet traffic

6.3 ALB - The Smart Receptionist

The Application Load Balancer sits in public subnets and is the entry point for HTTP/HTTPS traffic.

What it does:

```
[text]
1. Receives requests from the internet
2. Terminates TLS (handles HTTPS certificates)
3. Reads the HTTP request (path, headers, host)
4. Routes to the right backend based on rules:
     /api/*     -> Spring Boot service
     /admin/*   -> Admin service
     /health    -> Health check service
5. Only sends traffic to healthy targets
6. Distributes load across multiple instances/tasks/pods
```

Real-life analogy:

```
[text]
ALB = a reception desk in the lobby

"You want to see Engineering? Floor 3, room 302."
"You want Finance? Floor 5, room 510."
"Room 302 is being renovated? I’ll send you to room 303 instead."
```

6.4 Why the Backend Is NOT in a Public Subnet

You might think: "Why not just put Spring Boot in a public subnet and skip the ALB?"

Reasons:

```
[text]
 1. Direct exposure -> any attacker can probe your app directly
 2. No load balancing -> one instance = single point of failure
 3. No health checks -> broken instance still receives traffic
 4. No TLS termination -> each instance manages certificates
 5. No routing rules -> cannot split traffic by path
 6. No connection draining during deploys

 The ALB is not optional overhead. It is a critical safety and operations layer.
```

---

7. How Your Backend Reaches the Outside World

Your backend sits in a private subnet. It cannot reach the internet by default. But sometimes it needs to:

- call external APIs (Stripe, Twilio, email providers)
- download packages during setup
- pull container images
- reach AWS services that are outside the VPC

7.1 NAT Gateway - The Outbound-Only Door

NAT = Network Address Translation.

Real-life analogy:

```
[text]
 Imagine your office building has a security rule:
   "People outside cannot walk in uninvited."
   "But people inside CAN walk out, do their errands, and come back."

 The NAT Gateway is that one-way exit door.

   Outbound: allowed (your backend can call Stripe)
   Inbound:  blocked (Stripe cannot randomly connect to your backend)
```

How it works:

```
[text]
 Spring Boot (private subnet, IP: 10.0.10.15)
       |
       | "I need to call https://api.stripe.com"
       v
 NAT Gateway (public subnet, has a public IP: 54.23.xx.xx)
       |
       | Translates: 10.0.10.15 -> 54.23.xx.xx
       | Sends request to Stripe
       v
 Internet -> api.stripe.com
       |
       | Stripe responds to 54.23.xx.xx
       v
 NAT Gateway
       |
       | Translates back: 54.23.xx.xx -> 10.0.10.15
       v
 Spring Boot receives the response
```

The outside world only sees 54.23.xx.xx (the NAT Gateway's IP). It never sees 10.0.10.15.

Your backend stays hidden.

NAT Gateway Costs

This is important because NAT Gateway is a common surprise cost:

```
[text]
 NAT Gateway:
   $0.045/hour     = ~$32/month just to exist
   $0.045/GB       = extra for every GB of data

 If your backend sends 100 GB/month through NAT:
   $32 + ($0.045  100) = $36.50/month  PER NAT GATEWAY

 You usually want one per AZ for resilience, so double it.
```

7.2 VPC Endpoints - The Shortcut That Skips NAT

When your backend needs to call AWS services (S3, Secrets Manager, DynamoDB, SQS), it does not need to go through the NAT Gateway and out to the internet.

VPC Endpoints create a private tunnel directly to the AWS service.

Real-life analogy:

```
[text]
 Without VPC Endpoint:
   You (private room) -> exit building through NAT -> walk across the city -> enter AWS S3
 building

 With VPC Endpoint:
   You (private room) -> walk through an internal corridor -> arrive at S3 (never left the
 building)
```

```
[text]
 Without endpoint:
   Backend -> NAT Gateway -> Internet -> S3 public endpoint
   (costs NAT data processing fees, slower, exposed to internet)

 With endpoint:
   Backend -> VPC Endpoint -> S3
   (free for gateway endpoints, private, faster)
```

Two types:

```
[text]
 Gateway Endpoints:
   Available for: S3, DynamoDB
   Cost: FREE
   How: add an entry to the route table

 Interface Endpoints (powered by PrivateLink):
   Available for: Secrets Manager, SQS, ECR, CloudWatch, and many more
   Cost: ~$7/month per endpoint + data charges
   How: creates a private IP (ENI) in your subnet
```

When to Use VPC Endpoints

```
[text]
 Always use Gateway Endpoints for S3 and DynamoDB.
   -> They are free. No reason not to.

 Use Interface Endpoints when:
   -> your backend calls Secrets Manager, SQS, ECR frequently
   -> you want to avoid NAT Gateway data charges for those calls
   -> you want to keep traffic fully private
```

```
Skip Interface Endpoints when:
  -> the service is called rarely
  -> the $7/month per endpoint exceeds the NAT savings
```

7.3 Summary: Backend Outbound Traffic Decision

```
[text]
 Backend needs to reach:

   AWS service (S3, DynamoDB)?        -> Gateway VPC Endpoint (free)
   AWS service (Secrets Manager, SQS) -> Interface VPC Endpoint (if frequent)
   External API (Stripe, Twilio)?     -> NAT Gateway (only option)
   Nothing external?                  -> No NAT Gateway needed, save money
```

---

8. DNS - How Names Become Addresses

On your laptop, you type localhost. On AWS, you type myapp-db.abc123.us-east-1.rds.amazonaws.com. DNS is the system that turns names into IP addresses.

8.1 Real-Life Analogy

```
[text]
 DNS is like your phone’s contact list.

 You do not memorize phone numbers.
 You tap "Mom" and the phone knows to dial +1-555-123-4567.

 DNS works the same way:
   You type "api.yourcompany.com"
   DNS translates it to "52.23.178.42" (the ALB’s IP)
   Your browser connects to that IP
```

8.2 Route 53 - AWS DNS Service

Route 53 is where you manage DNS records for your domain.

Typical records for your app:

```
[text]
 app.yourcompany.com    -> CloudFront distribution   (React frontend)
 api.yourcompany.com    -> ALB                       (Spring Boot backend)
```

How you set this up:

```
[text]
 1. Buy or transfer your domain to Route 53 (or keep it elsewhere and point nameservers)
 2. Create a Hosted Zone for yourcompany.com
 3. Add records:

    Type: A (Alias)
    Name: app.yourcompany.com
    Target: CloudFront distribution ID

    Type: A (Alias)
    Name: api.yourcompany.com
    Target: ALB DNS name
```

Now when a user types app.yourcompany.com, DNS knows to send them to CloudFront. When React calls api.yourcompany.com, DNS knows to send traffic to the ALB.

8.3 Private DNS Inside the VPC

Inside your VPC, AWS provides private DNS automatically.

When you create an RDS instance, AWS gives it a DNS name like:

```
[text]
 myapp-db.abc123.us-east-1.rds.amazonaws.com
```

This name resolves to a private IP (like 10.0.100.42) that only works inside the VPC.

Your backend uses this hostname in its config. It never needs to know the raw IP.

Why hostnames instead of IPs?

```
[text]
 If RDS fails over to another AZ:
   Old IP: 10.0.100.42  (AZ-a, now dead)
   New IP: 10.0.200.18  (AZ-b, now active)

 If you used the IP directly, your app breaks.
 If you used the hostname, DNS updates automatically to point to the new IP.
 Your app reconnects seamlessly.
```

This is why you always use DNS names, not raw IPs, for databases and services.

8.4 How DNS Resolution Flows for Your App

```
[text]
 User’s browser -> "What is api.yourcompany.com?"
       |
       v
 Route 53 -> "It’s an alias for myapp-alb-123.us-east-1.elb.amazonaws.com"
       |
       v
 DNS resolves ALB name -> "52.23.178.42"  (public IP)
       |
       v
 Browser connects to 52.23.178.42 (the ALB)


 Spring Boot -> "What is myapp-db.abc123.us-east-1.rds.amazonaws.com?"
       |
       v
 VPC internal DNS -> "10.0.100.42"  (private IP, only works inside VPC)
       |
       v
 Spring Boot connects to 10.0.100.42 (the RDS instance)
```

Public DNS for external traffic. Private DNS for internal traffic. Same mechanism, different worlds.

---

9. Security - Who Can Talk to Whom

Networking is not just about connectivity. It is about controlled connectivity.

9.1 Security Groups - The Bouncers at Each Door

A Security Group is a firewall attached to a specific resource.

Real-life analogy:

```
[text]
 Security Group = a bouncer at the door of each room

 The bouncer has a list:
```

```
    "Allow anyone from the lobby (ALB) on port 8080"
    "Block everyone else"

  Every EC2 instance, ECS task, RDS instance, and ALB has its own bouncer.
```

Security Groups for Your App

```
[text]
 ALB Security Group:
   Inbound:
     - Allow TCP 443 (HTTPS) from 0.0.0.0/0  (anyone on the internet)
   Outbound:
     - Allow TCP 8080 to Backend Security Group

 Backend Security Group (EC2/ECS/EKS):
   Inbound:
     - Allow TCP 8080 from ALB Security Group  (only the ALB can reach the backend)
   Outbound:
     - Allow TCP 5432 to Database Security Group
     - Allow TCP 6379 to Redis Security Group
     - Allow TCP 443 to 0.0.0.0/0             (for calling external APIs via NAT)

 Database Security Group (RDS):
   Inbound:
     - Allow TCP 5432 from Backend Security Group  (only backend can reach the DB)
   Outbound:
     - None needed (DB does not initiate outbound connections)

 Redis Security Group (ElastiCache):
   Inbound:
     - Allow TCP 6379 from Backend Security Group
   Outbound:
     - None needed
```

Notice the pattern:

```
[text]
 Internet  -> ALB (only port 443)
 ALB       -> Backend (only port 8080)
 Backend   -> Database (only port 5432)
 Backend   -> Redis (only port 6379)

 Nobody skips a layer. The database is unreachable from the internet.
```

Why Security Groups Reference Other Security Groups

Instead of writing:

```
[text]
 "Allow port 5432 from IP 10.0.10.15"
```

You write:

```
[text]
 "Allow port 5432 from sg-backend-security-group"
```

Why?

- IPs change when instances scale or replace
- security group references work regardless of how many instances exist or what IPs they have
- this is the correct pattern in production

9.2 Network ACLs - The Building-Level Rules

NACLs are subnet-level rules. They apply to all traffic entering or leaving a subnet.

Real-life analogy:

```text
Security Group = bouncer at each room door
NACL           = security gate at the entrance of each floor

NACLs are coarser. Most teams rely on Security Groups for fine control
and only use NACLs for rare subnet-level deny rules.
```

For your app, Security Groups do 95% of the work. NACLs are there if you need to block a specific IP range at the subnet level.

9.3 TLS / HTTPS - Encryption in Transit

```text
User -> ALB:      HTTPS (encrypted with TLS certificate)
ALB -> Backend:   HTTP usually (within private network, TLS optional)
Backend -> RDS:   TLS recommended (can be enforced in RDS settings)
```

The TLS certificate for your domain lives in AWS Certificate Manager (ACM). It is free. ALB uses it automatically.

---

10. Putting It All Together - Full Traffic Flow

Here is every networking component working together for one API request:

```text
User types: https://app.yourcompany.com


[Route 53]  resolves app.yourcompany.com -> CloudFront


[CloudFront]  serves React static files from S3


React app loads in browser. User clicks "View Orders."


Browser sends: GET https://api.yourcompany.com/api/orders


[Route 53]  resolves api.yourcompany.com -> ALB public IP


[Internet Gateway]  allows the request into the VPC


[ALB]  (public subnet, SG allows port 443 from internet)
    terminates TLS
    checks path: /api/orders -> forward to backend target group
    picks a healthy backend instance


[Spring Boot]  (private subnet, SG allows port 8080 from ALB only)
    receives request
    needs to query database


[RDS PostgreSQL]  (private subnet, SG allows port 5432 from backend only)
    executes query
```

```
      returns result


  [Spring Boot]  builds JSON response


  [ALB]  sends response back through Internet Gateway


  [Browser]  receives JSON, React renders the orders page
```

And if the backend needs to call Stripe during order processing:

```text]
  [Spring Boot]  (private subnet)

          "I need to call https://api.stripe.com/v1/charges"

  [NAT Gateway]  (public subnet)

          translates private IP -> public IP

  [Internet]  -> api.stripe.com

          response comes back

  [NAT Gateway]  translates back


  [Spring Boot]  receives Stripe response
```

---

11. Real-Life Analogy: The Complete Picture

Here is the entire AWS networking model as an office building:

```text]

                      YOUR OFFICE BUILDING (VPC)

    MAIN ENTRANCE (Internet Gateway)



      LOBBY - Public Subnet

      Reception Desk (ALB)
        "Hello, who are you here to see?"
        "Engineering API? Let me check if they're available"
        "Yes, they're healthy. Follow me."

      Side Exit (NAT Gateway)
        "Staff can go out for errands."
        "Outsiders cannot enter through this door."




      OFFICE FLOOR - Private Subnet (App Tier)

      Room 301: Spring Boot Instance A
      Room 302: Spring Boot Instance B
```

```
    Door bouncer (Security Group):
      "Only people from the Reception Desk allowed in"
      "Only staff here can visit the Archive Floor"




    ARCHIVE FLOOR - Private Subnet (Data Tier)

    Vault A: PostgreSQL (RDS)
    Vault B: Redis (ElastiCache)

    Door bouncer (Security Group):
      "Only people from the Office Floor allowed in"
      "Absolutely nobody from outside"


  INTERNAL CORRIDOR (VPC Endpoint to S3):
    "Need files from the S3 warehouse next door?
     Use this private corridor. No need to go outside."



SEPARATE BUILDING (S3 + CloudFront):
  "The marketing materials (React frontend) are stored here.
   Anyone can pick up a copy from any CloudFront branch office
   around the world."

PHONE DIRECTORY (Route 53 / DNS):
  "Looking for the Engineering building? Here's the address."
  "Looking for the database? Here's the INTERNAL extension number."
```

---

12. Common Mistakes and Debugging Tips

12.1 "My backend cannot connect to RDS"

Check in this order:

```
[text]
 1. Is the RDS in the same VPC as the backend?
 2. Is the backend's security group allowed outbound to port 5432?
 3. Is the RDS security group allowing inbound from the backend's security group?
 4. Is the backend using the correct RDS endpoint hostname?
 5. Are the database credentials correct?
 6. Are they in the same (or peered) subnets?
```

90% of the time it is a security group misconfiguration.

12.2 "My backend cannot call external APIs"

```
[text]
 1. Is the backend in a private subnet? (It should be)
 2. Is there a NAT Gateway in a public subnet?
 3. Does the private subnet's route table have:
      0.0.0.0/0 -> NAT Gateway?
 4. Does the backend's security group allow outbound on port 443?
```

If any of these is missing, outbound traffic will silently fail.

12.3 "Users cannot reach my API"

```
[text]
 1. Does Route 53 have an A record pointing to the ALB?
 2. Is the ALB in a public subnet with an Internet Gateway route?
 3. Does the ALB security group allow inbound port 443 from 0.0.0.0/0?
 4. Does the ALB have a valid TLS certificate from ACM?
 5. Is the target group healthy? (Check ALB target group health in console)
 6. Does the backend security group allow inbound from the ALB security group?
```

12.4 "My ECS task cannot pull images from ECR"

```
[text]
 The task is in a private subnet and has no way to reach ECR.

 Fix options:
   a) Add a NAT Gateway (costs money)
   b) Add a VPC Interface Endpoint for ECR (com.amazonaws.region.ecr.dkr
      and com.amazonaws.region.ecr.api) + S3 Gateway Endpoint (for image layers)
```

This catches many teams on their first ECS deployment.

12.5 Quick Debug Mental Model

```
[text]
 If something cannot connect to something else:

   Step 1: Are they in the same VPC? (or peered/PrivateLinked?)
   Step 2: Can the network route get from A to B? (route tables)
   Step 3: Do security groups allow the traffic? (inbound on target, outbound on source)
   Step 4: Is DNS resolving correctly? (nslookup / dig)
   Step 5: Is the application actually listening on the expected port?
```

---

13. Interview-Ready Answers

13.1 "How does a user’s request reach your Spring Boot backend on AWS?"

```
[text]
 "The user’s browser resolves the API domain via Route 53, which points to
 an ALB in a public subnet. The ALB terminates TLS, evaluates routing rules,
 and forwards the request to a healthy Spring Boot instance in a private
 subnet. The backend processes the request, queries RDS in another private
 subnet, and sends the response back through the ALB."
```

13.2 "Why private subnets for the backend?"

```
[text]
 "Private subnets have no route to the Internet Gateway, so they cannot be
 directly reached from the internet. This reduces attack surface. The ALB
 in the public subnet acts as the controlled entry point. Backend instances
 only accept traffic from the ALB’s security group."
```

13.3 "How does your backend in a private subnet call external APIs?"

```
[text]
 "Through a NAT Gateway in a public subnet. The private subnet’s route table
 sends 0.0.0.0/0 traffic to the NAT Gateway, which translates the private IP
 to a public IP for outbound requests. Inbound internet traffic is still
 blocked. For AWS service calls like S3 or Secrets Manager, I use VPC
 endpoints to keep traffic private and avoid NAT costs."
```

13.4 "Explain security groups in your architecture"

```text
"Each layer has its own security group. The ALB allows inbound HTTPS from
the internet. The backend allows inbound on port 8080 only from the ALB’s
security group. The database allows inbound on port 5432 only from the
backend’s security group. This creates a chain where each layer only
accepts traffic from the layer directly above it. No layer is directly
exposed beyond its intended purpose."
```

13.5 "What is the difference between Internet Gateway and NAT Gateway?"

```text
"Internet Gateway allows two-way internet traffic for public subnets -
both inbound and outbound. It is used by ALBs and anything that needs
to be publicly reachable.

NAT Gateway allows one-way outbound internet access for private subnets.
Resources behind it can reach the internet, but the internet cannot
initiate connections to them. I use it when the backend needs to call
external APIs like payment gateways."
```

---

14. Quick Revision Sheet

Every Component in One Line

```text
VPC            = your private network on AWS
Subnet         = a slice of the VPC in one AZ, public or private
Internet GW    = the front door for public internet access (two-way)
NAT Gateway    = the back door for private subnet outbound-only internet
Route Table    = the map that says "traffic to X goes through Y"
Security Group = per-resource firewall (bouncer at each door)
NACL           = per-subnet firewall (security gate per floor)
ALB            = HTTPS load balancer and smart router (receptionist)
Route 53       = DNS service (phone directory)
CloudFront     = CDN (branch offices serving copies worldwide)
VPC Endpoint   = private shortcut to AWS services, skips internet
ACM            = free TLS certificates for your domains
```

The Standard 3-Tier Network Layout

```text
Public Subnet:   ALB, NAT Gateway
Private Subnet:  Backend compute (EC2, ECS, EKS)
Private Subnet:  Database (RDS, ElastiCache)
```

Traffic Flow Cheat Sheet

```text
User -> App:        Route 53 -> CloudFront -> S3
User -> API:        Route 53 -> ALB (public) -> Backend (private)
Backend -> DB:      Private subnet -> Private subnet (via security group)
Backend -> Redis:   Private subnet -> Private subnet (via security group)
Backend -> S3:      VPC Gateway Endpoint (free, private)
Backend -> Stripe:  NAT Gateway -> Internet
Backend -> Secrets: VPC Interface Endpoint (private) or NAT Gateway
```

Gold Standard Sentence

```text
```

```
"On AWS, I place the ALB in public subnets as the only internet-facing
entry point. The backend runs in private subnets, reachable only through the
ALB. The database sits in private subnets, reachable only from the backend.
Outbound internet access uses a NAT Gateway, and AWS service calls use VPC
endpoints to stay private and save costs. Security groups enforce least-privilege
at every layer, and Route 53 handles DNS for both external users and internal
service discovery."
```

Part 07: AWS Storage Through Story Mode: Where Your App’s Data Actually Lives

Source file: AWS-07-Storage-Story-Mode.md

> You have a React frontend, a Spring Boot backend, a database, maybe some uploaded files, maybe some cache. On your laptop, all of this feels simple. On AWS, you suddenly have many storage choices: S3, EBS, EFS, RDS, Aurora, DynamoDB, ElastiCache. This guide explains them in a practical, story-mode way so you can understand what goes where without getting lost.

---

1. How Storage Feels on Your Laptop

On your laptop, your app usually looks like this:

```
[text]
 frontend/
 backend/
 uploads/
 local-postgres-data/
```

And conceptually:

```
[text]
 React frontend
   -> browser loads built files from local dev server

 Spring Boot backend
   -> reads config from local files or env vars
   -> stores data in local PostgreSQL
   -> maybe writes uploads to a local folder like ./uploads
   -> maybe caches some things in memory
```

Everything sits on one machine, so storage feels invisible.

Examples:

- your React build files are just files on disk
- your Spring Boot JAR is just a file on disk
- PostgreSQL stores its data on your local disk
- uploaded PDFs may go into a local folder
- cache may live inside JVM memory

This works locally because:

- there is only one machine
- there is no scaling yet
- if the machine dies, it is just your dev environment

Production is different.

---

2. What Changes When You Move to AWS

On AWS, you stop asking:

```text
"Where is my file on disk?"
```

and start asking:

```text
"What kind of data is this?"
"Who needs to access it?"
"Does it need SQL?"
"Is it a file, a disk, or an object?"
"Does it need to survive server replacement?"
"Will one server use it, or many servers?"
```

That is the key shift.

AWS storage is not one big thing. Different data belongs in different places.

For your app, you usually split storage like this:

```text
Frontend static files        -> S3
Backend boot/root disk       -> EBS
Database records             -> RDS / Aurora / DynamoDB
User uploads                 -> S3
Shared filesystem            -> EFS (only if needed)
Cache / sessions / hot data  -> ElastiCache
```

That is the big picture.

---

3. The Big Mental Model: Three Kinds of Storage

This is the most important section.

3.1 Object Storage

Think: "store a whole file or blob by name."

Examples:

- image
- PDF
- video
- React build output
- logs archive
- backup file

AWS service:

- S3

Real-life analogy:

```text
S3 is like a giant warehouse.

You do not say:
  "Give me sector 4, block 19, byte 200"

You say:
  "Give me the box named invoices/2026/march/report.pdf"
```

3.2 Block Storage

Think: "a disk attached to a machine."

Examples:

- EC2 root volume
- database files on a VM
- application filesystem on one server

AWS service:

- EBS

Real-life analogy:

```
[text]
 EBS is like a hard drive attached to one computer.
 It behaves like a disk.
 The machine formats it, mounts it, reads and writes blocks.
```

3.3 File Storage

Think: "a shared filesystem multiple machines can mount."

Examples:

- shared document folder
- legacy app needing common shared files
- multiple app servers reading the same files

AWS service:

- EFS

Real-life analogy:

```
[text]
 EFS is like a shared company drive.
 Many servers can mount it and see the same files.
```

3.4 Database Storage

Think: "structured application records."

Examples:

- users
- orders
- payments
- inventory
- shopping cart items

AWS services:

- RDS
- Aurora
- DynamoDB

This is not file storage. This is application data storage.

3.5 Cache Storage

Think: "temporary fast-access data."

Examples:

- frequently read product data
- session data
- OTP/rate-limit counters
- expensive query results

AWS service:

- ElastiCache

Real-life analogy:

```
[text]
```

```
Cache is like the receptionist’s desk drawer.
Important things that are needed often stay close at hand.
If the drawer is cleared, the office still works, but some things get slower.
```

---

4. Story Mode: Your App Starts Simple

Let us follow a realistic app.

You have:

- React frontend
- Spring Boot backend
- PostgreSQL database
- users upload profile pictures and invoices
- dashboard reads same reports often
- maybe several app servers later

At the beginning, you do something like this locally:

```
[text]
Frontend files              -> frontend/dist on laptop
Backend JAR                 -> target/app.jar on laptop
Database                    -> local PostgreSQL data directory
User uploads                -> ./uploads folder
Cache                       -> JVM memory or none
```

That is okay for local development.

Then production starts and problems appear:

- what if the EC2 instance dies?
- what if you scale from 1 backend instance to 3?
- what if uploads are stored on one server and another server cannot see them?
- what if users need to download files globally?
- what if DB backups are needed?

Now AWS storage choices matter.

---

5. Storage for the Frontend

5.1 What the Frontend Actually Is

For a normal React SPA, the frontend build output is just static files:

- index.html
- JS bundles
- CSS
- images

This is not database data. This is not shared filesystem data. This is static asset data.

5.2 Best AWS Home for Frontend Files

Use:

- S3 to store the built files
- CloudFront to serve them globally

Why?

- cheap
- durable
- scalable
- perfect for static files

Real-life flow:

```text
1. You run npm run build
2. React produces static files
3. CI/CD uploads them to S3 bucket: myapp-frontend-prod
4. CloudFront serves them to users worldwide
```

This is much better than storing React files on an EC2 disk.

5.3 Why Not Store React Files on EC2 EBS?

You can, but it is usually the wrong default.

Why not?

- EC2 disk is tied to that server
- if the instance is replaced, you redeploy again
- serving static files from S3 + CloudFront is cheaper and cleaner
- no need to waste backend compute on static assets

So for frontend static files:

```text
Right answer   -> S3
Wrong default  -> EC2 disk
```

---

6. Storage for the Backend Server Itself

This section is about the backend machine, not the backend data.

6.1 If You Run on EC2

An EC2 instance needs a disk.

That disk is usually:

- EBS

Typical uses of EBS on EC2:

- OS boot volume
- storing application binaries temporarily
- logs before shipping them elsewhere
- local temp files

Real-life example:

```text
EC2 instance running Spring Boot
  -> root volume on EBS
  -> app.jar may sit on EBS
  -> Linux OS sits on EBS
```

6.2 Important Rule About EBS

Do not confuse:

```text
"disk of the server"
```

with:

```text
"long-term application data"
```

Your EC2 disk is not where you should keep business-critical uploaded files or your primary database unless you intentionally chose that architecture.

Why?

- servers get replaced
- Auto Scaling may terminate instances
- local files on one instance are not visible to other instances

Use EBS for the machine. Use better storage choices for shared or durable app data.

6.3 If You Run on ECS or EKS

Then you usually care less about a disk attached to a server.

Why?

- containers are treated as replaceable
- local container storage is temporary
- if a container restarts, local written files may disappear

So the rule becomes even stronger:

```
[text]
 Never store important application data inside container local storage.
```

Use S3, RDS, EFS, or ElastiCache instead depending on the data type.

---

7. Storage for Application Data

This is your real business data.

Examples:

- users
- orders
- invoices
- product catalog
- payments
- subscriptions

This is not a file problem. This is a database problem.

7.1 Default Choice for a Typical Spring Boot App

If your app has tables, joins, transactions, and relational data:

Use:

- RDS PostgreSQL or RDS MySQL

Real-life example:

```
[text]
 users table
 orders table
 order_items table
 payments table
```

That belongs in RDS.

Why?

- SQL queries
- ACID transactions
- joins
- indexes
- backups
- Multi-AZ availability options

7.2 RDS in Story Mode

Local version:

```
[text]
```

```
Spring Boot -> localhost:5432
```

AWS version:

```
[text]
Spring Boot -> myapp-db.abc123.us-east-1.rds.amazonaws.com:5432
```

Same idea, different host.

Spring Boot still connects with JDBC. But now AWS manages the database infrastructure better than a self-managed VM in many cases.

7.3 When Aurora Enters the Story

Aurora is still relational, still SQL, still for app data.

You choose Aurora instead of normal RDS when:

- you want stronger managed scalability
- you want higher performance characteristics
- you want faster failover and AWS-optimized DB internals
- your workload is growing and cost/benefit makes sense

Simple rule:

```
[text]
RDS PostgreSQL/MySQL -> normal default for many apps
Aurora               -> stronger managed relational option when scale/HA needs grow
```

7.4 When DynamoDB Enters the Story

Sometimes your app data is not best modeled relationally.

Examples:

- shopping cart by user ID
- session state
- key-value profile data
- huge request-driven workloads with simple access patterns

Then DynamoDB can be the right answer.

Real-life example:

```
[text]
Get cart by user_id
Update cart by user_id
Expire cart after 7 days
```

That is a classic DynamoDB use case.

But if you need:

- joins
- complex reporting queries
- normalized relational structure

then RDS/Aurora is usually better.

7.5 Practical Guidance for Your App

If you have a normal Spring Boot business application:

```
[text]
Users / orders / payments / subscriptions  -> RDS or Aurora
Session store / carts / high-scale key-value -> DynamoDB or Redis depending on use case
```

Start simple:

- choose RDS first for relational business data
- choose DynamoDB only when access patterns clearly justify it

---

8. Storage for User Uploaded Files

This is one of the most common real-life questions.

Users upload:

- profile images
- resumes
- invoices
- videos
- PDFs

Where do these go?

8.1 Wrong First Instinct

Many people think:

```
[text]
 "I'll save uploads in /uploads on the EC2 server."
```

This breaks quickly.

Why?

- if you have 3 backend servers, uploads saved on server A are not on B or C
- if the instance dies, files may be lost
- scaling becomes messy
- backups become awkward

8.2 Correct Default

Store uploads in:

- S3

Store metadata in:

- RDS or another database

Real-life pattern:

```
[text]
 S3 stores the actual file:
   s3://myapp-documents-prod/invoices/2026/03/invoice-123.pdf

 RDS stores metadata:
   file_id
   user_id
   s3_key
   uploaded_at
   content_type
   size_bytes
```

That is the correct separation.

8.3 Best Upload Flow in Real Life

Option A: Backend receives file and uploads to S3

```
[text]
 Browser -> Spring Boot -> S3
```

Simple, but backend handles the file bytes.

Option B: Pre-signed URL (better default)

```
[text]
 1. Browser asks Spring Boot: "I want to upload a PDF"
 2. Spring Boot creates a pre-signed S3 upload URL
 3. Browser uploads directly to S3
 4. Browser or backend stores metadata in RDS
```

Why this is better:

- backend does not carry large file upload traffic
- simpler scaling
- cheaper and cleaner architecture

8.4 Download Flow

For downloads:

- private download via pre-signed URL
- or serve through CloudFront if broad/global distribution matters

Example:

```
[text]
 Private invoice download:
   backend checks authorization
   backend returns pre-signed S3 URL valid for 5 minutes

 Public image download:
   CloudFront serves S3 object globally
```

---

9. Storage for Shared Files Across Servers

Sometimes teams ask:

```
[text]
 "What if multiple EC2 instances need to read the same files?"
```

That is where EFS comes in.

9.1 What EFS Is Good For

Use EFS when:

- multiple servers need the same filesystem
- app expects POSIX-style files and folders
- legacy app cannot easily move to S3 object model
- containers need shared RWX filesystem behavior

Real-life examples:

- CMS with shared media directory
- legacy Java app writing generated reports to a shared mounted folder
- ML jobs reading the same dataset as files

9.2 Why EFS Is Not the Default

Many teams ask about EFS too early.

Why not default to it?

- more expensive than S3 for many use cases
- slower than local disk for some patterns
- unnecessary if S3 object storage solves the problem better

Simple rule:

```
[text]
 Need shared files with filesystem semantics? -> EFS
 Need file/object storage, upload/download, assets? -> S3
```

9.3 Example: When EFS Is Actually Right

Imagine:

- 4 ECS tasks generate PDF reports

- another service reads them from the same shared folder
- the app is hardcoded for filesystem paths like /reports/monthly/report.pdf

Then EFS may be practical.

But if you control the design, S3 is often cleaner.

---

10. Storage for Cache and Fast Reads

Not all storage is about durability. Some storage is about speed.

10.1 What Cache Is For

Suppose your dashboard hits the same expensive query repeatedly:

```
[text]
 SELECT * FROM sales_summary WHERE region = 'US' AND month = '2026-03'
```

If 10,000 users ask for the same thing, hitting RDS every time is wasteful.

Use:

- ElastiCache (Redis)

10.2 Story Mode Example

```
[text]
 User opens dashboard
   -> backend checks Redis for dashboard:US:2026-03
   -> if present, return instantly
   -> if not present, query RDS, build response, store in Redis with TTL, return
```

That is cache-aside.

10.3 What Belongs in Cache

Good candidates:

- frequently read product details
- session data
- OTP / token validation data
- rate limit counters
- expensive query results

Bad candidates:

- your only copy of important business data
- things needing perfect permanence

10.4 Key Rule

If Redis is lost, the app should become slower, not incorrect forever.

That is the mindset.

Cache is a performance layer, not your system of record.

---

11. Putting It All Together for a Real App

Let us map a realistic app end to end.

11.1 Example App

You have:

- React frontend
- Spring Boot backend
- PostgreSQL database

- users upload profile images and invoices
- dashboard reads hot summary data
- maybe reports are generated nightly

11.2 Best Storage Mapping

```
[text]
 React build output             -> S3
 Global frontend delivery       -> CloudFront

 Spring Boot instance root disk -> EBS (if EC2)
 Container local storage        -> temporary only

 Users / Orders / Payments      -> RDS PostgreSQL
 Heavy relational scale upgrade -> Aurora PostgreSQL/MySQL

 Profile images / invoices      -> S3
 File metadata                  -> RDS

 Shared legacy report folder    -> EFS (only if truly needed)

 Cached dashboards / sessions   -> ElastiCache Redis
 Key-value ultra-scale pattern  -> DynamoDB when justified
```

11.3 Real-Life Flow: User Uploads Invoice

```
[text]
 1. User logs into React app
 2. User clicks "Upload invoice"
 3. React asks backend for upload permission
 4. Spring Boot generates pre-signed S3 URL
 5. Browser uploads PDF directly to S3
 6. Spring Boot stores invoice metadata in RDS:
      user_id, s3_key, upload_time, status
 7. Later, user requests invoice list
 8. Backend queries RDS for metadata
 9. If user downloads one, backend returns pre-signed S3 download URL
```

That is a very common real production pattern.

11.4 Real-Life Flow: Dashboard Load

```
[text]
 1. User opens analytics dashboard
 2. Backend checks Redis
 3. If cached -> return fast
 4. If not cached -> query RDS, build response, cache it for 5 minutes
 5. Future requests hit Redis first
```

11.5 Real-Life Flow: EC2 Instance Dies

What survives?

```
[text]
 EBS root volume on terminated instance      -> depends on termination settings
 Files only on that instance                 -> risky / maybe gone
 RDS data                                    -> safe, still there
 S3 uploaded files                           -> safe, still there
 Redis cache                                 -> may be rebuildable
 EFS shared files                            -> still there
```

This is why application data should not live only on one server’s local disk.

---

12. Backups, Durability, and Recovery

Storage is not just "where data lives." It is also:

- how safely it lives
- how recoverable it is
- how much data loss you can tolerate

12.1 S3 Durability

S3 is highly durable and excellent for stored objects.

Use features like:

- versioning
- lifecycle rules
- server-side encryption
- replication only if needed

Real-life example:

```
[text]
 Enable versioning on document bucket.
 If someone overwrites invoice.pdf accidentally,
 you can recover the older version.
```

12.2 EBS Backups

EBS uses snapshots.

Good for:

- EC2 disk backups
- AMI creation
- restore scenarios

But remember:

EBS snapshotting a server disk is not the same as designing proper application data storage.

12.3 RDS Backups

RDS gives:

- automated backups
- snapshots
- point-in-time recovery
- Multi-AZ for availability

Important distinction:

```
[text]
 Backup answers: "Can I recover old data?"
 Multi-AZ answers: "Can I survive infrastructure failure quickly?"
```

They solve different problems.

12.4 Cache Recovery

For Redis cache, your design should tolerate cache loss.

If cache disappears:

- backend should repopulate from DB
- performance degrades temporarily
- business truth remains in primary data store

---

13. Common Mistakes and Debugging Tips

13.1 "We stored uploads on the EC2 server"

This breaks when:

- you scale horizontally
- the instance is replaced
- one server has files another does not

Fix:

- move files to S3
- keep metadata in DB

13.2 "We put business data in Redis"

This is dangerous unless you truly designed for it.

Fix:

- keep source of truth in RDS/Aurora/DynamoDB
- treat Redis as performance layer

13.3 "We used EFS when S3 would do"

Teams often overuse shared filesystems.

Ask:

```
[text]
 Do I actually need mounted filesystem semantics?
 Or do I just need durable file/object storage?
```

If it is upload/download/object retrieval, S3 is usually better.

13.4 "We used DynamoDB without clear access patterns"

This causes pain.

Fix:

- define required queries first
- design partition key around access pattern
- if data is relational and query patterns are broad, use RDS instead

13.5 Quick Debug Mental Model

When confused, ask in this order:

```
[text]
 1. Is this file storage, block storage, or database data?
 2. Is it shared by many machines or tied to one machine?
 3. Does it need SQL transactions?
 4. Is it the source of truth or just cache?
 5. Does it need to survive server replacement?
```

Usually the correct AWS service becomes obvious after that.

---

14. Interview-Ready Answers

14.1 "Where would you store frontend assets for a React app?"

```
[text]
 "For a React SPA, I would build the static files and store them in S3,
 then serve them through CloudFront. S3 is durable and cheap for static
 assets, and CloudFront gives low-latency global delivery. I would not
 default to serving React files from EC2 disks unless there is a strong
 reason to keep everything on compute."
```

14.2 "Where would you store user uploaded files?"

```text
"I would store the actual files in S3 and keep metadata in a database
like RDS. For uploads, I prefer pre-signed URLs so the browser can upload
directly to S3 without routing large file traffic through the backend."
```

14.3 "When would you use EBS vs EFS vs S3?"

```text
"I use EBS when one machine needs a disk, like an EC2 root volume.
I use EFS when multiple machines need a shared filesystem with mounted
file semantics. I use S3 for object storage like assets, documents,
backups, and uploads. For most application files, S3 is the default.
EFS is only when shared filesystem semantics are genuinely required."
```

14.4 "When RDS vs DynamoDB?"

```text
"I choose RDS when the data is relational and I need transactions,
joins, and flexible SQL queries. I choose DynamoDB when access patterns
are known up front and the workload fits key-value or document access
at large scale with low-latency reads and writes. For a typical Spring
Boot business app with users, orders, and payments, I would usually start
with RDS."
```

14.5 "How do you think about cache?"

```text
"I treat cache as a speed layer, not as the source of truth. I use
ElastiCache Redis for hot reads, sessions, or counters, but the durable
business truth remains in RDS, Aurora, or DynamoDB. If the cache is lost,
the app should slow down temporarily, not lose correctness permanently."
```

---

15. Quick Revision Sheet

One-Line Mapping

```text
S3          = object storage for files, assets, backups, uploads
EBS         = disk attached to one EC2 instance
EFS         = shared filesystem for multiple servers
RDS         = managed relational database
Aurora      = AWS-optimized relational database
DynamoDB    = managed key-value/document database
ElastiCache = in-memory cache, often Redis
```

What Goes Where for a Normal App

```text
React static build              -> S3 + CloudFront
Spring Boot server disk         -> EBS (if using EC2)
Users / orders / payments       -> RDS
Uploaded files                  -> S3
File metadata                   -> RDS
Shared mounted filesystem       -> EFS only if needed
Hot reads / sessions / counters -> ElastiCache
High-scale key-value pattern    -> DynamoDB when justified
```

Gold Standard Sentence

```
[text]
 "For a typical React plus Spring Boot application, I store frontend static
 assets in S3, business data in RDS, uploaded files in S3 with metadata in
 the database, and hot read data in ElastiCache. I use EBS only for the
 server’s disk and EFS only when multiple servers genuinely need a shared
 filesystem. I choose DynamoDB only when the workload is naturally key-value
 or document-driven and the access patterns are well understood."
```

Part 08: AWS Security Through Story Mode: Who Can Do What and How Nothing Leaks

Source file: AWS-08-Security-Story-Mode.md

> On your laptop, security barely exists. Your Spring Boot talks to your PostgreSQL with postgres/postgres. Your React calls localhost:8080 with no authentication. There are no firewalls, no roles, no encrypted secrets. Then you move to AWS and suddenly you need IAM roles, security groups, encrypted databases, secret managers, TLS certificates, and user authentication. This guide explains AWS security the way your app actually needs it - layer by layer, starting from what you already know.

---

1. How Security Feels on Your Laptop

On your laptop, your app probably works like this:

```
[text]
 React frontend
   -> calls localhost:8080 with no auth token (or a hardcoded dev token)

 Spring Boot backend
   -> connects to PostgreSQL with username: postgres, password: postgres
   -> JWT secret is "my-dev-secret" in application.yml
   -> AWS credentials might be in ~/.aws/credentials from aws configure
   -> no TLS anywhere, everything is HTTP

 PostgreSQL
   -> listens on localhost:5432
   -> accepts connections from anyone on the machine
   -> no encryption
```

Why does this "work"?

- there is only one user: you
- there is no internet exposure
- there is no attacker
- convenience is more important than security in local dev

That is fine for development. But on AWS, every one of these shortcuts becomes a vulnerability.

---

2. What Changes When You Move to AWS

On AWS, your app is exposed to real threats:

```
[text]
```

```
Local:   nobody can reach your laptop from the outside
AWS:     your ALB is on the public internet, anyone can send requests

Local:   postgres/postgres is fine
AWS:     default credentials on a reachable database = immediate breach

Local:   secrets in application.yml are fine
AWS:     secrets in code or environment files can leak through logs, repos, or image
layers

Local:   no encryption needed
AWS:     data at rest and in transit must be encrypted for compliance and protection

Local:   you are the only user
AWS:     your app has real users who need authentication and authorization
```

So the question becomes:

```
[text]
How do I keep everything locked down while still letting
the right people and the right services do their jobs?
```

---

3. The Big Picture: Four Security Questions

Every security decision on AWS maps to one of four questions:

```
[text]


    1. WHO CAN CALL AWS SERVICES?
       "Can my Spring Boot app read from S3?"
       "Can my ECS task pull secrets?"
       Answer: IAM roles and policies

    2. WHO CAN REACH MY APP OVER THE NETWORK?
       "Can the internet hit my database?"
       "Can only the ALB talk to my backend?"
       Answer: Security groups, subnets, NACLs

    3. HOW ARE SECRETS AND DATA PROTECTED?
       "Where is my database password stored?"
       "Is my data encrypted on disk and in transit?"
       Answer: Secrets Manager, KMS, TLS, encryption settings

    4. WHO ARE MY APPLICATION USERS AND WHAT CAN THEY DO?
       "How do users log in?"
       "Can this user access admin features?"
       Answer: Cognito/JWT/OAuth + application-level authorization
```

Think of it as four layers:

```
[text]
Layer 1: IAM            -> controls what AWS resources your code can touch
Layer 2: Network        -> controls what traffic can flow where
Layer 3: Secrets + Encryption -> protects data at rest and in transit
Layer 4: App Auth       -> controls who your end users are and what they can do
```

Each layer is independent. You need all four.

---

4. Story Mode: Your App Grows Up

Phase 1: "We just deployed, it works"

The team puts Spring Boot on EC2, PostgreSQL on RDS.

They hardcode the DB password in application.yml. They use the default security group that allows all traffic. There is no HTTPS.

This works. It is also completely insecure.

Phase 2: "Security review said we have problems"

Someone audits the setup and finds:

```
[text]
 Problem 1: DB password is in the source code repo
 Problem 2: RDS is reachable from the internet
 Problem 3: No TLS - API traffic is unencrypted
 Problem 4: EC2 instance has admin-level AWS permissions
 Problem 5: No user authentication on the API
```

Now the team starts fixing things.

Phase 3: "We locked it down properly"

After fixing:

```
[text]
 [ok] DB password in Secrets Manager, injected at runtime
 [ok] RDS in private subnet, security group allows only backend
 [ok] ALB terminates HTTPS with ACM certificate
 [ok] EC2/ECS uses scoped IAM role with least privilege
 [ok] API validates JWT tokens from Cognito or custom auth
 [ok] S3 buckets are private with block public access
 [ok] EBS and RDS volumes encrypted with KMS
```

That is the journey. Let us walk through each layer.

---

5. Who Can Call AWS Services - IAM

This is the most fundamental security concept on AWS.

5.1 The Problem

Your Spring Boot app needs to:

- read secrets from Secrets Manager
- upload files to S3
- send messages to SQS
- pull images from ECR

How does AWS know your app is allowed to do these things?

5.2 Real-Life Analogy

```
[text]
 IAM is like a company badge system.

 Every employee (user) and every robot (service/application) gets a badge.
 The badge says what doors they can open and what rooms they can enter.

 Without a badge, you cannot do anything.
 With a badge scoped to "Floor 3 only," you cannot reach Floor 5.
```

5.3 The Key Concept: Roles, Not Passwords

On your laptop, you probably ran aws configure and pasted an access key.

That is a static credential. Like writing a password on a sticky note.

On AWS, the correct approach is:

```
[text]
 Local dev:     AWS access keys in ~/.aws/credentials (acceptable for dev)
 EC2 in prod:   IAM Instance Role (no keys stored anywhere)
 ECS in prod:   IAM Task Role (no keys stored anywhere)
 EKS in prod:   IRSA / Pod Identity (no keys stored anywhere)
 Lambda:        Execution Role (no keys stored anywhere)
```

The application never sees or stores AWS credentials. AWS injects temporary credentials automatically through the role.

5.4 How It Actually Works for Your Spring Boot App

On EC2

```
[text]
 1. You create an IAM role: myapp-backend-role
 2. You attach policies to it:
      - s3:GetObject on the uploads bucket
      - secretsmanager:GetSecretValue on the DB password secret
      - sqs:SendMessage on the orders queue
 3. You attach this role to the EC2 instance
 4. Spring Boot uses the AWS SDK
 5. The SDK automatically finds the role credentials via instance metadata
 6. No access keys anywhere in your code or config
```

On ECS

```
[text]
 1. You create an IAM Task Role: myapp-backend-task-role
 2. Same policy attachments as above
 3. You put the role ARN in the ECS task definition
 4. Each running task gets its own temporary credentials
 5. Different services can have different task roles -> least privilege
```

On EKS

```
[text]
 1. You create an IAM role: myapp-backend-role
 2. You create a Kubernetes ServiceAccount annotated with the role ARN (IRSA)
 3. Pods using that ServiceAccount automatically get temporary AWS credentials
 4. Pod A (needs S3) gets a different role than Pod B (needs DynamoDB)
```

5.5 What an IAM Policy Looks Like

```
[json]
 {
   "Version": "2012-10-17",
   "Statement": [
     {
       "Effect": "Allow",
       "Action": [
         "s3:GetObject",
         "s3:PutObject"
       ],
       "Resource": "arn:aws:s3:::myapp-uploads-prod/*"
     },
     {
       "Effect": "Allow",
```

```
        "Action": "secretsmanager:GetSecretValue",
        "Resource": "arn:aws:secretsmanager:us-east-1:123456789012:secret:myapp/db-
password-*"
      }
    ]
  }
```

Notice:

- specific actions, not "Action": "*"
- specific resources, not "Resource": "*"
- this is least privilege

5.6 The Two IAM Roles in ECS (Important Distinction)

This confuses many people:

```text
Execution Role:
  WHO: ECS agent (not your app)
  WHAT: pull container image from ECR, push logs to CloudWatch, read secrets
  WHEN: before and during task startup

Task Role:
  WHO: your application code running inside the container
  WHAT: call S3, SQS, DynamoDB, whatever your app needs
  WHEN: while the app is running
```

Real-life analogy:

```text
Execution Role = the delivery person who brings supplies to your office
Task Role      = you, the employee, doing your actual job

The delivery person needs access to the loading dock.
You need access to the office floor.
They are different permissions for different purposes.
```

5.7 Common Mistake: Over-Permissive Roles

```text
BAD:
{
  "Effect": "Allow",
  "Action": "*",
  "Resource": "*"
}

This gives your app permission to do ANYTHING in your AWS account.
Delete databases, terminate instances, create new users.

GOOD:
Scope every permission to the exact action and resource needed.
```

---

6. Who Can Reach My App - Network Security

This was covered in detail in the networking guide. Here is the security-focused summary.

6.1 The Layered Model

```text
Internet
    v
```

```
ALB (public subnet, SG: allow 443 from internet)
    v
Spring Boot (private subnet, SG: allow 8080 from ALB only)
    v
RDS (private subnet, SG: allow 5432 from backend only)
```

Each layer can only be reached by the layer above it.

6.2 Security Groups as the Primary Guard

```
[text]
ALB Security Group:
  Inbound:  TCP 443 from 0.0.0.0/0

Backend Security Group:
  Inbound:  TCP 8080 from ALB Security Group only

Database Security Group:
  Inbound:  TCP 5432 from Backend Security Group only

Redis Security Group:
  Inbound:  TCP 6379 from Backend Security Group only
```

Rule: always reference security groups, not IPs. IPs change. Security groups don't.

6.3 Private Subnets as the Foundation

Your backend and database should never be in a public subnet.

```
[text]
Public subnet  = has a route to the Internet Gateway = reachable from internet
Private subnet = no route to IGW = unreachable from internet directly
```

If your database is in a public subnet with a public IP, it is one misconfigured security group away from exposure.

6.4 No SSH in Production

Old pattern: SSH into servers to debug.

Better pattern:

```
[text]
Use AWS Systems Manager (SSM) Session Manager.
  - no SSH port open
  - no key pair management
  - audit trail of who connected
  - works through IAM, not network rules
```

---

7. Where Secrets Live - Secrets Manager and Parameter Store

7.1 The Problem

Your Spring Boot app needs secrets:

- database password
- JWT signing key
- API keys for Stripe, Twilio, SendGrid
- OAuth client secrets

On your laptop:

```
[yaml]
# application.yml - this is fine locally, dangerous in production
spring:
```

```
    datasource:
      password: postgres
  app:
    jwt-secret: my-dev-secret
    stripe-key: sk_test_xxxxxxxxxxxx
```

If this file ends up in a Docker image, a Git repo, or a log, those secrets are compromised.

7.2 Real-Life Analogy

```
[text]
 Secrets Manager is like a locked safe in the office.

 You do not tape the office door code to the front door.
 You store it in a safe. Only people with the right badge (IAM role)
 can open the safe and read the code.
```

7.3 How It Works for Your App

Step 1: Store the secret in Secrets Manager

```
[text]
 Secret name: myapp/prod/db-password
 Secret value: s3cureP@ssw0rd!2026
```

Step 2: Give your app permission to read it

IAM policy on the task/instance role:

```
[json]
 {
   "Effect": "Allow",
   "Action": "secretsmanager:GetSecretValue",
   "Resource": "arn:aws:secretsmanager:us-east-1:123456789012:secret:myapp/prod/*"
 }
```

Step 3: Your app reads the secret at startup

Option A - Spring Boot reads it via AWS SDK at startup:

```
[java]
 // In a @Configuration class or custom EnvironmentPostProcessor
 SecretsManagerClient client = SecretsManagerClient.create();
 GetSecretValueResponse response = client.getSecretValue(
     GetSecretValueRequest.builder()
         .secretId("myapp/prod/db-password")
         .build()
 );
 String dbPassword = response.secretString();
```

Option B - ECS injects it as an environment variable (simpler):

```
[json]
 {
   "secrets": [
     {
       "name": "DB_PASSWORD",
       "valueFrom": "arn:aws:secretsmanager:us-east-1:123456789012:secret:myapp/prod/db-
 password"
     }
   ]
 }
```

The container sees DBPASSWORD as an environment variable. Spring Boot picks it up via ${DBPASSWORD}. No code changes needed.

Option C - Kubernetes External Secrets (for EKS):

An operator syncs Secrets Manager values into Kubernetes Secrets, which are mounted as environment variables or volumes.

7.4 Secrets Manager vs Parameter Store

```
[text]
 Use Secrets Manager when:
   -> secrets need automatic rotation (e.g., RDS passwords)
   -> compliance requires managed secret lifecycle
   -> you want built-in rotation for supported services

 Use Parameter Store when:
   -> storing non-sensitive config (feature flags, endpoints)
   -> budget matters (free tier is generous)
   -> simple key-value config for your application
   -> secrets that do not need automatic rotation

 Both encrypt with KMS. Both integrate with IAM.
```

7.5 What Should NEVER Be in Your Code or Docker Image

```
[text]
 [x] Database passwords
 [x] JWT signing keys
 [x] API keys (Stripe, Twilio, etc.)
 [x] OAuth client secrets
 [x] AWS access keys
 [x] TLS private keys

 All of these belong in Secrets Manager or Parameter Store,
 retrieved at runtime via IAM role permissions.
```

---

8. How Data Is Protected - Encryption

8.1 Two Types of Encryption

```
[text]
 Encryption at rest:
   Data is encrypted when stored on disk.
   Even if someone steals the physical disk, they cannot read the data.

 Encryption in transit:
   Data is encrypted while moving over the network.
   Even if someone intercepts the traffic, they cannot read it.
```

You need both.

8.2 Real-Life Analogy

```
[text]
 Encryption at rest = locking your documents in a filing cabinet
 Encryption in transit = putting your letter in a sealed envelope before mailing it

 Without at-rest encryption:
   someone who breaks into the office reads your files

 Without in-transit encryption:
   someone who intercepts the mail reads your letters
```

8.3 Encryption at Rest for Your App

S3

```text
Default: S3 encrypts all new objects automatically (SSE-S3).
Better: use SSE-KMS with a customer managed key if you need audit and key control.
```

RDS

```text
Enable encryption when creating the instance.
Cannot be enabled after creation on an unencrypted instance.
So always enable it from the start.
```

EBS

```text
Enable encryption on volumes.
Use default AWS-managed key or a customer managed KMS key.
```

ElastiCache

```text
Redis supports at-rest encryption. Enable it.
```

DynamoDB

```text
Encrypted by default with an AWS-owned key.
Can use customer managed KMS key for more control.
```

Rule: enable encryption at rest everywhere. There is almost no reason not to.

8.4 Encryption in Transit for Your App

```text
User -> ALB:
  HTTPS (TLS certificate from ACM, free)

ALB -> Backend:
  HTTP is common (private network)
  HTTPS is better if compliance requires end-to-end encryption

Backend -> RDS:
  TLS can be enforced in RDS parameter group
  Use sslmode=require in JDBC connection string

Backend -> ElastiCache:
  Redis supports in-transit encryption. Enable it.

Backend -> S3:
  HTTPS by default via AWS SDK
```

8.5 KMS - The Key Manager

KMS manages the encryption keys used by all the services above.

You usually do not interact with KMS directly. But you should know:

```text
AWS-managed key:
  AWS creates and manages it for you per service.
  You cannot control rotation or access beyond defaults.
  Good enough for many workloads.
```

```
Customer-managed key:
  You create the key in KMS.
  You control the key policy (who can encrypt/decrypt).
  You control rotation schedule.
  Better for regulated or compliance-heavy workloads.
```

For a normal app:

- AWS-managed keys are fine to start
- customer-managed keys when compliance or audit requirements demand it

8.6 Envelope Encryption - How It Actually Works

This sounds complex but is simple:

```
[text]
 1. AWS generates a short-lived data key
 2. Your data is encrypted with the data key (fast, local)
 3. The data key itself is encrypted with the KMS master key
 4. Both the encrypted data and encrypted data key are stored

 To decrypt:
 1. Send the encrypted data key to KMS
 2. KMS decrypts it using the master key (KMS never sees your data)
 3. Use the decrypted data key to decrypt your data locally
```

Why?

- encrypting large data directly with KMS would be slow and expensive
- envelope encryption keeps large data local while keeping key security centralized

---

9. Who Your Users Are - Authentication

Everything above secures your infrastructure. This section secures your application layer.

9.1 The Problem

Your app has real users:

- customers
- admins
- internal staff
- maybe API consumers

You need to know:

- is this user who they claim to be? (authentication)
- is this user allowed to do what they are requesting? (authorization)

9.2 Real-Life Analogy

```
[text]
 Authentication = checking someone’s ID at the door
   "Are you really John Smith? Show me your driver’s license."

 Authorization = checking if they have permission
   "Okay, you are John Smith. But are you allowed in the VIP room?"
```

9.3 How Authentication Works for Your App

The Common Pattern: JWT

```
[text]
 1. User sends username + password to login endpoint
 2. Backend validates credentials against user store
 3. Backend creates a JWT token signed with a secret key
```

```
4. Browser stores the token
5. Every subsequent request includes the token in the Authorization header
6. Backend verifies the token signature and extracts user identity
```

Where the User Store Lives

Option A - Your own database:

```text
You store users in RDS.
You handle password hashing (bcrypt), login, token creation.
Full control, more code to maintain.
```

Option B - AWS Cognito:

```text
Cognito handles:
  - user registration
  - email verification
  - password policy
  - login
  - JWT token issuance
  - MFA
  - social login (Google, Facebook, etc.)

Your Spring Boot backend just validates the JWT token that Cognito issues.
You do not store passwords. You do not handle email verification flows.
```

Option C - External identity provider:

```text
Auth0, Okta, Firebase Auth, or corporate SSO.
Your backend validates their tokens.
```

9.4 How Spring Boot Validates JWT

Regardless of who issues the token, Spring Boot validates it:

```java
// Spring Security with JWT validation
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/public/**").permitAll()
                .requestMatchers("/api/admin/**").hasRole("ADMIN")
                .anyRequest().authenticated()
            )
            .oauth2ResourceServer(oauth2 -> oauth2
                .jwt(Customizer.withDefaults())    // validates JWT signature
            );
        return http.build();
    }
}
```

```yaml
# application.yml - point to token issuer
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
```

```
          issuer-uri: https://cognito-idp.us-east-1.amazonaws.com/us-east-1_XXXXXXX
```

Spring Boot:

1. receives request with Authorization: Bearer <token>
2. fetches Cognito's public key (JWKS)
3. verifies the token signature and expiration
4. extracts user identity and roles from token claims
5. applies authorization rules

No password handling in your backend. The identity provider does that.

9.5 Cognito in Story Mode

```
[text]
 Before Cognito:
   You build: registration page, email verification, password reset,
   MFA, token issuance, token refresh, social login integration.
   That is weeks of work and ongoing maintenance.

 With Cognito:
   You configure a User Pool.
   Users sign up and log in through Cognito.
   Cognito issues JWT tokens.
   Your backend validates tokens.
   You focus on business logic.
```

When Cognito is NOT the right fit:

- you need very custom authentication flows
- pricing does not work at your scale
- you already have a corporate identity provider (use that instead)
- you want zero vendor lock-in on identity

9.6 IAM vs Cognito - They Solve Different Problems

```
[text]
 IAM:
   Controls what AWS resources (S3, RDS, SQS) your services can access.
   Used by: your backend code, CI/CD pipelines, AWS services.
   Not for: end users logging into your web app.

 Cognito:
   Controls who your application users are and gives them tokens.
   Used by: end users of your web/mobile app.
   Not for: controlling which S3 bucket your backend can read.
```

This is the single most common confusion in AWS security interviews.

---

10. What Your Users Can Do - Authorization

Authentication tells you WHO the user is. Authorization tells you WHAT they can do.

10.1 Where Authorization Happens

```
[text]
 Layer 1: API Gateway or ALB
   Can block requests before they reach your backend.
   Example: only allow requests with valid JWT.

 Layer 2: Spring Boot Security
   Checks roles, scopes, and permissions in the token.
   Example: /api/admin/* requires ADMIN role.
```

```
Layer 3: Business Logic
  Application-level rules.
  Example: user can only edit their own orders, not other users' orders.
```

10.2 Example Authorization Flow

```
[text]
 1. User sends: GET /api/orders/123
    Header: Authorization: Bearer <jwt-token>

 2. Spring Security extracts token.
    Token contains: { "sub": "user-42", "roles": ["USER"] }

 3. Security filter: is this user authenticated? Yes.

 4. Controller receives request.
    Business logic: does user-42 own order 123?
    Query: SELECT * FROM orders WHERE id = 123 AND user_id = 'user-42'

 5. If yes -> return order.
    If no -> return 403 Forbidden.
```

This is defense in depth:

- Spring Security handles identity verification
- business logic handles data-level access control

10.3 Role-Based vs Attribute-Based

```
[text]
Role-Based (RBAC):
  "ADMIN can do everything. USER can only read their own data."
  Simple. Works for most apps.

Attribute-Based (ABAC):
  "User can access resource if user.department == resource.department
   AND user.clearance >= resource.sensitivityLevel"
  More flexible. More complex. Used in enterprise/compliance scenarios.
```

For most Spring Boot apps, RBAC is enough.

---

11. Putting It All Together for a Real App

Let us map the complete security posture for your React + Spring Boot + PostgreSQL app on AWS.

11.1 The Full Security Stack

```
[text]

                      YOUR APPLICATION

  LAYER 4: APPLICATION AUTH

   Cognito issues JWT tokens to users
   Spring Security validates tokens on every request
   Business logic enforces data-level permissions


  LAYER 3: SECRETS + ENCRYPTION
```

```
   DB password in Secrets Manager (auto-rotated)
   JWT secret in Secrets Manager
   API keys in Secrets Manager
   RDS encrypted at rest (KMS)
   S3 encrypted at rest (SSE-S3 or SSE-KMS)
   All traffic TLS-encrypted (ACM cert on ALB)


  LAYER 2: NETWORK SECURITY

   ALB in public subnet (only HTTPS from internet)
   Backend in private subnet (only from ALB)
   RDS in private subnet (only from backend)
   Security groups enforce every connection


  LAYER 1: IAM

   Backend has scoped IAM role (S3, Secrets, SQS only)
   No static AWS credentials anywhere
   Execution role for image pull + logging
   Task/instance role for application AWS calls
```

11.2 Request Flow With All Security Layers Active

```
[text]
 1. User opens React app (served from CloudFront + S3)

 2. User logs in
    -> React sends credentials to Cognito
    -> Cognito validates, returns JWT access token + refresh token
    -> React stores tokens in memory (not localStorage for sensitive apps)

 3. User requests their orders
    -> React sends: GET /api/orders
    -> Header: Authorization: Bearer <jwt>

 4. Request hits ALB
    -> ALB is in public subnet
    -> ALB terminates TLS (ACM certificate)
    -> ALB forwards to healthy backend in private subnet

 5. Spring Boot receives request
    -> Spring Security validates JWT (checks signature, expiry, issuer)
    -> Extracts user identity from token claims
    -> Controller calls service layer

 6. Service layer queries RDS
    -> Backend connects to RDS using credentials from Secrets Manager
    -> Connection uses TLS (sslmode=require)
    -> Security group allows backend -> RDS on port 5432

 7. Service layer reads profile image URL
    -> Backend generates pre-signed S3 URL for the user’s image
    -> This uses the IAM task role’s S3 permissions

 8. Response returns
    -> JSON with order data + signed image URL
    -> Backend -> ALB -> Internet -> Browser
```

```
    -> All encrypted in transit
```

Every layer participated. Nothing was bypassed.

---

12. Common Mistakes and How to Avoid Them

12.1 "We put the DB password in the Docker image"

```
[text]
 Problem:
   Anyone who pulls the image can extract the password.
   ECR images, CI/CD logs, or leaked layers expose it.

 Fix:
   Store in Secrets Manager.
   Inject via ECS secrets or read at app startup via SDK.
   The Docker image should contain zero secrets.
```

12.2 "We gave the backend AdministratorAccess"

```
[text]
 Problem:
   If the app is compromised, the attacker owns your entire AWS account.

 Fix:
   Scope IAM role to exact actions and resources needed.
   s3:GetObject on one bucket = s3:* on *.
```

12.3 "We put RDS in a public subnet"

```
[text]
 Problem:
   One security group mistake = database exposed to the internet.

 Fix:
   RDS in private subnet. Always.
   Security group allows only backend security group on port 5432.
```

12.4 "We hardcoded AWS access keys in the app"

```
[text]
 Problem:
   Keys in code get committed to Git.
   Keys in environment files get logged.
   Rotation becomes manual and error-prone.

 Fix:
   Use IAM roles. The AWS SDK finds credentials automatically.
   Zero access keys in application code, config, or Docker images.
```

12.5 "We disabled HTTPS because it was hard to set up"

```
[text]
 Problem:
   All traffic including login credentials transmitted in cleartext.

 Fix:
   ACM provides free TLS certificates.
   ALB handles TLS termination with one configuration.
   There is no valid reason to skip HTTPS in production.
```

12.6 "We store JWT tokens in localStorage"

```
[text]
 Problem:
   XSS attacks can steal tokens from localStorage.

 Fix:
   For sensitive apps, use httpOnly cookies or keep tokens in memory.
   Short-lived access tokens + refresh tokens reduce exposure.
```

12.7 "We log everything including secrets"

```
[text]
 Problem:
   Database passwords, tokens, and API keys appear in CloudWatch logs.

 Fix:
   Never log secret values.
   Mask or redact sensitive fields in log output.
   Review what your framework logs at DEBUG level before enabling it in prod.
```

---

13. Interview-Ready Answers

13.1 "How do you handle secrets in your application?"

```
[text]
 "I store all secrets - database credentials, API keys, JWT signing keys -
 in AWS Secrets Manager. The application reads them at startup using the
 AWS SDK, authorized by its IAM task role. No secrets are hardcoded, stored
 in environment files, or baked into Docker images. For ECS, I use the
 secrets field in the task definition so ECS injects them as environment
 variables from Secrets Manager directly."
```

13.2 "How do you implement least privilege?"

```
[text]
 "Every service gets its own IAM role scoped to exactly the actions and
 resources it needs. The Spring Boot backend might have
 s3:GetObject/PutObject on the uploads bucket and
 secretsmanager:GetSecretValue on its own secrets. No wildcard actions,
 no wildcard resources. On EKS, I use IRSA so each pod gets its own
 IAM identity. On ECS, each service gets its own task role."
```

13.3 "How does your app authenticate users?"

```
[text]
 "I use Cognito User Pools for user registration and login. Cognito issues
 JWT tokens. The React frontend includes the token in every API request.
 Spring Security validates the JWT signature using Cognito’s JWKS endpoint,
 checks expiration, and extracts user identity. Authorization is handled
 at the controller level with role-based access and at the service level
 with data ownership checks."
```

13.4 "How is data encrypted?"

```
[text]
 "Encryption at rest is enabled on RDS, S3, and EBS using KMS. In transit,
 the ALB terminates HTTPS using a free ACM certificate. Backend-to-RDS
 connections enforce TLS. S3 access via the SDK uses HTTPS by default.
```

```
For regulated workloads, I use customer-managed KMS keys for audit trail
and key rotation control."
```

13.5 "What is the difference between IAM and Cognito?"

```
[text]
"IAM controls which AWS services and resources your infrastructure and
applications can access. Cognito controls who your end users are and
issues them tokens. IAM is for machine-to-service authorization.
Cognito is for human-to-application authentication. They serve
completely different layers."
```

13.6 "Walk me through the security of a single API request"

```
[text]
"The request arrives at the ALB over HTTPS, so it is encrypted in transit.
The ALB is in a public subnet; the backend and database are in private
subnets. The ALB forwards to the backend, which validates the JWT token
issued by Cognito. The backend queries RDS over a TLS connection, using
credentials from Secrets Manager that it has permission to access via
its IAM task role. The response returns through the same encrypted path.
Network access is enforced by security groups at every layer, and IAM
ensures the backend can only touch the specific AWS resources it needs."
```

---

14. Quick Revision Sheet

The Four Security Layers

```
[text]
IAM              = what can my code touch in AWS?
Network          = what traffic can flow where?
Secrets + Crypto = how are credentials and data protected?
App Auth         = who are my users and what can they do?
```

One-Line Mapping

```
[text]
IAM Role            = badge that lets your app call AWS services
Security Group      = bouncer at each resource’s door
Secrets Manager     = locked safe for passwords and API keys
Parameter Store     = config shelf for non-sensitive settings
KMS                 = key manager for encryption
ACM                 = free TLS certificates
Cognito             = managed user login and JWT tokens
JWT                 = signed token proving who the user is
Spring Security     = validates tokens and enforces access rules
```

What Goes Where

```
[text]
DB password          -> Secrets Manager
JWT signing key      -> Secrets Manager
Stripe API key       -> Secrets Manager
Feature flags        -> Parameter Store
API base URLs        -> Parameter Store or ConfigMap
User identities      -> Cognito or your own DB
Access control       -> Spring Security + business logic
AWS permissions      -> IAM roles (never static keys)
TLS certificates     -> ACM (free, auto-renewed)
Encryption keys      -> KMS
```

Gold Standard Sentence

```
[text]
 "I secure my AWS application in four layers: IAM roles for least-privilege
 AWS access with no static credentials, private subnets and security groups
 for network isolation, Secrets Manager for credentials and KMS for
 encryption at rest and in transit, and Cognito or JWT-based authentication
 with Spring Security for user identity and authorization. Each layer is
 independent - compromising one does not automatically compromise the others."
```

Part 09: AWS Messaging, Integration, and Observability Through Story Mode

Source file: AWS-09-Messaging-Integration-Observability-Story-Mode.md

> Your Spring Boot app runs fine with synchronous REST calls. Then traffic grows, processes take longer, things fail independently, and you realize you need queues, events, workflows, and monitoring. This guide explains SQS, SNS, EventBridge, Lambda, Step Functions, CloudWatch, and the supporting cast - starting from real problems you will actually face.

---

1. How It Works on Your Laptop

On your laptop, your app is simple and synchronous:

```
[text]
 User clicks "Place Order"
        |
        v
 React calls POST /api/orders
        |
        v
 Spring Boot does EVERYTHING in one request:
   1. validate order
   2. charge payment
   3. reserve inventory
   4. send confirmation email
   5. update analytics
   6. return 200 OK
```

Total time: maybe 3 seconds.

Why does this work?

- only one user (you)
- if the email service is slow, you just wait
- if something fails, you restart and try again
- no pressure, no SLAs, no real traffic

That is the baseline. Now let us see what breaks.

---

2. What Changes When You Move to AWS

Real production brings real problems:

```
[text]
```

```
Problem 1: The payment gateway takes 2 seconds. The email provider takes 3 seconds.
           Total response time: 5+ seconds. Users see a spinner.

Problem 2: The email service is down for 10 minutes.
           Every order fails because the email step throws an exception.
           Users cannot buy anything because of an email problem.

Problem 3: You need to add analytics tracking.
           Now every order request does even more work.
           Adding features makes the order endpoint slower.

Problem 4: Traffic spikes. 1000 orders per second during a sale.
           The payment gateway rate-limits you.
           Orders start failing.

Problem 5: An order partially completes.
           Payment charged, but inventory update fails.
           You have no clear way to roll back or retry.
```

All of these are solved by the same family of ideas:

- do not do everything in one synchronous call
- separate the "acknowledge the order" step from all the downstream processing
- let each downstream concern handle itself independently
- watch everything so you know when things go wrong

---

3. The Big Mental Model: Sync vs Async vs Event-Driven

3.1 Synchronous (What You Have Now)

```
[text]
Caller waits for the response.

Request -> Service -> does everything -> Response

Good for:
  simple read operations
  operations where the user needs the result immediately

Bad for:
  anything involving slow downstream calls
  anything where one failure should not block the whole flow
```

3.2 Asynchronous with Queues

```
[text]
Caller sends a message and moves on. Consumer processes it later.

Producer -> Queue -> Consumer

Good for:
  background jobs
  spike absorption
  decoupling services that do not need instant results
```

3.3 Event-Driven with Pub/Sub

```
[text]
Something happens. Multiple interested parties react independently.
```

```
Producer -> "order-created" event -> Listener A, Listener B, Listener C

Good for:
  fan-out to many consumers
  loosely coupled systems
  adding new listeners without changing the producer
```

3.4 Workflow Orchestration

```
[text]
A coordinator manages multi-step processes with branching and retries.

Step 1 -> Step 2 -> if success -> Step 3
                    if failure -> compensate

Good for:
  complex business processes
  saga patterns
  anything with conditional logic and error recovery
```

3.5 The Mental Map

```
[text]


   "I need to do something later"              -> SQS
   "I need to tell many listeners"             -> SNS
   "I need to route events by content"         -> EventBridge
   "I need a short function to react"          -> Lambda
   "I need a multi-step coordinated workflow"  -> Step Functions
   "I need to see what is happening"           -> CloudWatch
```

---

4. Story Mode: Your App Starts Breaking Under Real Traffic

Phase 1: "Orders are slow"

The order endpoint does everything synchronously. Response time is 5 seconds.

The team says: "We should not make users wait for email and analytics."

Fix: put email and analytics work on a queue. Respond to the user immediately after payment + inventory, then process the rest asynchronously.

That is when SQS enters the story.

Phase 2: "Adding features to orders is painful"

Every new downstream concern (analytics, notifications, fraud check) requires changing the order service.

The team says: "The order service should just publish an event. Other services should listen."

Fix: publish an "order-created" event. Subscribers handle their own work independently.

That is when SNS or EventBridge enters the story.

Phase 3: "We need a background function for image resizing"

Users upload profile images. The app needs to resize them. Running a whole server for this is overkill.

Fix: trigger a short function when a file lands in S3.

That is when Lambda enters the story.

Phase 4: "Our order fulfillment has 7 steps and things fail in the middle"

The flow is: validate -> charge -> reserve -> notify -> ship -> email -> update-status.

If step 4 fails, you need to retry step 4, not re-run everything. If step 3 fails after step 2 succeeded, you need to refund the charge.

Fix: use a workflow coordinator that manages steps, retries, and compensation.

That is when Step Functions enters the story.

Phase 5: "Something is wrong but we don't know what"

Orders are failing. But which service? What error? Since when?

Fix: metrics, logs, alarms, tracing.

That is when CloudWatch and friends enter the story.

---

5. SQS - The Queue That Decouples Everything

5.1 Real-Life Analogy

```
[text]
 SQS is like a restaurant kitchen ticket system.

 The waiter (producer) puts an order ticket on the rail.
 The cook (consumer) picks it up when ready.
 The waiter does not wait for the food to be cooked.

 If the kitchen is backed up, tickets pile up on the rail.
 Nobody loses an order. The waiter can keep taking new orders.
```

5.2 How It Solves Your Problem

Before SQS:

```
[text]
 POST /api/orders
   -> validate
   -> charge payment
   -> reserve inventory
   -> send email         <- slow, can fail
   -> update analytics   <- slow, can fail
   -> return 200 OK      <- user waits for ALL of this
```

After SQS:

```
[text]
 POST /api/orders
   -> validate
   -> charge payment
   -> reserve inventory
   -> put message on email queue       <- instant
   -> put message on analytics queue   <- instant
   -> return 200 OK                    <- fast response

 Later:
   email-consumer picks message -> sends email
   analytics-consumer picks message -> updates analytics
```

User gets fast response. Email is sent eventually. Analytics are processed independently.

5.3 Standard Queue vs FIFO Queue

```text
Standard Queue:
  - at-least-once delivery (message may arrive twice)
  - best-effort ordering (not strictly in order)
  - nearly unlimited throughput
  - use for: most background jobs, notifications, analytics

FIFO Queue:
  - exactly-once processing (within dedup window)
  - strict ordering (per message group)
  - 300 messages/sec (3000 with batching)
  - use for: financial transactions, anything where order matters
```

For most Spring Boot apps, Standard Queue is the default. Use FIFO only when order or deduplication is critical.

5.4 Key Concepts You Must Understand

Visibility Timeout

```text
Consumer picks up message -> message becomes invisible to other consumers.
If consumer finishes processing -> it deletes the message.
If consumer crashes -> after visibility timeout, message reappears for another consumer.

Default: 30 seconds.
Set it higher than your expected processing time.
```

Dead-Letter Queue (DLQ)

```text
If a message fails processing many times (maxReceiveCount), it moves to the DLQ.

Why this matters:
  Without DLQ: a poison message blocks the queue forever.
  With DLQ: bad messages are parked. The queue keeps flowing.
  You monitor the DLQ and investigate failures.
```

Real-life analogy:

```text
DLQ is like the "undeliverable mail" pile at the post office.
Letters that cannot be delivered after several attempts get parked there.
Someone reviews them periodically.
```

Idempotency - The Consumer Must Handle Duplicates

```text
Standard SQS delivers at-least-once.
That means: your consumer MIGHT receive the same message twice.

If your consumer is "charge the credit card":
  Without idempotency: customer charged twice.
  With idempotency: consumer checks "did I already process order 123?" -> skips duplicate.

How to implement:
  Store a processed message ID in DB before processing.
  Check it before executing business logic.
```

5.5 Spring Boot Consuming from SQS

Using Spring Cloud AWS or AWS SDK:

```
[java]
@SqsListener("order-email-queue")
public void handleEmailMessage(OrderEvent event) {
    // Check idempotency
    if (alreadyProcessed(event.getOrderId())) {
        return;
    }

    // Process
    emailService.sendOrderConfirmation(event);

    // Mark processed
    markProcessed(event.getOrderId());
}
```

Spring Boot backend publishes to SQS:

```
[java]
@Service
public class OrderService {

    private final SqsTemplate sqsTemplate;

    public Order placeOrder(OrderRequest request) {
        // 1. Validate
        // 2. Charge payment
        // 3. Reserve inventory
        Order order = orderRepository.save(newOrder);

        // 4. Queue async work
        sqsTemplate.send("order-email-queue", new OrderEvent(order.getId()));
        sqsTemplate.send("order-analytics-queue", new OrderEvent(order.getId()));

        // 5. Return fast
        return order;
    }
}
```

---

6. SNS - Broadcasting to Many Listeners

6.1 Real-Life Analogy

```
[text]
SNS is like a PA system in an office.

Someone makes an announcement: "All-hands meeting at 3 PM."
Every department hears it simultaneously.
Each department decides what to do with the information independently.

The announcer does not know or care how many departments are listening.
```

6.2 How SNS Differs from SQS

```
[text]
SQS = one message, one consumer picks it up
SNS = one message, many subscribers receive copies

SQS:  Producer -> Queue -> one Consumer
SNS:  Producer -> Topic -> Subscriber A, Subscriber B, Subscriber C
```

6.3 When to Use SNS Alone

```
[text]
 - push notifications to mobile devices
 - send email/SMS alerts
 - simple broadcast where you do not need per-subscriber queuing
```

6.4 Why SNS Alone Is Often Not Enough

```
[text]
 Problem:
   If Subscriber A is down when the message arrives, it misses it.
   SNS is push-based. If the target is not ready, the message may be lost.

 Solution:
   Combine SNS with SQS. Each subscriber gets its own queue.
```

That leads to the most important pattern in AWS messaging.

---

7. SNS + SQS - The Fan-Out Pattern

This is the single most important messaging pattern in AWS.

7.1 The Pattern

```
[text]
 Order Service
       |
       | publishes "order-created" event
       v
 SNS Topic: order-events
       |
       -> SQS Queue: email-queue        -> Email Consumer
       -> SQS Queue: inventory-queue     -> Inventory Consumer
       -> SQS Queue: analytics-queue     -> Analytics Consumer
       -> SQS Queue: fraud-queue         -> Fraud Consumer
```

7.2 Why This Is Powerful

```
[text]
 1. The order service publishes ONCE.
    It does not know or care how many listeners exist.

 2. Each consumer has its OWN queue.
    If the email consumer is slow, analytics is unaffected.
    If the fraud consumer crashes, email still works.

 3. Each consumer retries INDEPENDENTLY.
    Email fails three times -> goes to email DLQ.
    Inventory succeeds immediately.
    No coupling between them.

 4. Adding a new listener is trivial.
    Want to add "loyalty points" processing?
    Create a new SQS queue. Subscribe it to the SNS topic.
    The order service changes NOTHING.
```

7.3 Real-Life Analogy

```
[text]
 SNS + SQS is like a newspaper delivery system.
```

```
The printing press (producer) prints the newspaper once.
Each subscriber (consumer) gets their own copy in their own mailbox (queue).

If Subscriber A is on vacation, their mailbox fills up. Other subscribers are unaffected.
If there are no subscribers, the newspaper is still printed. Nobody gets angry.
```

7.4 Before and After

```text
BEFORE (direct calls):
  Order Service -> calls Email Service directly
  Order Service -> calls Inventory Service directly
  Order Service -> calls Analytics Service directly
  Order Service -> calls Fraud Service directly

  Problems:
    - order service knows about all downstream services
    - one failure blocks or delays the order
    - adding a consumer requires changing the order service

AFTER (SNS + SQS):
  Order Service -> publishes to SNS topic
  Each consumer -> reads from its own SQS queue

  Benefits:
    - order service is decoupled
    - failures are isolated
    - adding consumers is configuration, not code change
```

---

8. EventBridge - The Smart Event Router

8.1 Real-Life Analogy

```text
EventBridge is like a smart mail sorting machine.

Letters arrive at the sorting center.
Each letter has details on the envelope (event content).
The machine reads the details and routes each letter to the right department.

"This letter mentions 'high-value order' -> send to VIP team"
"This letter mentions 'refund request' -> send to finance"
"This letter mentions 'new signup' -> send to marketing AND analytics"
```

8.2 How It Differs from SNS

```text
SNS:
  "Send this message to everyone subscribed to this topic."
  Simple, broad fan-out.

EventBridge:
  "Look at the content of this event. Route it based on rules."
  Smarter, content-based routing.
```

Example:

```json
// Event published to EventBridge
```

```
{
  "source": "com.myapp.orders",
  "detail-type": "OrderCreated",
  "detail": {
    "order_id": "123",
    "amount": 5000,
    "type": "premium"
  }
}
```

```
[text]
Rule 1: If detail-type = "OrderCreated" AND detail.type = "premium"
        -> Send to VIP notification Lambda

Rule 2: If detail-type = "OrderCreated"
        -> Send to general analytics SQS queue

Rule 3: If detail-type = "OrderCreated" AND detail.amount > 10000
        -> Send to fraud detection Step Function
```

The producer publishes one event. EventBridge routes it to different targets based on rules.

8.3 When to Use EventBridge vs SNS + SQS

```
[text]
Use SNS + SQS when:
  -> simple fan-out to known subscribers
  -> each subscriber always gets every message
  -> you want the simplest possible setup

Use EventBridge when:
  -> you need content-based routing (different events go to different targets)
  -> AWS service events are part of the flow (EC2 state changes, S3 events, etc.)
  -> you have many event types and many consumers with different interests
  -> you want schema registry and event discovery
```

For your first app, SNS + SQS is usually enough. EventBridge becomes valuable as the system grows and event routing becomes complex.

8.4 EventBridge and AWS Service Events

One powerful feature: AWS services emit events to EventBridge automatically.

```
[text]
EC2 instance stopped         -> EventBridge rule -> Lambda sends Slack alert
S3 object created            -> EventBridge rule -> Step Function starts processing
ECS task failed              -> EventBridge rule -> SNS sends PagerDuty alert
CodePipeline deployment done -> EventBridge rule -> Lambda runs smoke tests
```

You do not write code to detect these events. AWS emits them. You write rules to react.

---

9. Lambda - The Glue That Runs Without Servers

9.1 Real-Life Analogy

```
[text]
Lambda is like a freelance worker.

You do not hire them full-time.
You call them when there is a specific job.
They do the job and leave.
```

```
You pay only for the time they worked.

No job? No cost.
```

9.2 Lambda Is Not a Replacement for Your Backend

This is a critical misunderstanding to avoid.

```
[text]
Lambda is NOT meant to replace your Spring Boot backend.

Lambda IS meant for:
  -> short-lived event-driven functions
  -> glue logic between services
  -> processing queue messages
  -> reacting to S3 uploads
  -> scheduled cron-like tasks
  -> lightweight API endpoints
```

Your Spring Boot backend handles business logic, API serving, and stateful processing.
Lambda handles the small reactive tasks around it.

9.3 Where Lambda Fits in Your Architecture

```
[text]
S3 upload triggers Lambda     -> resize image, scan for viruses
SQS message triggers Lambda   -> send notification email
EventBridge event triggers Lambda -> update analytics
CloudWatch alarm triggers Lambda  -> auto-remediate (restart service, alert on-call)
Scheduled rule triggers Lambda -> nightly report generation, cleanup
API Gateway triggers Lambda   -> lightweight serverless API (not for heavy Spring Boot
apps)
```

9.4 Lambda for Your Spring Boot App (Practical Examples)

Example 1: Image Processing

```
[text]
User uploads profile picture via pre-signed S3 URL
      v
S3 event notification
      v
Lambda function:
  - reads the uploaded image from S3
  - resizes to thumbnail (200x200)
  - saves thumbnail back to S3
  - updates metadata in RDS (or via API call to your backend)
```

Example 2: SQS Consumer for Email

```
[text]
Spring Boot puts message on SQS
      v
Lambda triggers on SQS message
      v
Lambda function:
  - reads order details from message
  - calls email API (SendGrid, SES)
  - if failure, SQS retries automatically
  - after max retries, message goes to DLQ
```

Example 3: Scheduled Cleanup

```
[text]
```

```
CloudWatch scheduled rule: every day at 2 AM
      v
Lambda function:
  - deletes expired sessions from DynamoDB
  - cleans up old temporary S3 objects
  - sends summary report to Slack
```

9.5 Lambda Limitations You Must Know

```
[text]
Max execution time:    15 minutes
Max memory:            10 GB
Cold start latency:    depends on runtime (JVM is worst, Node/Python is fast)
No persistent state:   each invocation is independent
Concurrency limits:    1000 default per account per region (can be increased)

For Java/Spring Boot as Lambda:
  Cold starts can be 5-15 seconds. Not acceptable for user-facing APIs.
  Use Lambda for background tasks, not for replacing your main API.
```

---

10. Step Functions - The Workflow Coordinator

10.1 Real-Life Analogy

```
[text]
Step Functions is like a project manager with a checklist.

"Step 1: Validate the order. [ok]"
"Step 2: Charge payment. [ok]"
"Step 3: Reserve inventory. [x] Failed!"
"Okay, retry step 3."
"Still failing? Run compensation: refund the payment from step 2."
"Send alert to operations team."

The project manager tracks where we are, what succeeded, what failed,
and what to do next. The individual workers just do their assigned tasks.
```

10.2 Why Not Just Chain Lambda Functions?

```
[text]
Without Step Functions:
  Lambda A calls Lambda B calls Lambda C.

  Problems:
    - if B fails after A succeeds, how does A know?
    - retry logic is scattered across each function
    - error handling is hidden in code
    - no visibility into where the workflow is
    - timeouts cascade unpredictably

With Step Functions:
  Coordinator says: run A, then B, then C.
  If B fails: retry 3 times, then run compensation D.
  If C times out: send alert and pause for manual review.

  Everything is explicit. Everything is visible. Everything is auditable.
```

10.3 When to Use Step Functions

```
[text]
```

```
Use Step Functions when:
  -> process has multiple steps with dependencies
  -> some steps can fail and need retries
  -> failure of one step requires compensating earlier steps (saga pattern)
  -> you need human approval gates
  -> you want visible workflow state and history

Do NOT use Step Functions for:
  -> simple one-step processing (just use Lambda or SQS consumer)
  -> real-time request/response (too slow for API responses)
  -> high-throughput event streaming (use Kinesis or SQS)
```

10.4 Practical Example: Order Fulfillment Saga

```
[text]
Start

  -> Validate Order (Lambda)

        Success -> Charge Payment (Lambda)

                      Success -> Reserve Inventory (Lambda)

                                    Success -> Send Confirmation (Lambda)

                                                  Done [ok]

                                    Failure -> Refund Payment (Lambda)
                                                 -> Notify Support

                      Failure -> Notify User: "Payment failed"

        Failure -> Reject Order
```

Each step is a separate Lambda or service call. Step Functions manages the flow.

10.5 The Saga Pattern in Plain Language

```
[text]
Saga = a multi-step process where each step has a compensating action.

Step 1: Charge payment     -> Compensate: Refund payment
Step 2: Reserve inventory  -> Compensate: Release inventory
Step 3: Send to shipping   -> Compensate: Cancel shipment

If step 3 fails:
  Run compensate for step 2 (release inventory)
  Run compensate for step 1 (refund payment)
  Notify user
```

This is the distributed-system alternative to database transactions that span multiple services.

---

11. CloudWatch - Seeing What Is Actually Happening

11.1 Real-Life Analogy

```
[text]
CloudWatch is like the security camera system plus the building’s sensor network.

Cameras (logs): record what happens in each room
```

```
Sensors (metrics): measure temperature, door opens, electricity usage
Alarms: "if temperature exceeds 100F, call the fire department"
Dashboard: a wall of monitors showing everything at once
```

11.2 The Three Pillars of Observability

```text
Metrics:  numbers over time
          "CPU is at 78%", "Request latency is 230ms", "Error rate is 2.3%"

Logs:     detailed records of events
          "Order 123 failed at payment step: timeout after 5000ms"

Traces:   end-to-end path of a request across services
          "This request took 450ms: 10ms in ALB, 200ms in backend, 240ms in RDS"
```

CloudWatch covers metrics and logs. For tracing, AWS X-Ray adds the third pillar.

11.3 Metrics You Should Watch for Your App

Infrastructure Metrics (CloudWatch gives these for free)

```text
EC2/ECS/EKS:
  - CPU utilization
  - Memory utilization (custom metric on EC2, native on Fargate)
  - Network in/out

ALB:
  - Request count
  - Target response time (latency)
  - HTTP 5xx count (server errors)
  - HTTP 4xx count (client errors)
  - Healthy host count

RDS:
  - CPU utilization
  - Database connections
  - Read/write IOPS
  - Free storage space
  - Replica lag (if using read replicas)

SQS:
  - ApproximateNumberOfMessagesVisible (queue depth)
  - ApproximateAgeOfOldestMessage (how long messages wait)
  - NumberOfMessagesSent / Received / Deleted

Lambda:
  - Invocations
  - Errors
  - Duration
  - Throttles
  - ConcurrentExecutions
```

Application Metrics (You emit these from your code)

```text
  - orders_created_total
  - payment_failures_total
  - api_request_duration_ms (by endpoint)
  - cache_hit_rate
  - external_api_latency_ms (by dependency)
```

Spring Boot Actuator + Micrometer can push these to CloudWatch.

11.4 CloudWatch Alarms - The Alert System

```
[text]
An alarm watches a metric and triggers an action when a threshold is crossed.

Example alarms for your app:

Alarm: "ALB 5xx errors > 10 in 5 minutes"
  -> Send SNS notification to on-call team

Alarm: "SQS queue depth > 1000 for 10 minutes"
  -> Scaling issue or consumer is down

Alarm: "RDS CPU > 85% for 15 minutes"
  -> Database needs attention (slow queries? scale up?)

Alarm: "Lambda errors > 5% of invocations"
  -> Something is wrong with the function
```

Good alarms are:

- actionable (someone can do something about it)
- not noisy (avoid alerting on things that self-resolve)
- tied to real user impact

11.5 CloudWatch Logs - What Happened and When

Your Spring Boot app writes logs. On AWS, those logs go to CloudWatch Logs.

```
[text]
On EC2:
  Install CloudWatch agent -> ships logs to CloudWatch

On ECS:
  Task definition logConfiguration with awslogs driver -> automatic

On EKS:
  Fluent Bit or CloudWatch agent DaemonSet -> ships container logs

On Lambda:
  Automatic. Every console.log or System.out goes to CloudWatch Logs.
```

CloudWatch Logs Insights - Querying Logs

```
[text]
You can query logs with SQL-like syntax:

# Find all errors in the last hour
fields @timestamp, @message
| filter @message like /ERROR/
| sort @timestamp desc
| limit 50

# Find slow API calls
fields @timestamp, @message
| filter @message like /duration/
| parse @message "duration=* ms" as duration
| filter duration > 1000
| sort duration desc
```

This is invaluable for post-incident debugging.

11.6 CloudWatch Dashboard - The War Room Screen

```
[text]
 You create a dashboard showing:

   Row 1: ALB request count, latency, error rate
   Row 2: ECS CPU, memory, running tasks
   Row 3: RDS CPU, connections, IOPS
   Row 4: SQS queue depth, message age
   Row 5: Lambda invocations, errors, duration
```

One glance tells you if the system is healthy.

---

12. The Supporting Cast

These services often appear alongside the main ones.

12.1 SES - Simple Email Service

```
[text]
 What: managed email sending service.

 Your app needs to send emails? Use SES instead of running your own mail server.

 How it fits:
   Order placed -> SQS message -> Lambda -> SES sends confirmation email
```

12.2 S3 Event Notifications

```
[text]
 What: S3 can emit events when objects are created, deleted, or modified.

 How it fits:
   User uploads file to S3
     -> S3 event triggers Lambda (resize image)
     -> or S3 event triggers SQS -> consumer processes the file

 This turns S3 from "dumb storage" into an event source.
```

12.3 CloudWatch Events / EventBridge Scheduled Rules

```
[text]
 What: cron jobs without servers.

 How it fits:
   "Every day at 2 AM, run a Lambda that cleans up expired sessions."
   "Every 5 minutes, run a Lambda that checks if all services are healthy."

 You do not need a dedicated EC2 instance running cron.
```

12.4 X-Ray - Distributed Tracing

```
[text]
 What: shows the path of a request across multiple services.

 How it fits:
   One user request -> ALB -> Spring Boot -> RDS + Redis + SQS + Lambda
   X-Ray shows which step was slow, which step failed, where time was spent.

   Without X-Ray: "Something is slow, but I don’t know where."
   With X-Ray:    "The RDS query in the order service took 800ms."
```

12.5 ACM - Certificate Manager

```
[text]
 What: free TLS certificates for your domains.

 How it fits:
   ALB needs HTTPS -> ACM provides the certificate.
   CloudFront needs HTTPS -> ACM provides the certificate.
   No manual certificate management. Auto-renewal.
```

12.6 WAF - Web Application Firewall

```
[text]
 What: protects your API from common web attacks.

 How it fits:
   Attach WAF to ALB or CloudFront.
   Rules block SQL injection, XSS, bad bots, rate abuse.

   Think of it as a security guard checking every HTTP request
   before it even reaches your application.
```

---

13. Putting It All Together: Order Processing Pipeline

Here is the complete architecture for order processing using everything covered:

```
[text]

   User clicks "Place Order"

   React -> ALB -> Spring Boot

                      validate order
                      charge payment (sync, user needs result)
                      save order in RDS
                      publish "order-created" to SNS topic
                      return 201 Created to user



                     SNS Topic: order-events



     SQS: email-q        SQS: inventory-q    SQS: analytics-q

     Lambda:              Spring Boot         Lambda:
     send email via SES   reserve inventory   update analytics

      (if fails ->         (if fails ->         (if fails ->
       DLQ + alarm)        DLQ + alarm)        DLQ + alarm)


   CloudWatch watches everything:

   Metrics: queue depth, lambda errors, API latency
   Logs: application errors, order IDs, processing times
   Alarms: queue depth > 500? DLQ has messages? 5xx spike?
   Dashboard: all of the above on one screen
```

What makes this good:

```
[text]
 1. User gets fast response (sync part is only validate + charge + save)
 2. Downstream work is decoupled (each consumer independent)
 3. Failures are isolated (email failure does not block inventory)
 4. Adding a new consumer is easy (subscribe new queue to topic)
 5. Everything is observable (CloudWatch metrics, logs, alarms)
 6. Failed messages are not lost (DLQ catches them)
```

---

14. Common Mistakes and Debugging Tips

14.1 "We do everything synchronously"

```
[text]
 Problem: one slow downstream call makes the entire API slow.
 Fix: move non-critical work to queues.
 Ask: "Does the user need this result right now?" If no -> queue it.
```

14.2 "Our SQS consumer is not idempotent"

```
[text]
 Problem: duplicate messages cause duplicate charges or emails.
 Fix: check if the message was already processed before executing business logic.
 Use: order_id as idempotency key, stored in DB.
```

14.3 "We have no DLQ"

```
[text]
 Problem: one bad message blocks the queue forever.
 Fix: configure DLQ with maxReceiveCount (e.g., 3).
 Monitor: alarm on DLQ message count > 0.
```

14.4 "We use Lambda for everything including our main API"

```
[text]
 Problem: Java Lambda cold starts are 5-15 seconds. Users see spinners.
 Fix: use Lambda for event-driven background tasks.
 Use ECS/EKS for the main Spring Boot API.
```

14.5 "We have no alarms"

```
[text]
 Problem: you find out about outages from users, not from monitoring.
 Fix: at minimum, alarm on ALB 5xx, SQS queue depth, DLQ messages, Lambda errors.
```

14.6 "Our SNS subscriber is down and messages are lost"

```
[text]
 Problem: SNS pushes to an endpoint that is down. Message is gone.
 Fix: never subscribe a direct HTTP endpoint to SNS for critical flows.
 Instead: SNS -> SQS -> consumer. SQS stores messages until consumer is ready.
```

14.7 Quick Debug Mental Model

```
[text]
 Message not arriving at consumer?
   1. Is the producer actually sending? (check CloudWatch metrics or logs)
   2. Is the SQS queue receiving? (check ApproximateNumberOfMessagesVisible)
```

```
  3. Is the consumer polling? (check consumer logs or Lambda invocations)
  4. Is the message going to DLQ? (check DLQ queue depth)
  5. Is the visibility timeout too short? (message reappears before processing finishes)

Something is slow but you don't know where?
  1. Check ALB target response time (is the backend slow?)
  2. Check RDS CPU and connections (is the DB the bottleneck?)
  3. Check SQS message age (is the queue backing up?)
  4. Enable X-Ray for request-level tracing
```

---

15. Interview-Ready Answers

15.1 "How do you decouple services in your architecture?"

```
[text]
 "I use the SNS + SQS fan-out pattern. The producing service publishes an
 event to an SNS topic. Each consuming service has its own SQS queue
 subscribed to the topic. This way, consumers are independent - they scale,
 retry, and fail without affecting each other. The producer is decoupled
 from knowing who consumes the event."
```

15.2 "When would you use EventBridge over SNS?"

```
[text]
 "I use SNS when the fan-out is simple - every subscriber gets every message.
 I use EventBridge when I need content-based routing - different events go
 to different targets based on rules that inspect the event body. EventBridge
 also integrates natively with AWS service events like EC2 state changes or
 S3 notifications, which makes it stronger for infrastructure-level eventing."
```

15.3 "Where does Lambda fit in your architecture?"

```
[text]
 "I use Lambda for event-driven glue tasks: processing SQS messages, reacting
 to S3 uploads, running scheduled cleanups, and lightweight integrations.
 I do not use Lambda for the main Spring Boot API because JVM cold starts
 are too slow for user-facing requests. The main API runs on ECS or EKS."
```

15.4 "How do you handle a multi-step process that can fail?"

```
[text]
 "I use Step Functions to orchestrate the workflow. Each step is a Lambda
 or service integration. Step Functions handles retries, error branching,
 and compensation logic. For example, in an order saga, if inventory
 reservation fails after payment is charged, Step Functions runs the refund
 step automatically. The state of the workflow is always visible and auditable."
```

15.5 "How do you monitor your system?"

```
[text]
 "I use CloudWatch for three things: metrics for system health (CPU, latency,
 error rate, queue depth), logs for debugging (application logs shipped from
 ECS or Lambda), and alarms for alerting (5xx spike, DLQ messages, saturated
 resources). For distributed tracing across microservices, I add X-Ray.
 I build a CloudWatch dashboard so the team can see the entire system's
 health at a glance."
```

15.6 "Why must SQS consumers be idempotent?"

```
[text]
```

```
"Standard SQS guarantees at-least-once delivery, meaning a message can
arrive more than once. If My consumer processes a payment and gets the
same message twice, it would charge the user twice without idempotency.
So I store a processed message ID before executing the business logic
and check it on every invocation. If already processed, I skip."
```

---

16. Quick Revision Sheet

One-Line Mapping

```text]
SQS            = durable message queue for async processing
SNS            = pub/sub fan-out to multiple subscribers
EventBridge    = smart event router with content-based rules
Lambda         = serverless function triggered by events
Step Functions = workflow orchestrator for multi-step processes
CloudWatch     = metrics, logs, alarms, dashboards
X-Ray          = distributed request tracing
SES            = managed email sending
WAF            = web application firewall at L7
ACM            = free TLS certificates
```

When to Use What

```text]
"I need to do something later"                -> SQS
"I need to tell many listeners at once"        -> SNS
"I need to route events by content"            -> EventBridge
"I need a short function to react to an event" -> Lambda
"I need a multi-step workflow with retries"    -> Step Functions
"I need to see what is happening"              -> CloudWatch
"I need to trace a request across services"    -> X-Ray
"I need to send email"                         -> SES
"I need to block attacks at the HTTP layer"    -> WAF
"I need TLS certificates"                      -> ACM
```

The Key Patterns in One Place

```text]
Async background work:        Producer -> SQS -> Consumer
Fan-out to many consumers:    Producer -> SNS -> SQS  N -> Consumers
Content-based event routing:  Producer -> EventBridge -> Rules -> Targets
File processing on upload:    S3 event -> Lambda
Scheduled cron job:           EventBridge rule -> Lambda
Multi-step saga:              Step Functions -> Lambdas with retry/compensate
Full observability:           CloudWatch metrics + logs + alarms + X-Ray traces
```

Gold Standard Sentence

```text]
"I keep my API fast by doing only critical work synchronously and offloading
the rest to SQS queues. I use SNS for fan-out when multiple services need
the same event, EventBridge when routing depends on event content, Lambda
for short reactive tasks, and Step Functions when the process has multiple
steps that can fail independently. CloudWatch gives me metrics, logs, and
alarms so I know what is happening, and X-Ray traces requests across service
boundaries."
```